

12-A1

优先级队列

概述：动机与接口

I cannot choose the best.

The best chooses me.

邓俊辉

deng@tsinghua.edu.cn

循优先级访问

❖ 应用举例

- 离散事件模拟
- 操作系统：任务调度/中断处理/MRU/...
- 输入法：词频调整

❖ 极值元素：须反复地、快速地定位

集合组成：可动态变化

元素优先级：可动态变化

❖ 作为底层数据结构所支持的高效操作

是很多高效算法的基础

- 内部、外部、在线排序
- 贪心算法：Huffman编码、Kruskal
- 平面扫描算法中的事件队列
- ...

优先级队列

```
❖ template <typename T> struct PQ { //priority queue

    virtual void insert( T ) = 0;

    virtual T delMax() = 0;

    virtual T& getMax() = 0;

};
```

❖ **栈和队列**，都是PQ的特例——优先级完全取决于元素的**插入次序**

最大栈和最大队列，也是PQ的特例——插入和删除的**位置受限**

12-A2

优先级队列

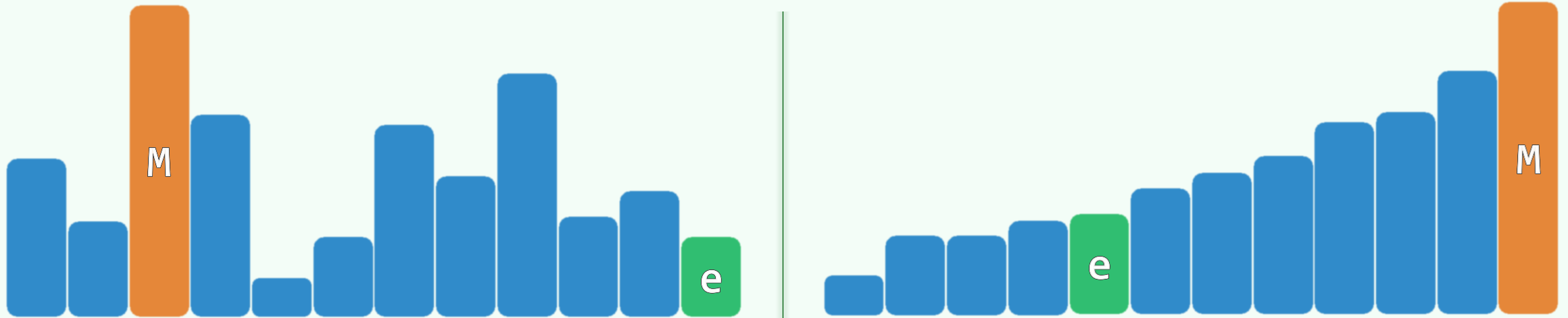
概述：基本实现

邓俊辉

deng@tsinghua.edu.cn

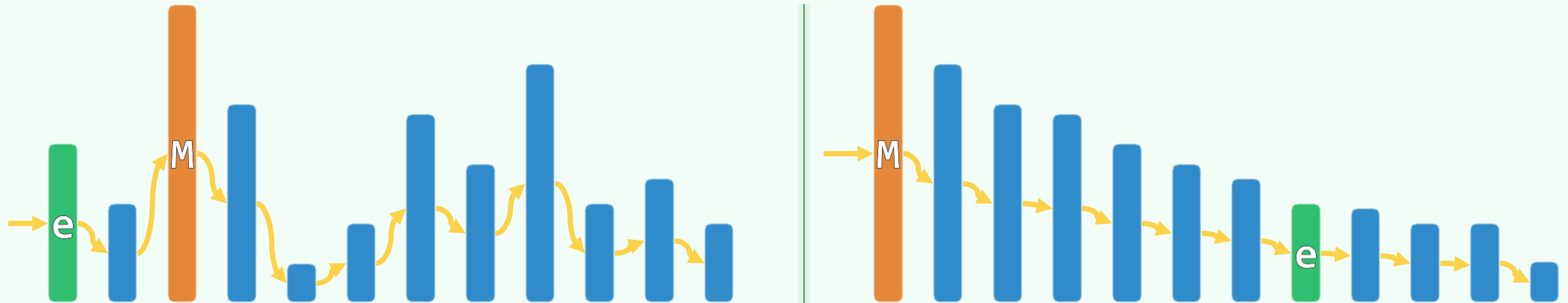
大兒孔文舉，小兒楊德祖；與子碌碌，莫足數也

Vector



Vector	getMax()	delMax()	insert()
Unsorted	traverse() $\Theta(n)$	remove(traverse()) $\Theta(n) + \mathcal{O}(n) = \Theta(n)$	insertLast(e) $\mathcal{O}(1)$
Sorted	$[n-1]$ $\mathcal{O}(1)$	remove(n-1) $\mathcal{O}(1)$	insert(1 + search(e), e) $\mathcal{O}(\log n) + \mathcal{O}(n) = \mathcal{O}(n)$

List



List	getMax()	delMax()	insert()
Unsorted	traverse() $\Theta(n)$	remove(traverse()) $\Theta(n) + \mathcal{O}(1) = \Theta(n)$	insertFirst(e) $\mathcal{O}(1)$
Sorted	first() $\mathcal{O}(1)$	remove(first()) $\mathcal{O}(1)$	insert(search(e), e) $\mathcal{O}(n) + \mathcal{O}(1) = \mathcal{O}(n)$

BBST

❖ AVL、Splay、Red-black: 三个接口均只需 $\mathcal{O}(\log n)$ 时间

但是, BBST的功能远远超出了PQ的需求:

$$\text{PQ} = \boxed{1 \times \text{insert}()} + \boxed{0.5 \times \text{search}()} + \boxed{0.5 \times \text{remove}()}$$

❖ 若只需查找极值元, 则不必维护所有元素之间的全序关系, 偏序足矣

❖ 因此有理由相信, 存在某种更为简单、维护成本更低的实现方式

使得各功能接口的时间复杂度依然为 $\mathcal{O}(\log n)$, 而且实际效率更高

❖ 当然, 就最坏情况而言, 这类实现方式已属最优——为什么?

统一测试

```
template <typename PQ> void testHeap( int n ) {  
    Rank* A = new Rank[ 2*n/3 ]; //创建容量为2n/3的数组, 并  
    for ( Rank i = 0; i < 2*n/3; i++ ) A[i] = dice( 3*n ); //随机化  
    PQ heap( A + n/6, n/3 ); delete [] A; //Robert Floyd  
    while ( heap.size() < n ) //随机测试  
        if ( dice( 100 ) < 70 ) heap.insert( dice( 3*n ) ); //70%概率插入  
        else if ( ! heap.empty() ) heap.delMax(); //30%概率删除  
    while ( ! heap.empty() ) heap.delMax(); //清空  
} //testHeap
```


12-B1

优先级队列

完全二叉堆：结构

邓俊辉

deng@tsinghua.edu.cn

何人將亂石作堆？如何亂石堆中有殺氣沖起？

结构性：逻辑上等同于完全二叉树，物理上直接借助向量实现

❖ 内部节点的最大秩

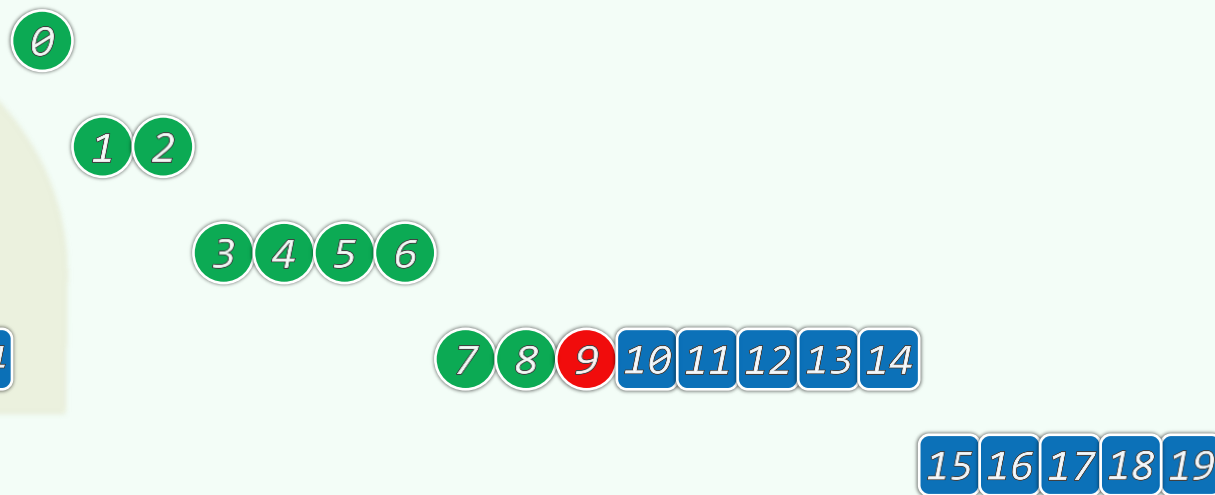
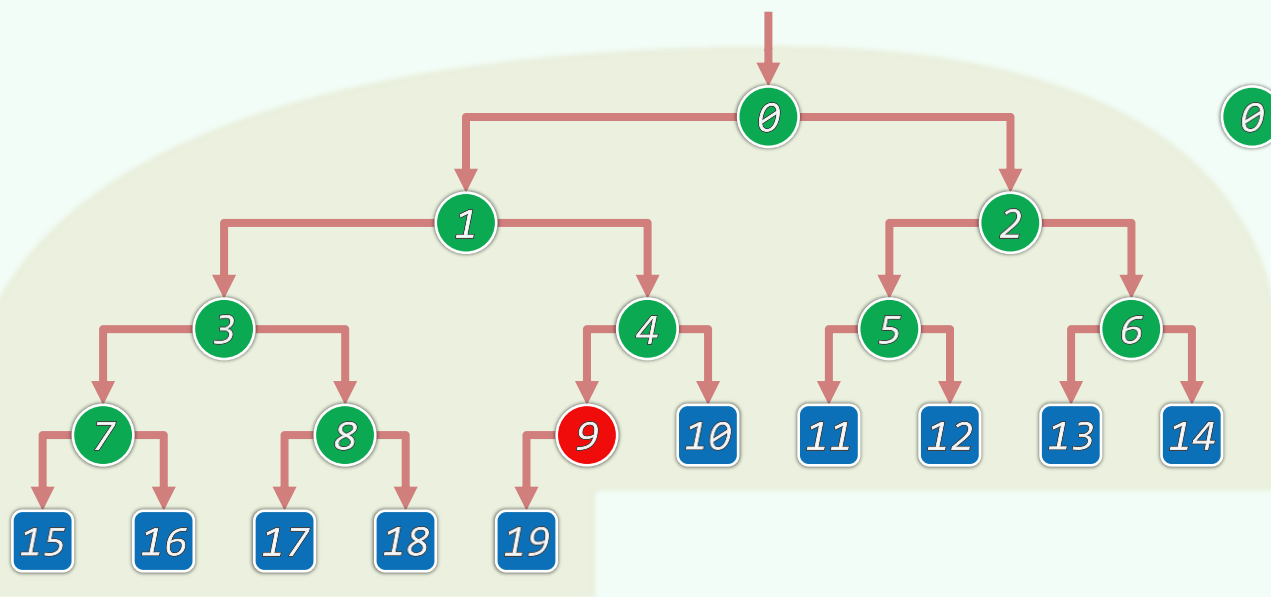
$$= \left\lfloor \frac{n-2}{2} \right\rfloor = \left\lceil \frac{n-3}{2} \right\rceil$$

比如 $n = 20$ 或 21 时，为 9

❖ `#define Parent(i) ( ( (i) - 1 ) >> 1 )`

`#define LChild(i) ( 1 + ( (i) << 1 ) )`

`#define RChild(i) ( ( 1 + (i) ) << 1 )`



PQ_ComplHeap = PQ + Vector

- ❖ `template <typename T> struct PQ_ComplHeap : public PQ<T>, public Vector<T> {
 PQ_ComplHeap(T* A, Rank n) { copy(A, 0, n); heapify(_elem, n); }
 void insert(T); T delMax(); T& getMax();
};`
- ❖ `template <typename T> Rank percolateDown( T* A, Rank n, Rank i ); //下滤`
`template <typename T> Rank percolateUp( T* A, Rank i ); //上滤`
- ❖ `template <typename T> void heapify( T* A, Rank n); //Floyd建堆算法`

堆序性：处处满足 $H[i] \leq H[\text{Parent}(i)]$

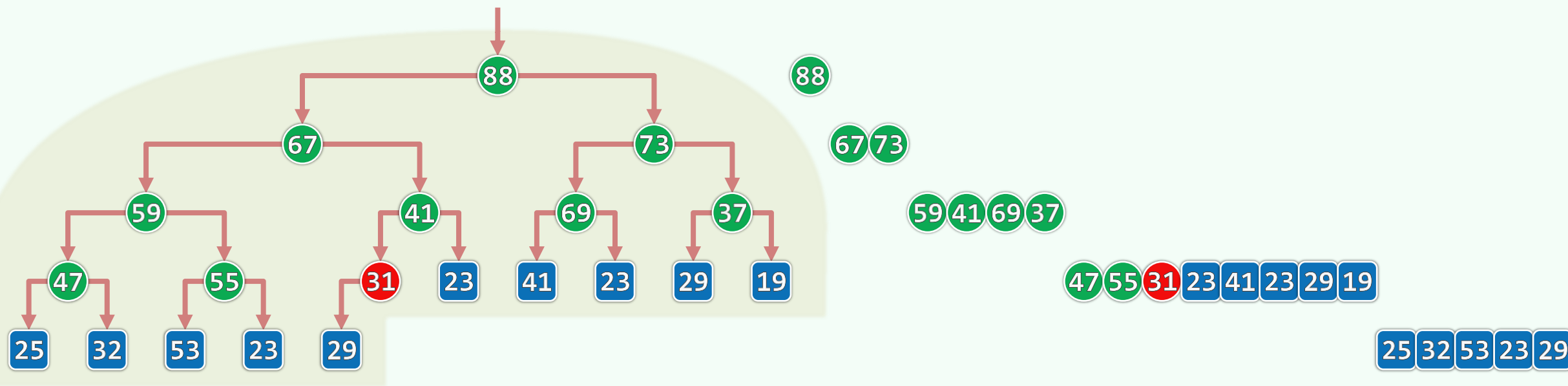
❖ 故唯有**无父**结点者，即 $H[0]$

才是全局的**最大**元素

❖ `template <typename T>`

`T& PQ_ComplHeap<T>::getMax()`

`{ return _elem[0]; }`



12-B2

优先级队列

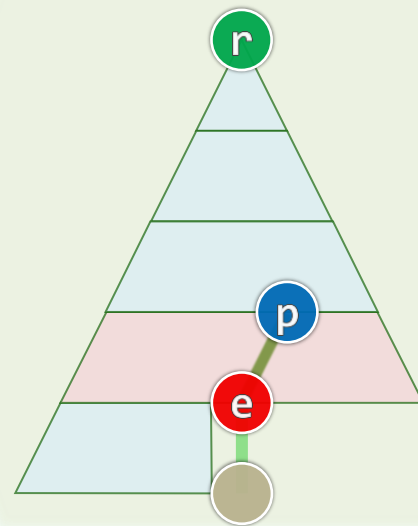
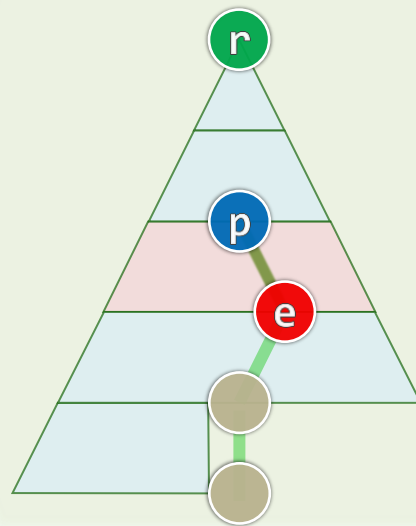
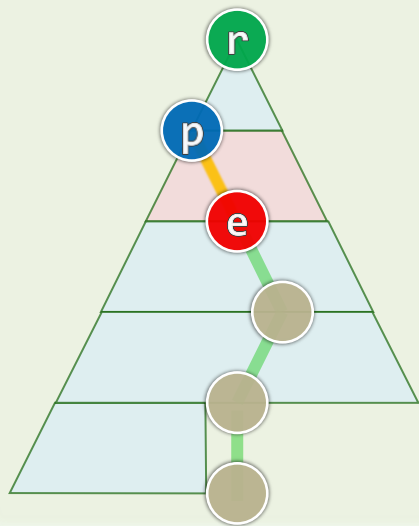
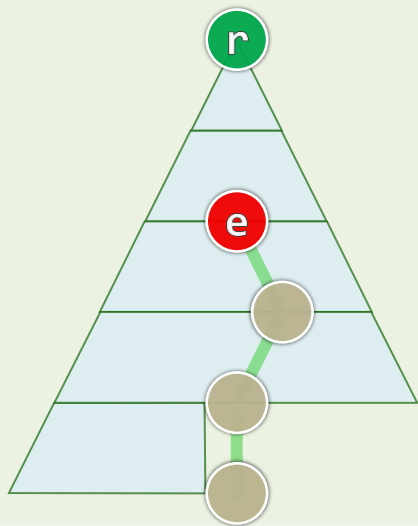
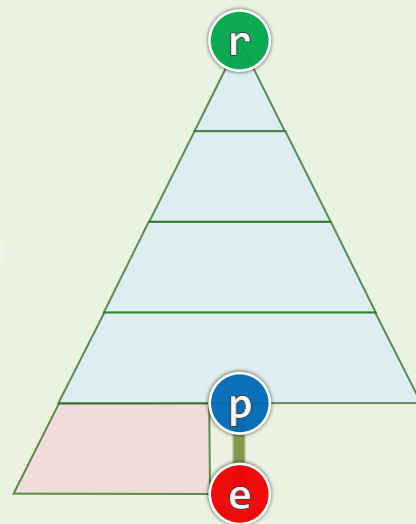
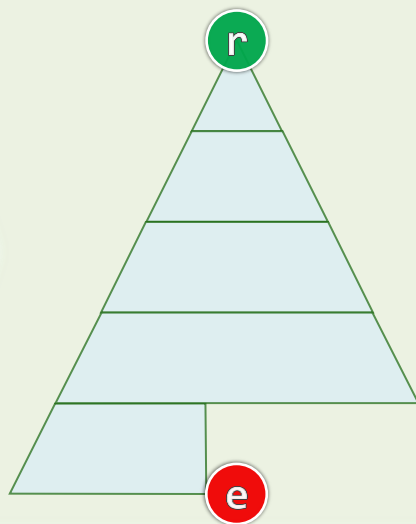
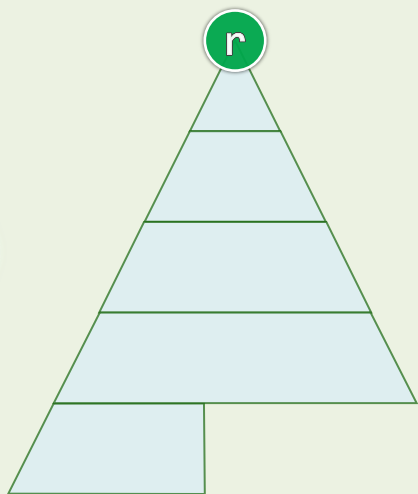
完全二叉堆：插入

時遷看見土地廟後一株大柏樹，便把兩隻腿夾定，一節節爬將上去樹頭頂，騎馬兒坐在枝柯上

邓俊辉

deng@tsinghua.edu.cn

算法：逐层上滤

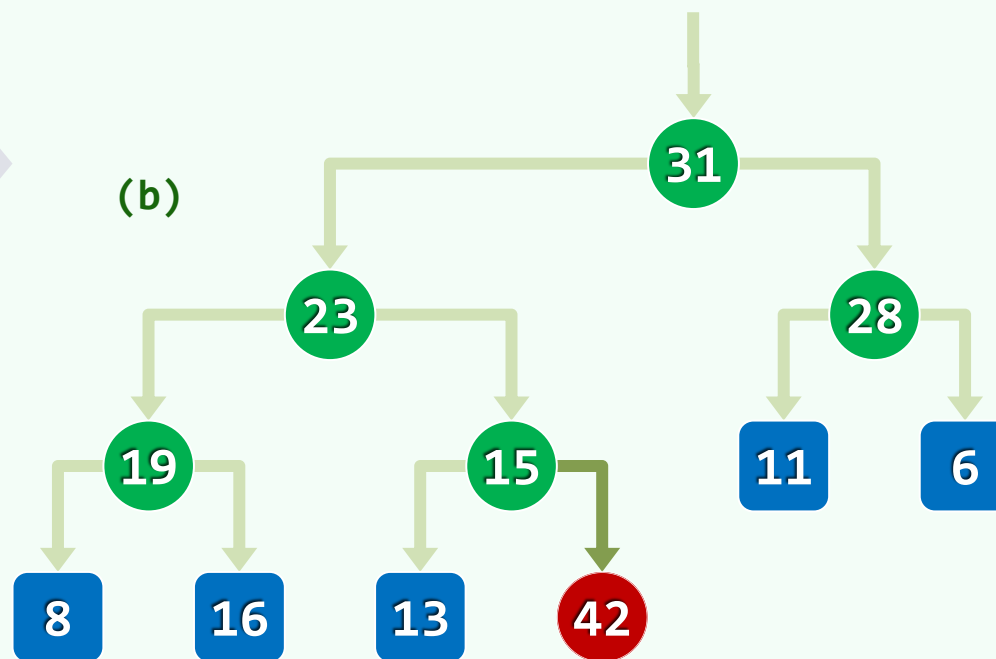
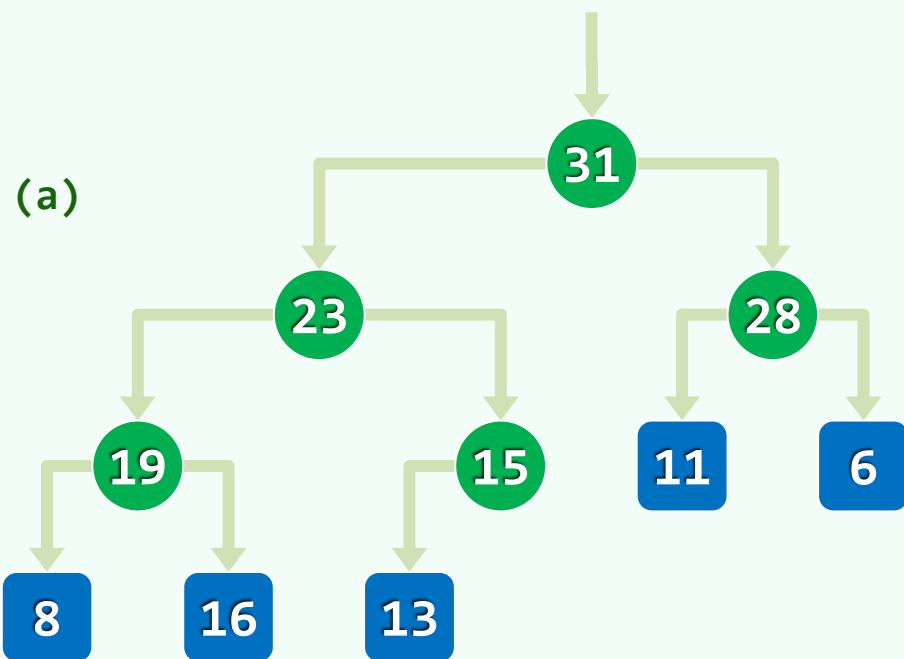


实现

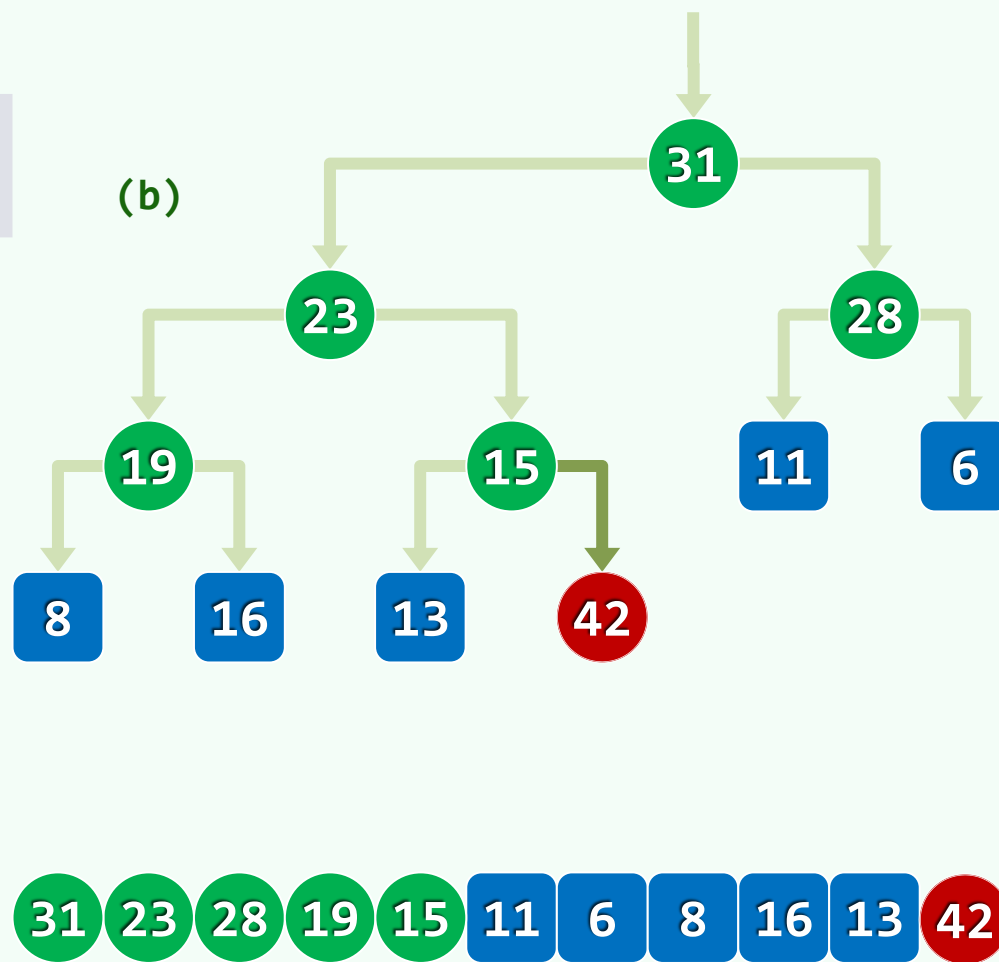
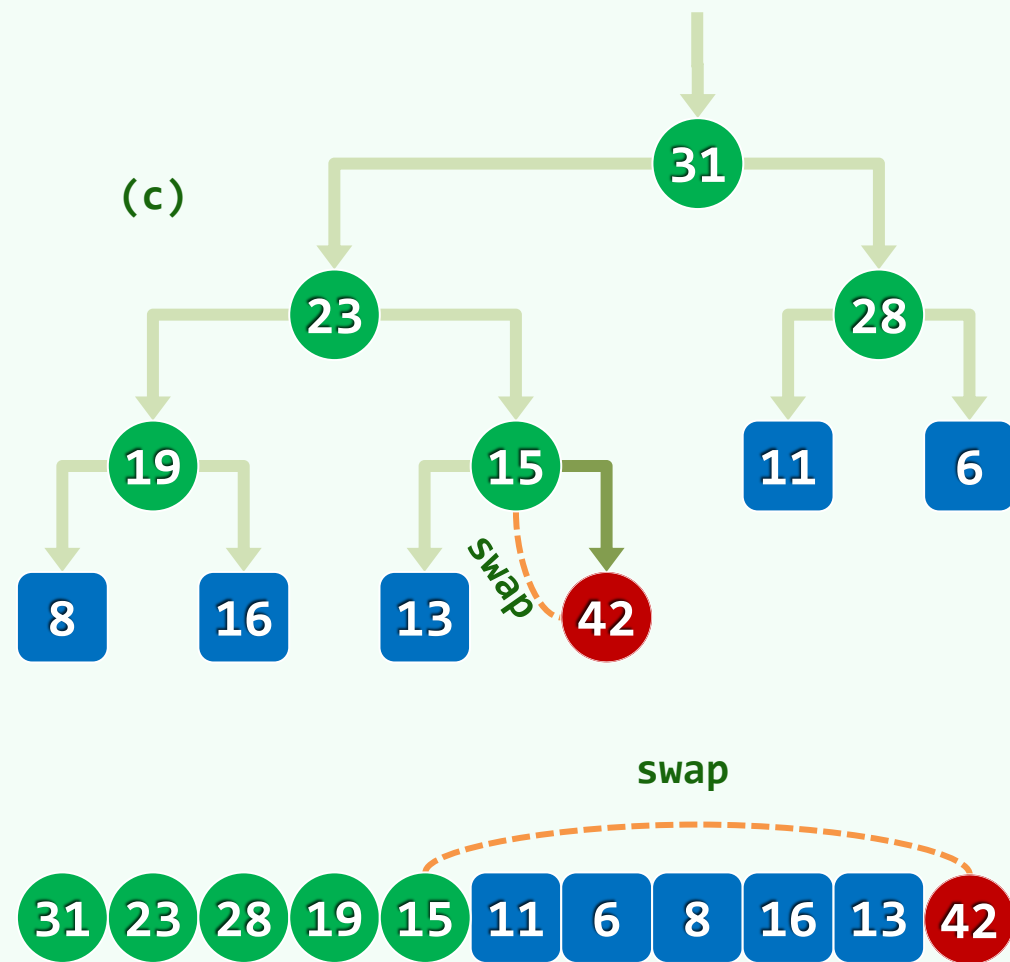
```
template <typename T> void PQ_ComplHeap<T>::insert( T e ) { //插入
    Vector<T>::insert( e ); //将新词条接至向量末尾（可能引发扩容）
    percolateUp( _elem, _size - 1 ); //再对该词条上滤调整
} //insert

template <typename T> Rank percolateUp( T* A, Rank i ) { //0 <= i < _size
    for ( Rank j; (0 < i) && (A[j = Parent(i)] < A[i]); i = j ) //自下而上，逐层检视
        swap( A[i], A[j] ); //只要父不如子，随即易位
    return i; //返回最终抵达的位置
} //percolateUp
```

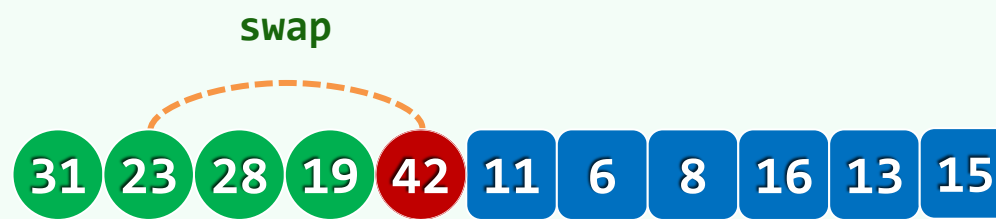
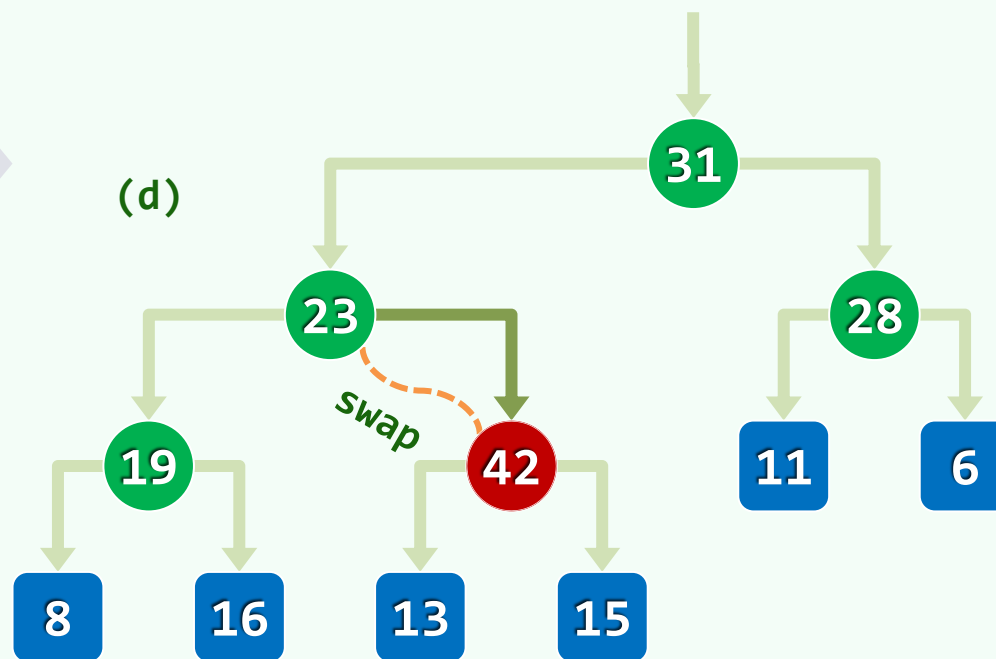
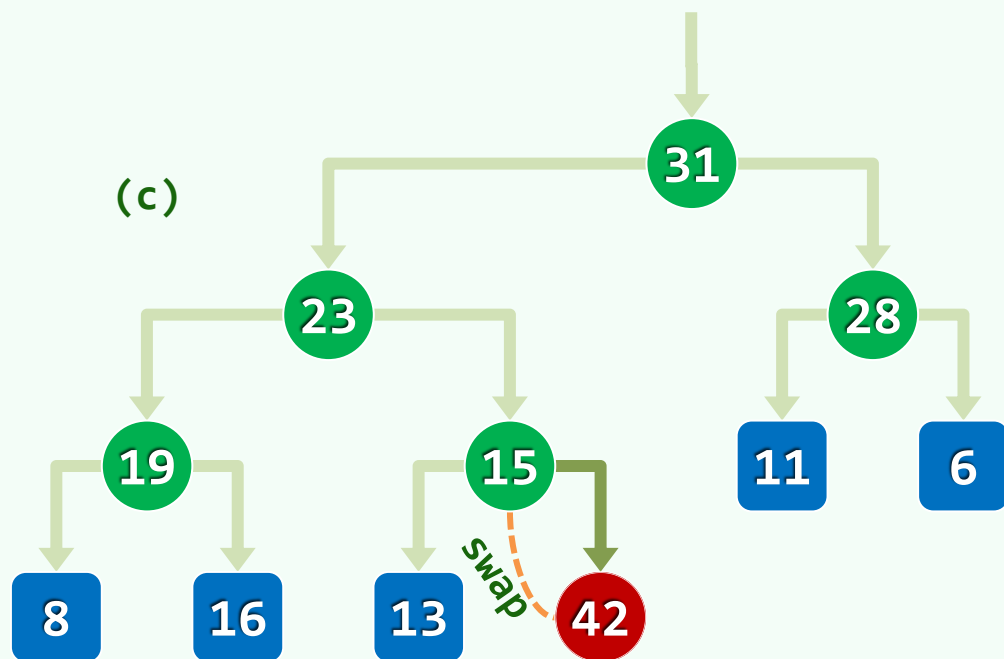
实例 (1/5)



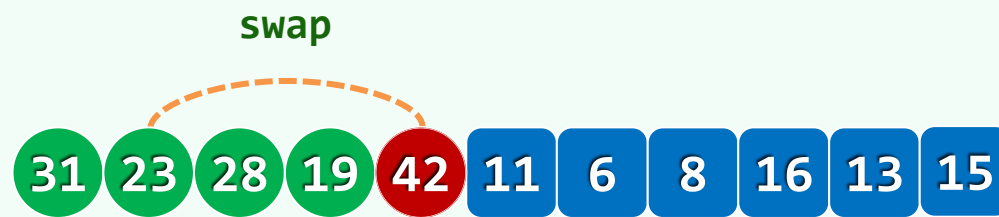
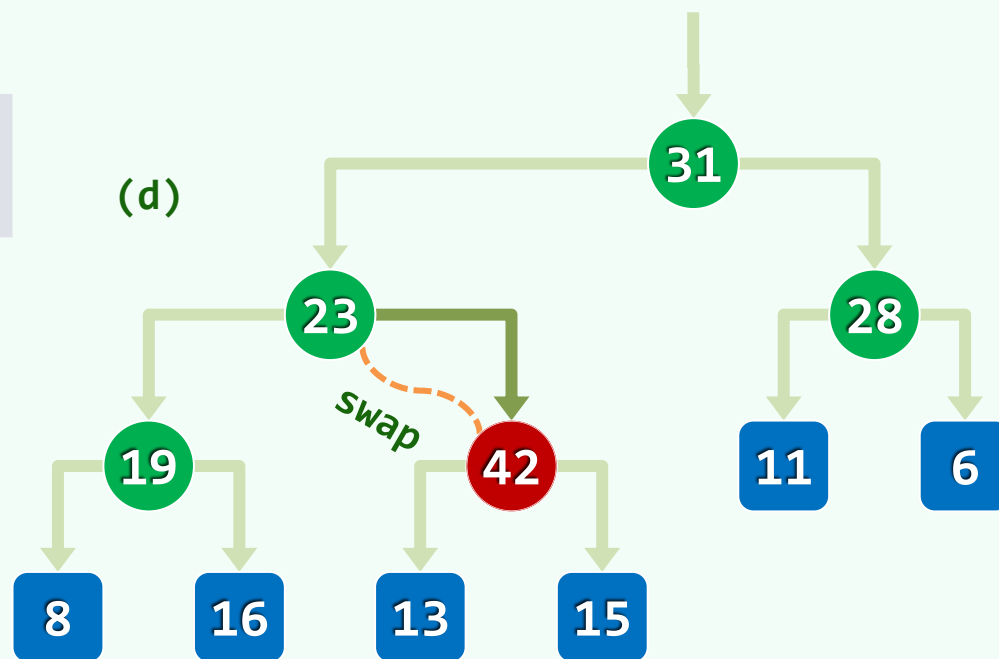
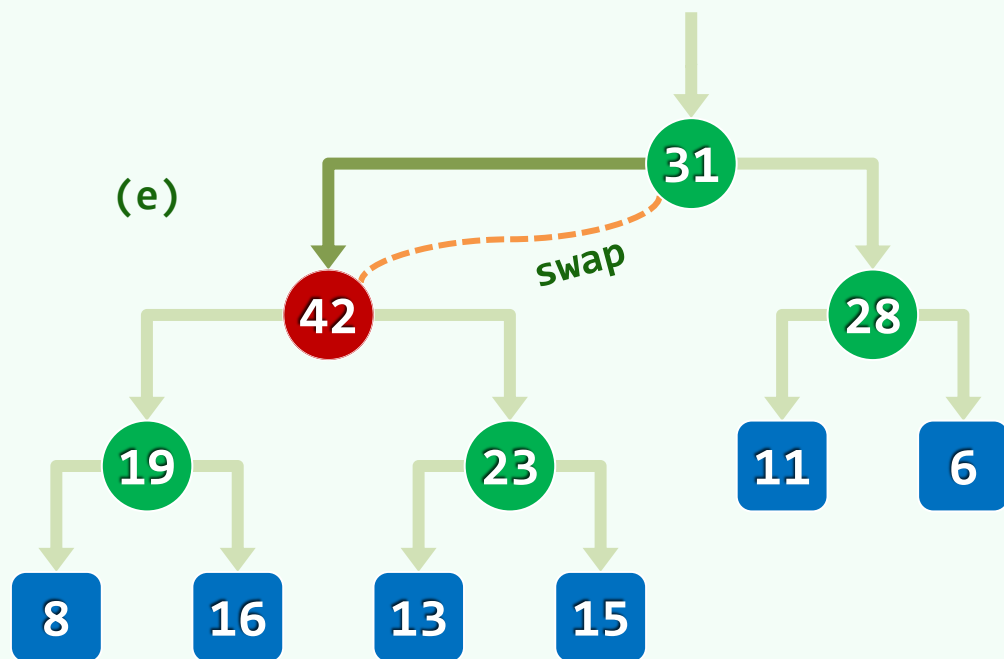
实例 (2/5)



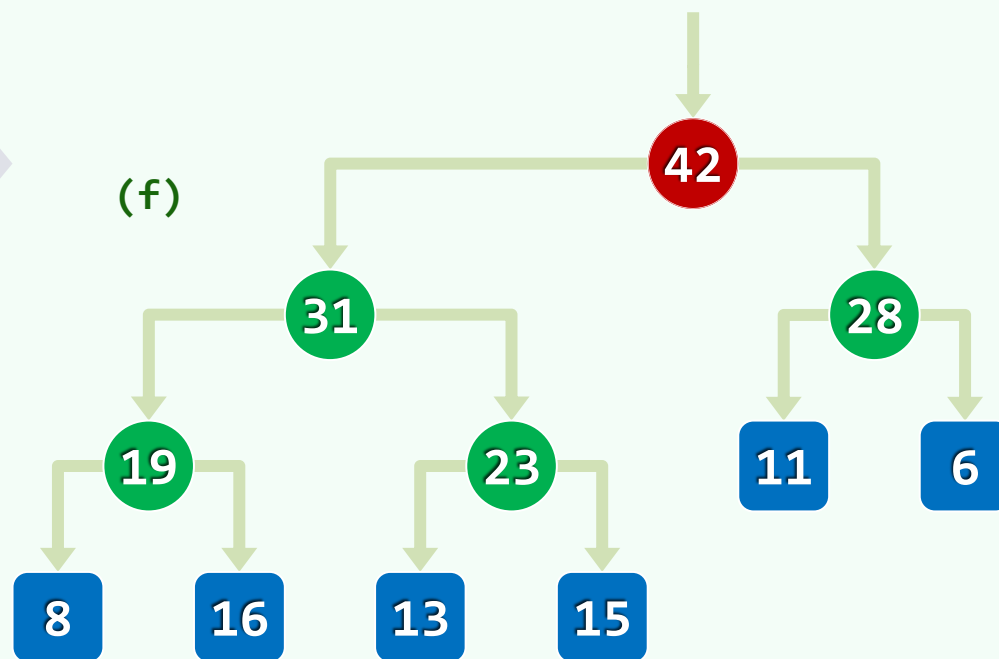
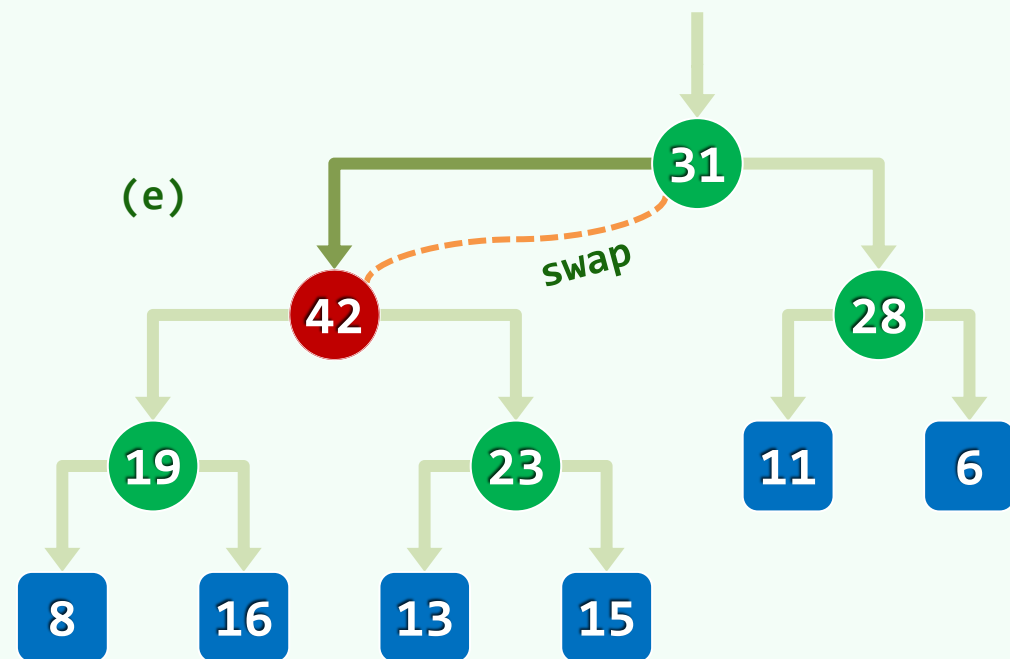
实例 (3/5)



实例 (4/5)



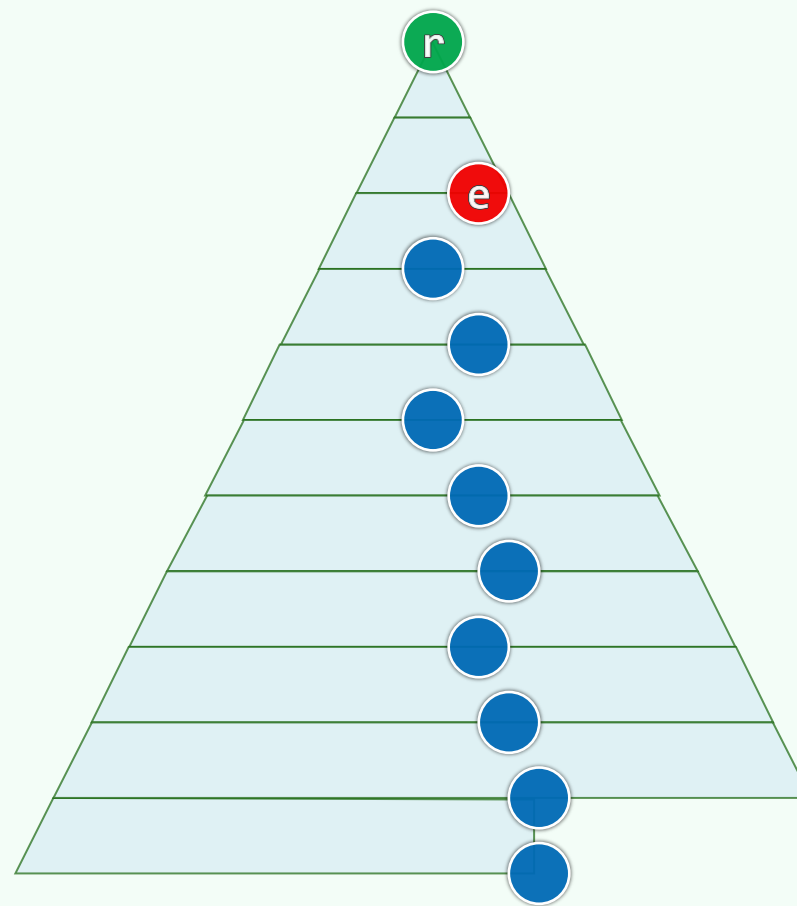
实例 (5/5)



效率

- ❖ e在上滤过程中，只可能与**祖先**们交换
- ❖ 完全树必平衡，e的祖先不超过 $\mathcal{O}(\log n)$ 个
- ❖ 故知插入操作可在 $\mathcal{O}(\log n)$ 时间内完成
- ❖ 然而就**数学期望**而言

实际效率往往远远更高...



12-B3

优先级队列

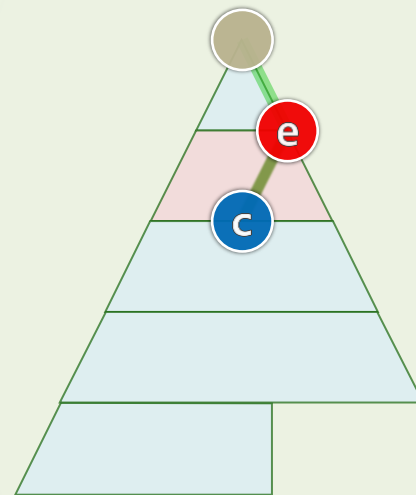
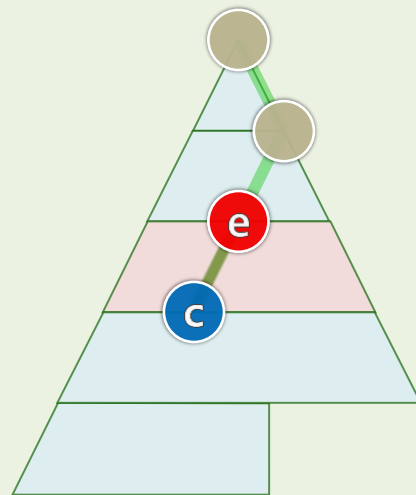
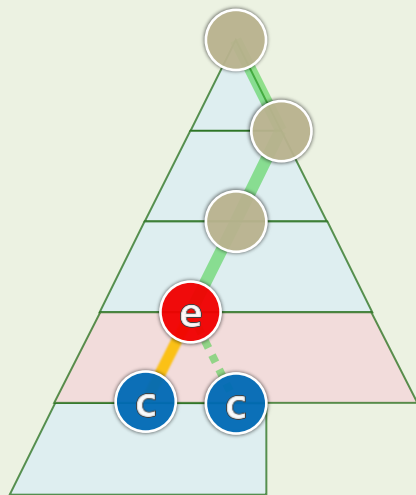
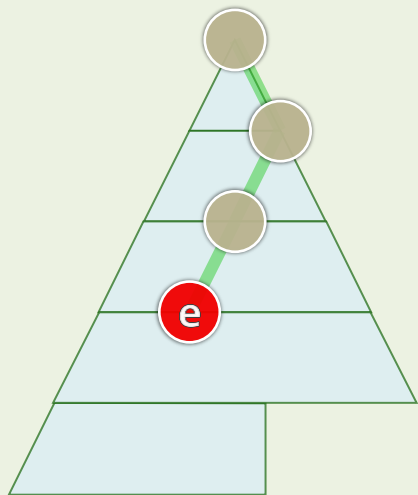
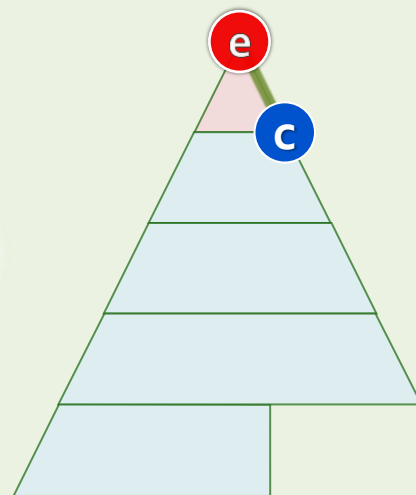
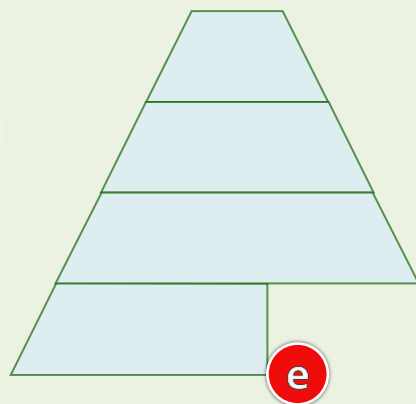
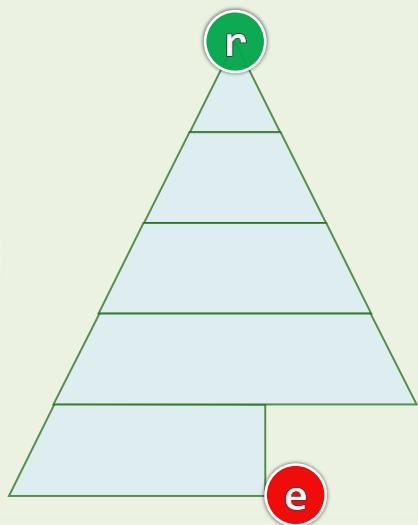
完全二叉堆：删除

I have scaled the peak and
found no shelter in
fame's bleak and barren height

邓俊辉

deng@tsinghua.edu.cn

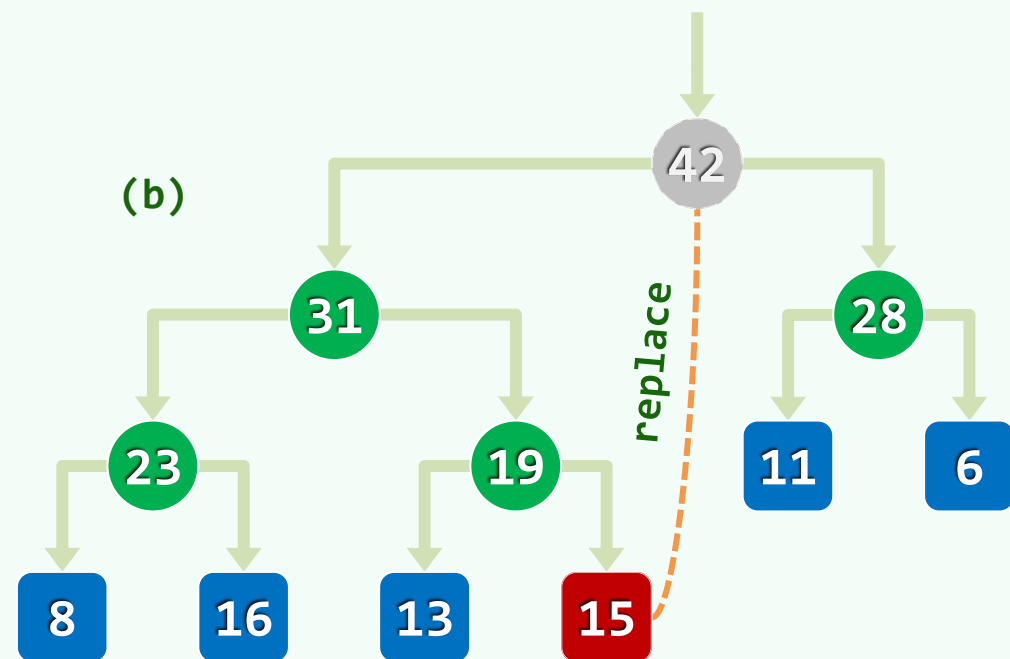
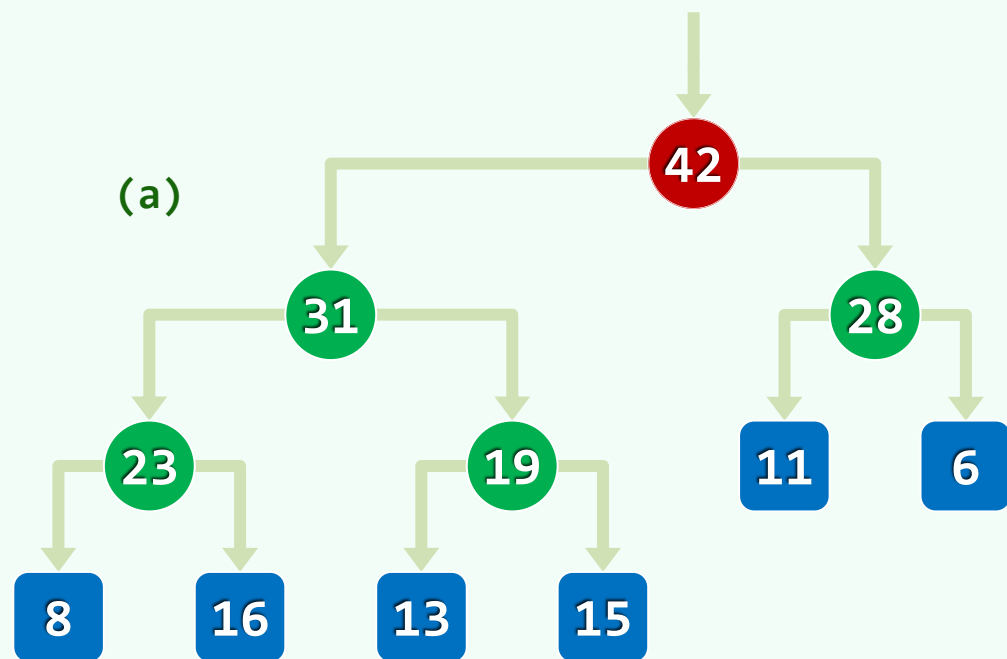
算法：割肉补疮 + 逐层下滤



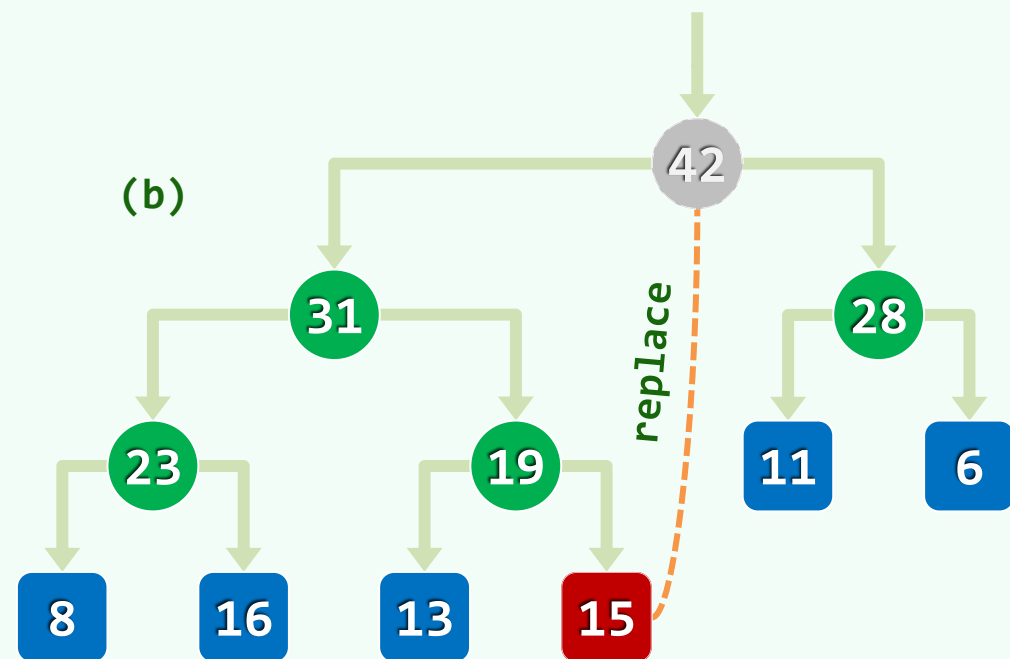
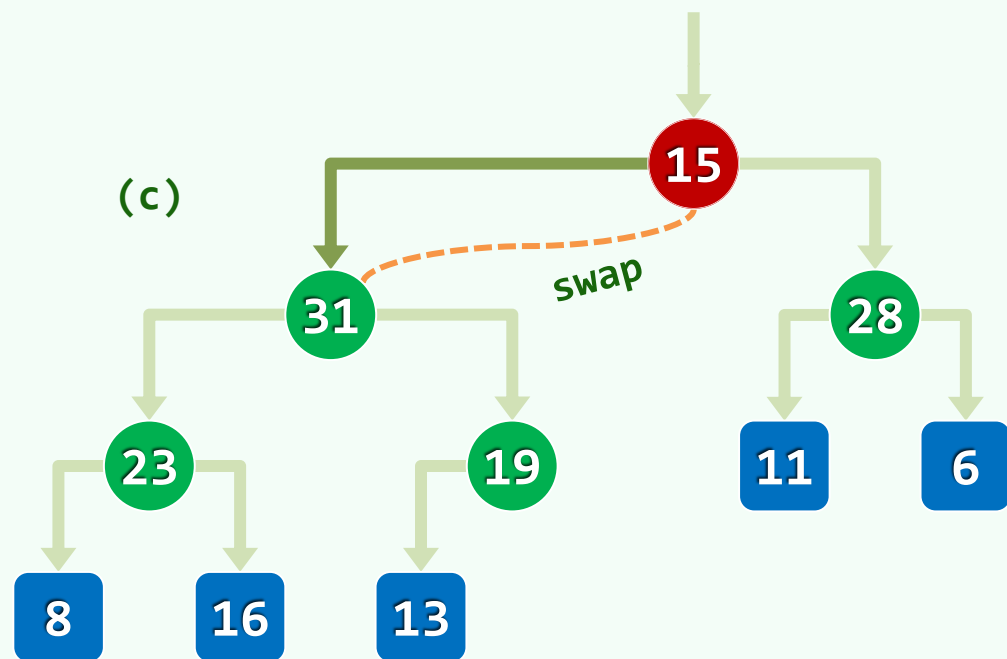
实现

```
template <typename T> T PQ_ComplHeap<T>::delMax() {  
    swap( _elem[0], _elem[ --_size ] ); //堆顶、堆尾互换（不致引发shrink()）  
    percolateDown( _elem, _size, 0 ); //新堆顶下滤  
    return _elem[ _size ]; //返回原堆顶  
} //delMax  
  
template <typename T> Rank percolateDown( T* A, Rank n, Rank i ) { //0 <= i < n  
    for ( Rank j; i != ( j = ProperParent( A, n, i ) ); i = j ) //自上而下，逐层检视  
        swap( A[i], A[j] ); //只要i不堪为父，随即与更强的孩子j易位  
    return i; //返回最终抵达的位置（亦i亦j）  
} //percolateDown
```

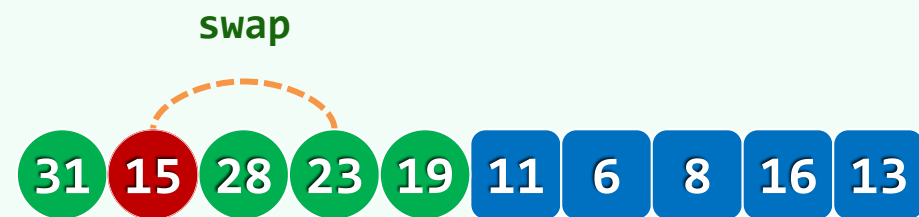
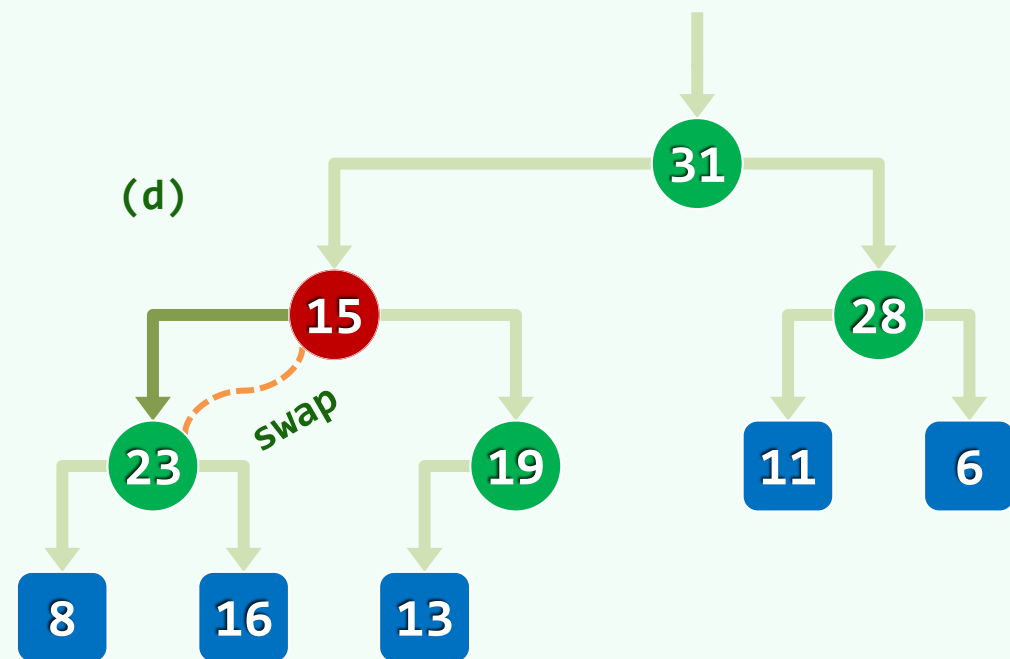
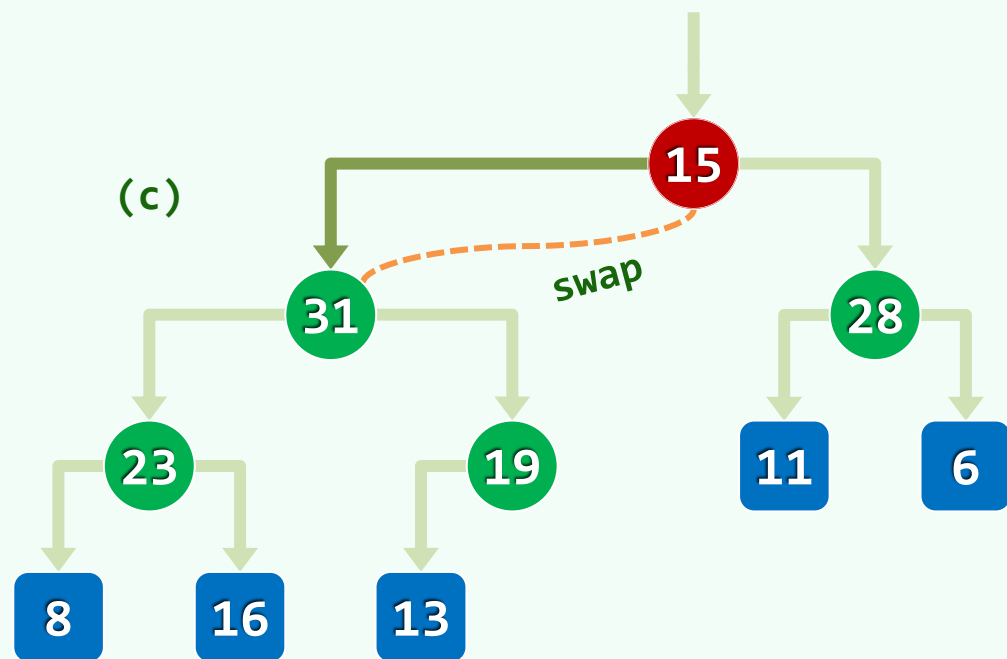

实例 (1/5)



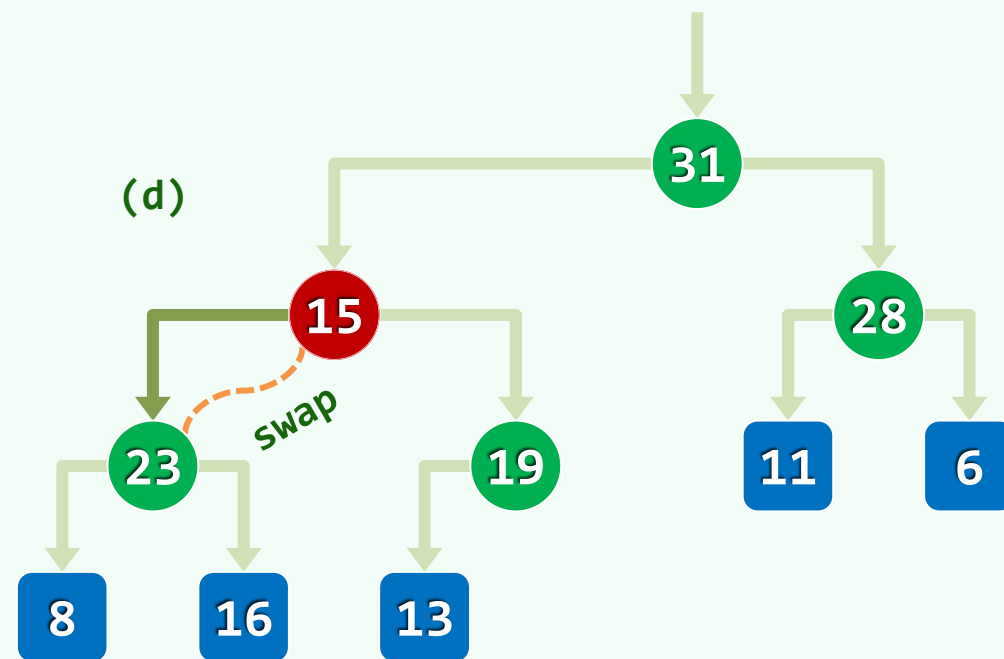
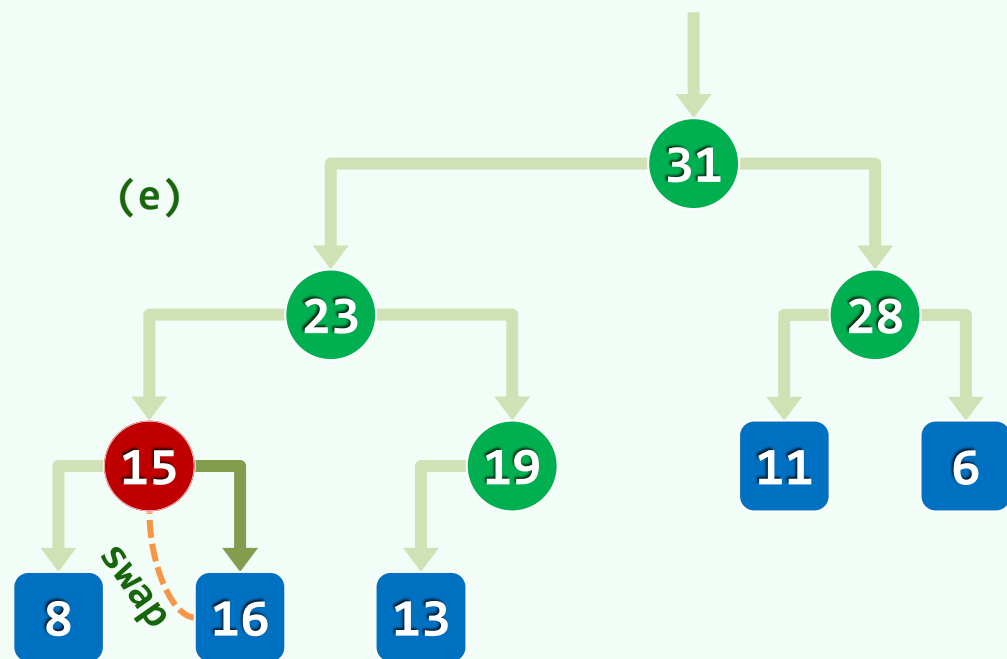
实例 (2/5)



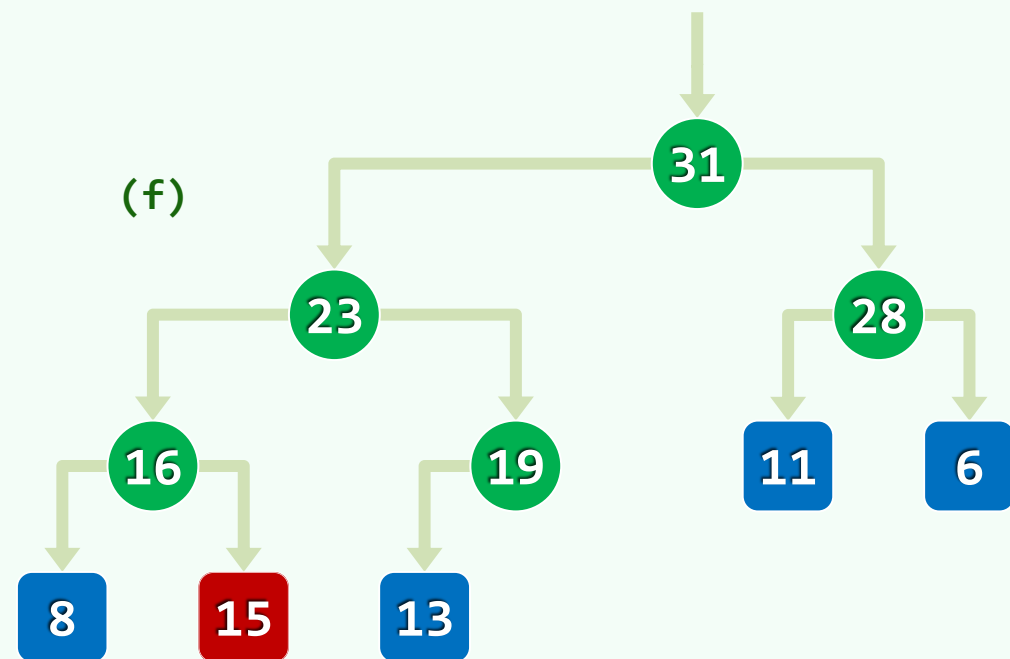
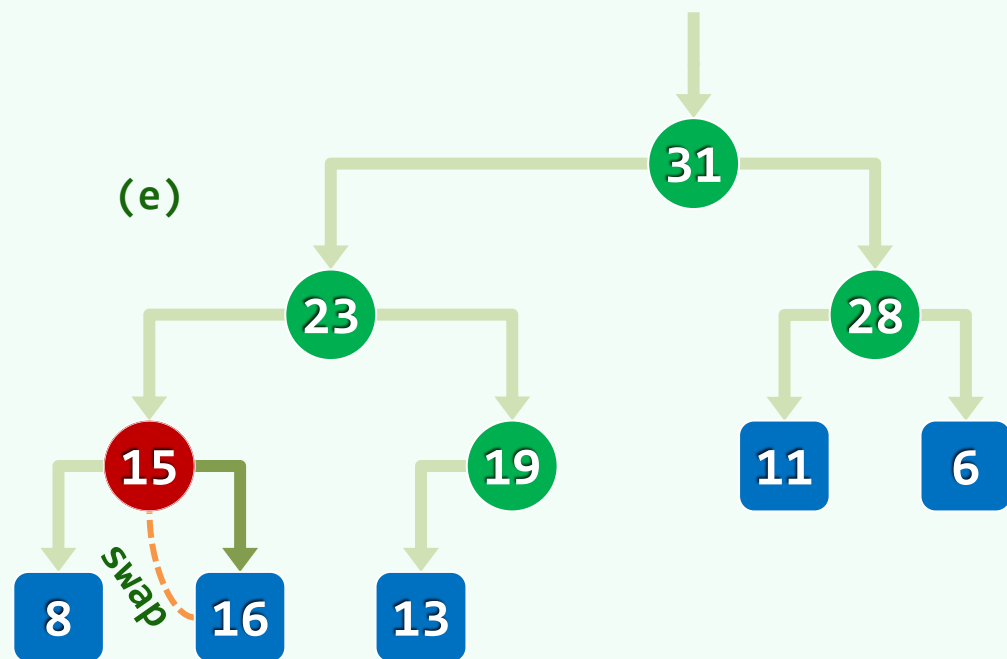
实例 (3/5)



实例 (4/5)



实例 (5/5)



效率

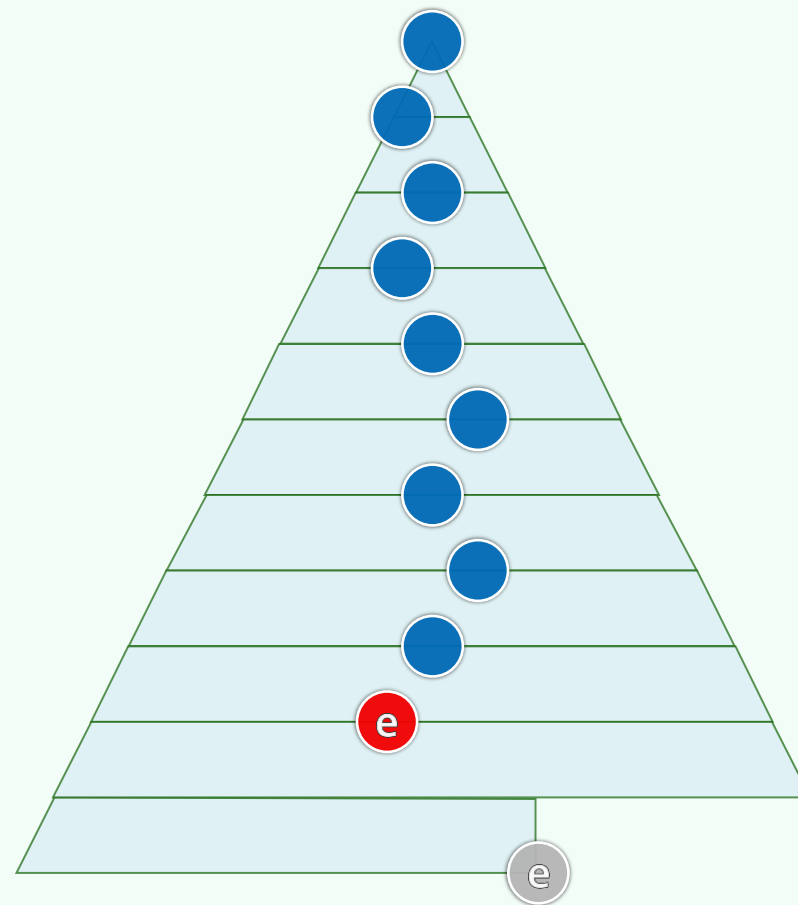
❖ e在每一高度至多交换**一次**

累计交换不超过 $\mathcal{O}(\log n)$ 次

❖ 通过下滤，可在 $\mathcal{O}(\log n)$ 时间内

- 删除堆顶节点，并
- 整体重新调整为堆

❖ 数学期望呢？



优先级队列

完全二叉堆：批量建堆

12-B4

邓俊辉

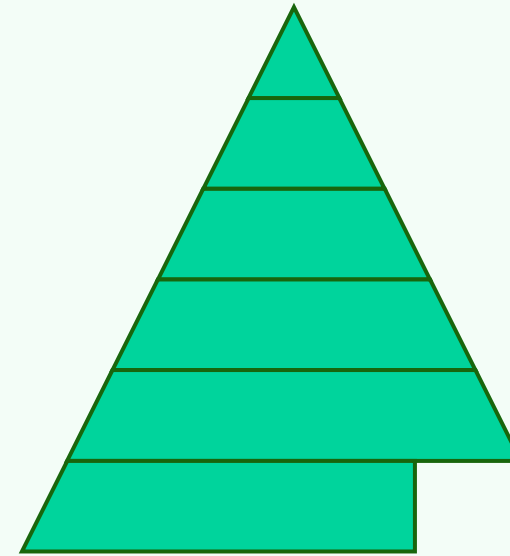
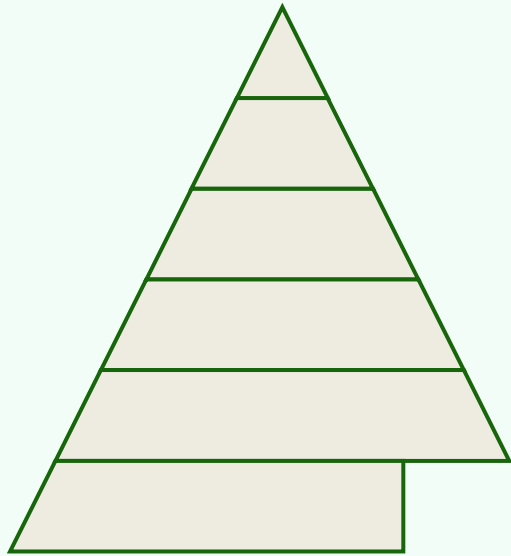
deng@tsinghua.edu.cn

今世号通人者，务为艰深之文，陈过高之义，以为士大夫劝，而独不为彼什伯千万倍里巷乡闾之子计，则是智益智，愚益愚，智日少，愚日多也

现在，培训富人应对贫穷要比教育穷人获得财富更切合实际，因为从人数比例上来说，富人破产的越来越多，而穷人变富的越来越少

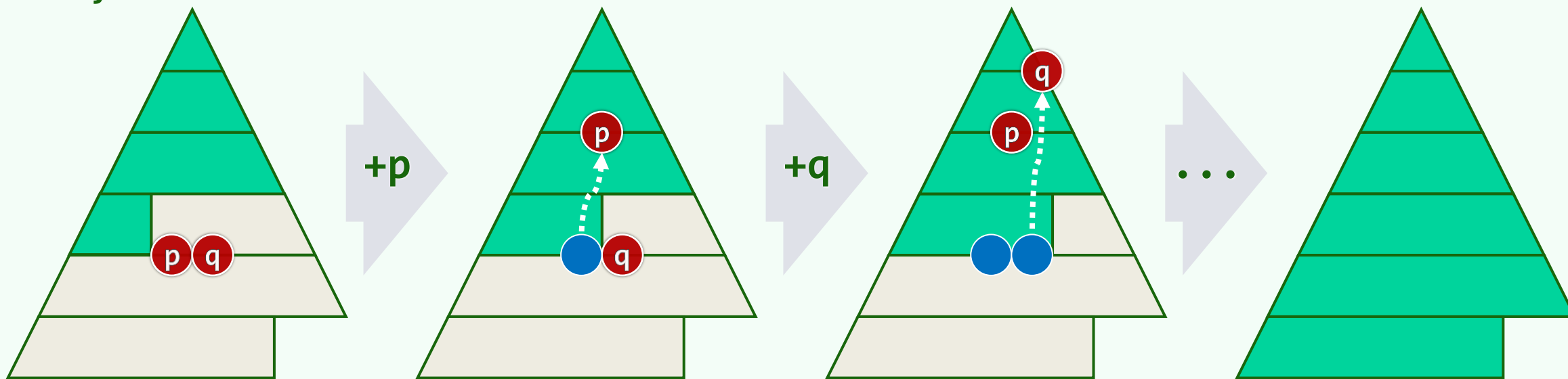
自上而下的上滤 (1/2)

```
PQ_ComplHeap( T* A, Rank n ) {  
    copy( A, 0, n ); //O(n)  
    heapify( _elem, n ); //O(?)  
}
```



自上而下的上滤 (2/2)

```
template <typename T> void heapify( T* A, const Rank n ) { //蛮力  
    for ( Rank i = 1; i < n; i++ ) //按照层次遍历次序, 逐个对节点做上滤  
        percolateUp( A, i ); //等效于省略Vector::insert(A[i]), 直接插入  
}
```



效率

❖ 最坏情况下

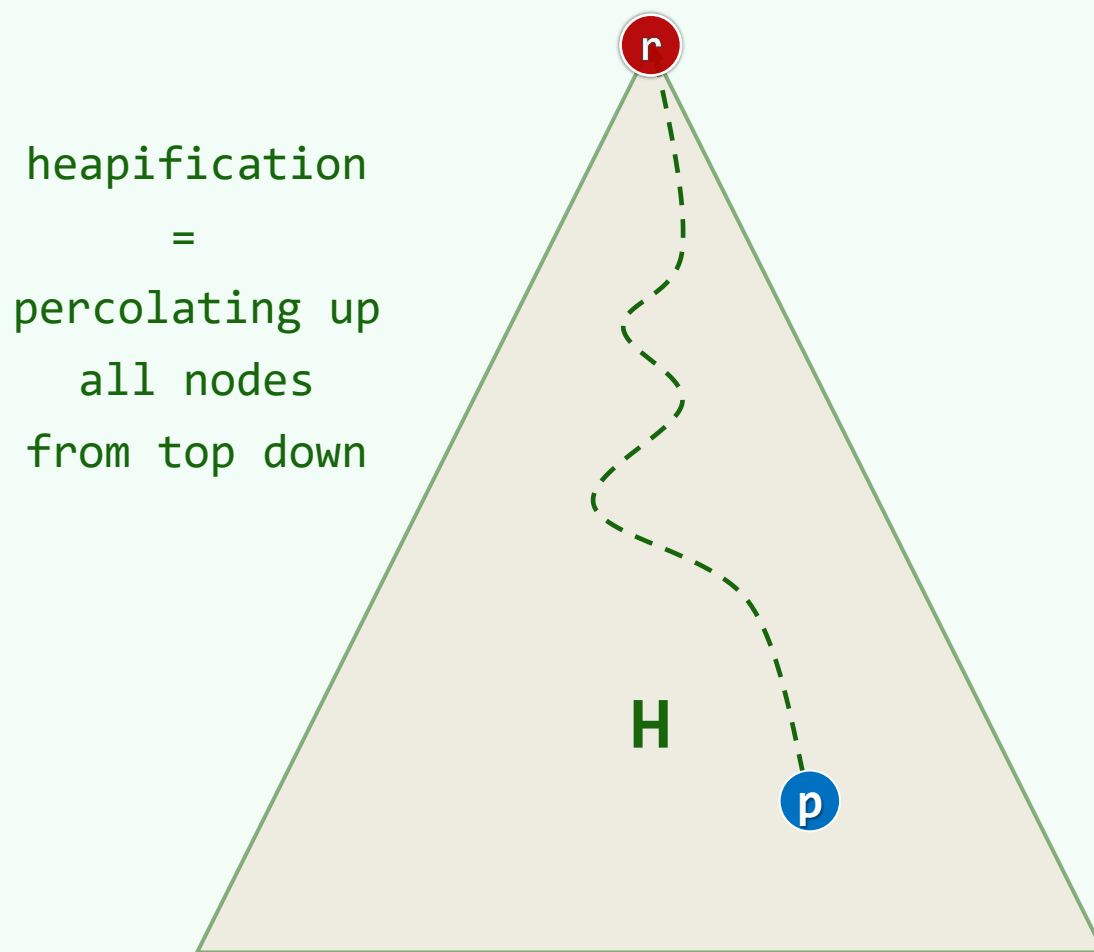
- 每个节点都需上滤至**根**
- 所需成本线性正比于其**深度**

❖ 即便只考虑**底层**

- $n/2$ 个叶节点, 深度均为 $\mathcal{O}(\log n)$
- 亦累计耗时 $\mathcal{O}(n \log n)$

❖ 这样长的时间, 本足以**全排序**!

所以...应该能够...更快的...



自下而上的下滤

❖ 任意给定堆 $\mathcal{H}_0$ 和 $\mathcal{H}_1$, 以及节点 p

❖ 为得到堆 $\mathcal{H}_0 \cup \{p\} \cup \mathcal{H}_1$, 只需

将 r_0 和 r_1 当作 p 的孩子, 再对 p 下滤

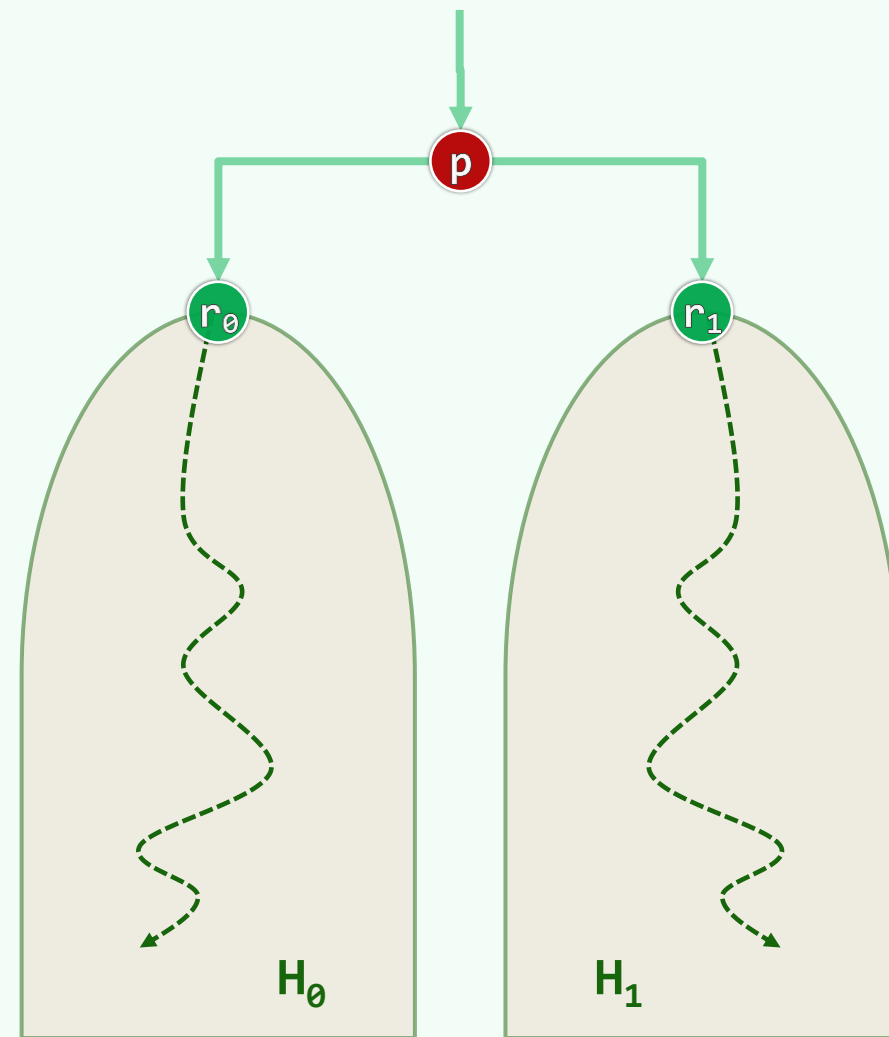
❖ `template <typename T> //Robert Floyd, 1964`

```
void heapify( T* A, Rank n ) { //自下而上, 对内部
```

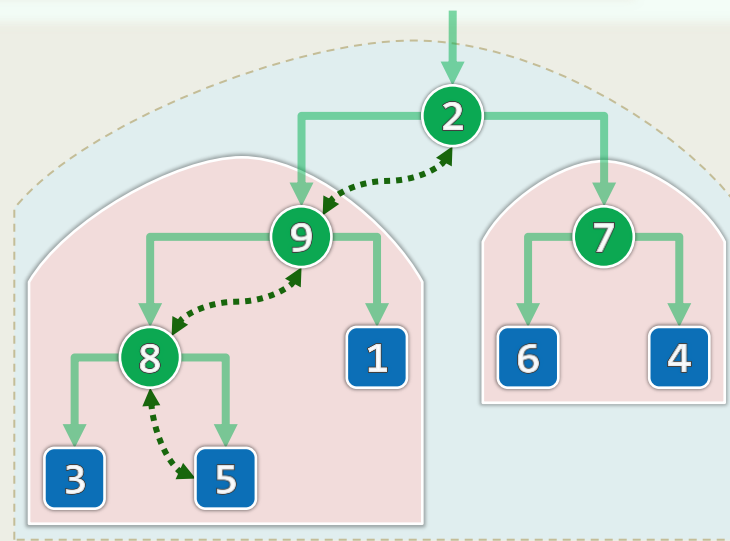
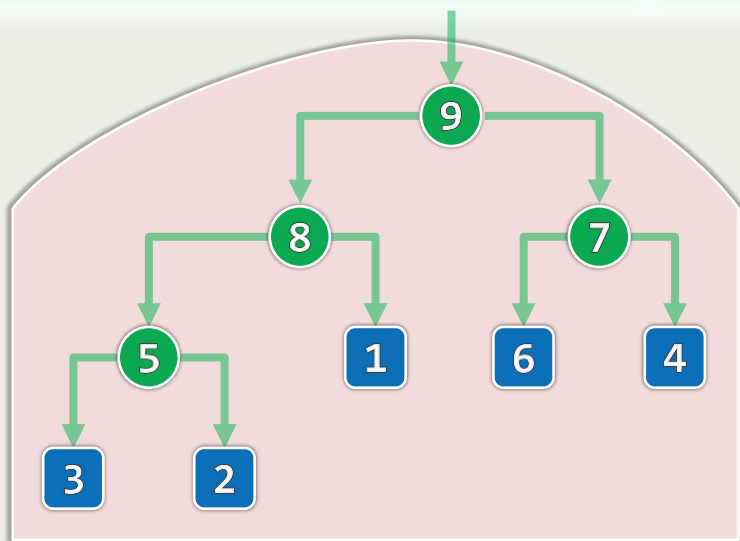
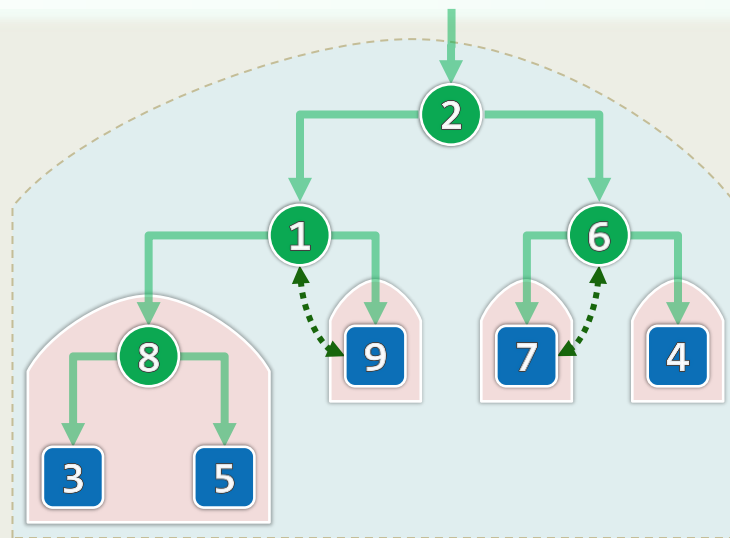
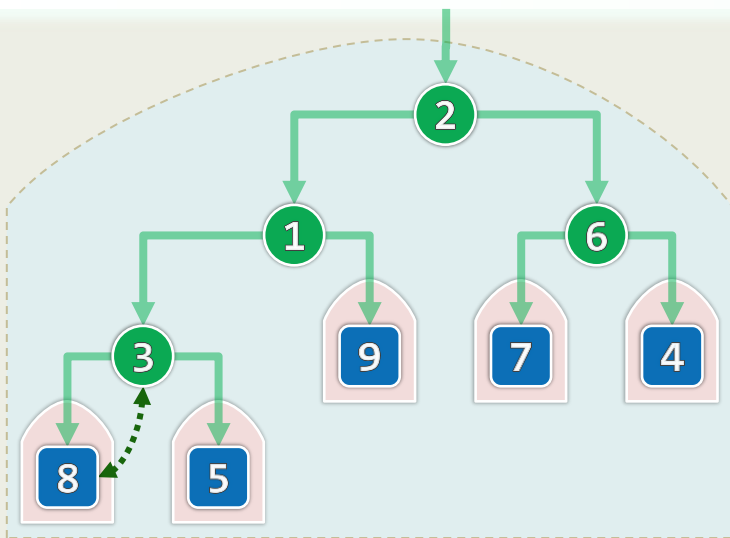
```
    for ( Rank i = n/2-1; -1 != i; i-- ) //节点
```

```
        percolateDown( A, n, i ); //逐个下滤
```

```
    } //可理解为子堆的逐层合并, 堆序性最终必在全球恢复
```



实例



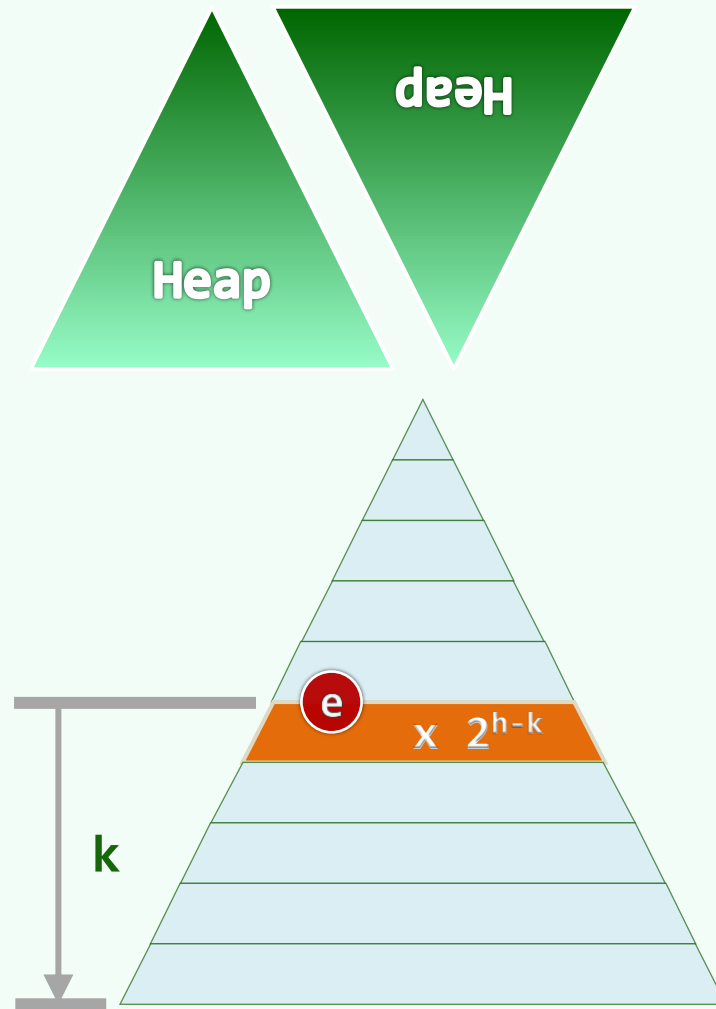
效率

❖ 每个内部节点所需的调整时间，正比于其**高度**而非**深度**

❖ 不失一般性，考查满树： $n = 2^{h+1} - 1$

❖ 所有节点的**高度**总和

$$\begin{aligned} S(n) &= \sum_{k=1}^h k \cdot 2^{h-k} = \sum_{k=1}^h \sum_{i=1}^k 2^{h-k} = \sum_{i=1}^h \sum_{k=i}^h 2^{h-k} \\ &= \sum_{i=1}^h \sum_{k=0}^{h-i} 2^k = \sum_{i=1}^h \{2^{h-i+1} - 1\} = \sum_{i=1}^h 2^{h-i+1} - h \\ &= \sum_{i=1}^h 2^i - h = 2^{h+1} - 2 - h = \mathcal{O}(n) \end{aligned}$$



课后

- ❖ insert(): 最坏情况下效率为 $\mathcal{O}(\log n)$, 平均情况呢?
- ❖ heapify(): 构造次序颠倒后, 为什么复杂度会有实质降低?
这一算法在哪些场合不适用?
- ❖ 扩充接口: `decrease( i, delta )` //任一元素`_elem[i]`的数值减小`delta`
`increase( i, delta )` //任一元素`_elem[i]`的数值增加`delta`
`remove( i )` //删除任一元素`_elem[i]`
- ❖ 借助完全堆, 在 $\mathcal{O}(n \log n)$ 时间内构造Huffman树
- ❖ 大顶堆的`delMin()`操作, 能否也在 $\mathcal{O}(\log n)$ 时间内完成?
难道, 为此需要同时维护一个小顶堆?

12-B5

优先级队列

完全二叉堆：堆排序

邓俊辉

deng@tsinghua.edu.cn

原理不变，还是逐个选取

❖ 在 selectionSort() 中

将 **U** 替换为 **H**...

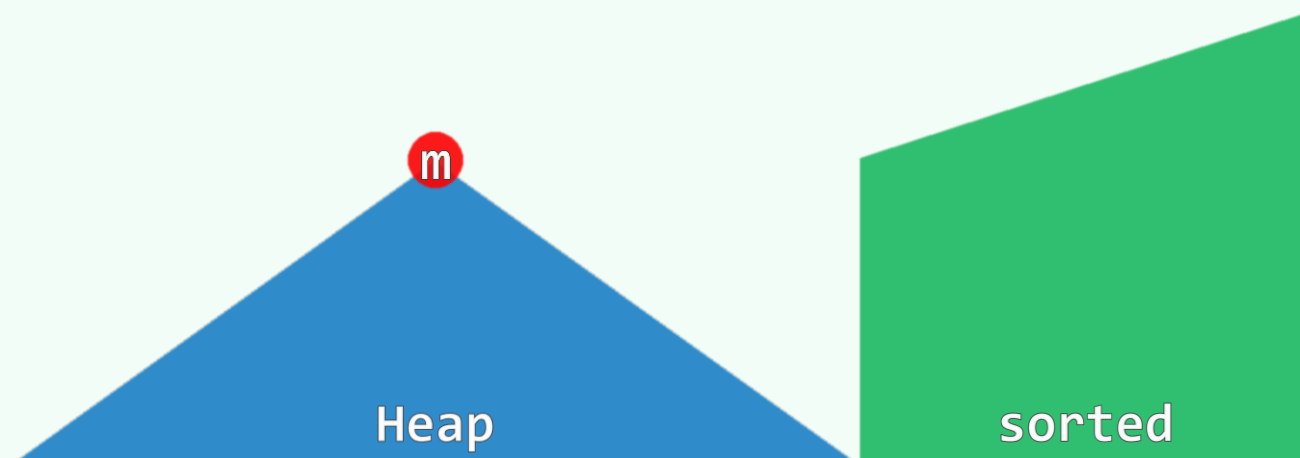
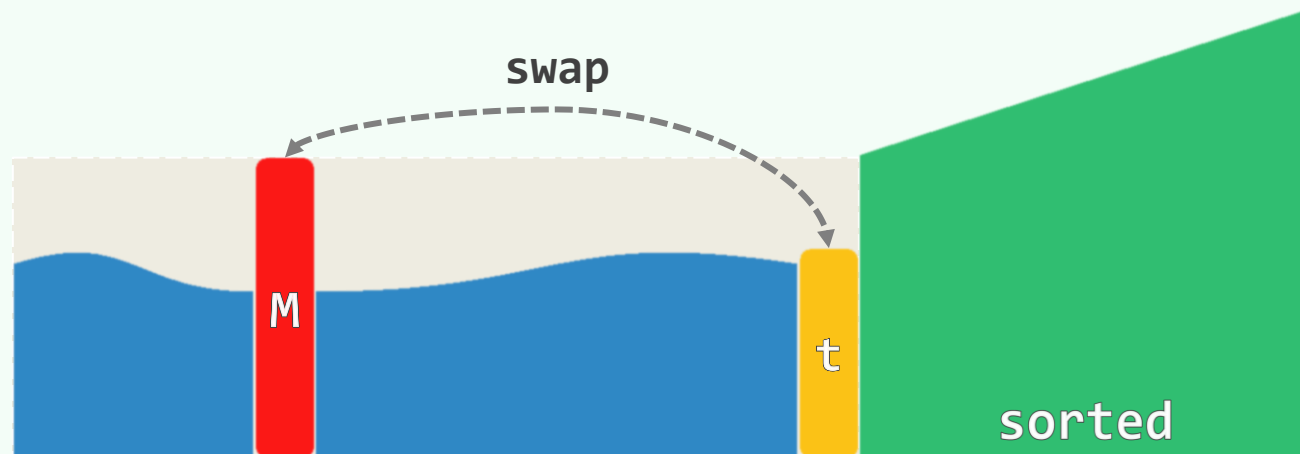
❖ J. Williams, 1964

初始化 : heapify(), $\mathcal{O}(n)$

迭代 : delMax(), $\mathcal{O}(\log n)$

不变性 : $H \leq S$

❖ $\mathcal{O}(n) + n \cdot \mathcal{O}(\log n)$
 $= \mathcal{O}(n \cdot \log n)$



就地

❖ 在物理上

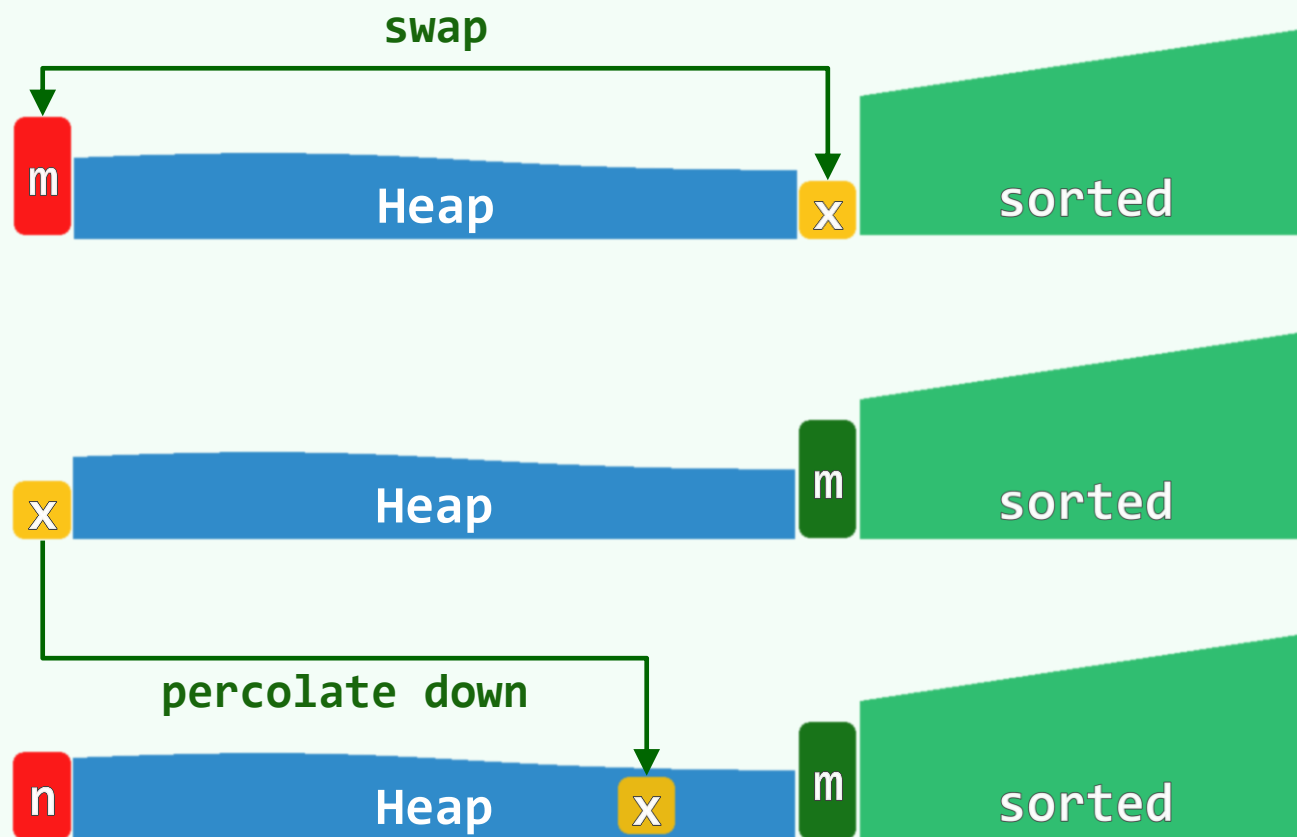
完全二叉堆即是向量

❖ 既然此前有：

- $m = H[0]$
- $x = H[n-1]$

何不随即就：

- $\text{swap}(m, x) = H.\text{insert}(x) + S.\text{insert}(m)$



实现

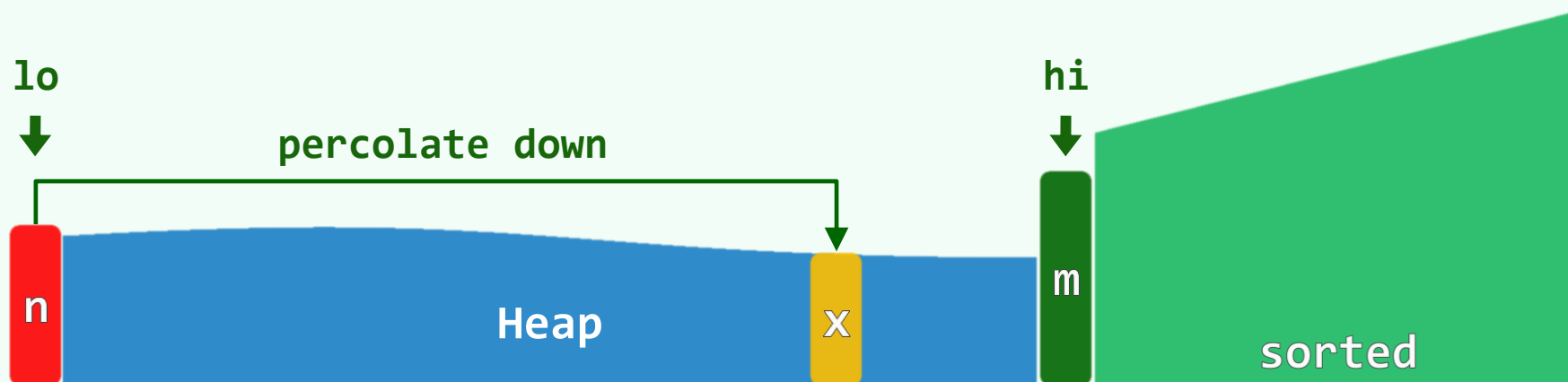
```
template <typename T> void Vector<T>::heapSort( Rank lo, Rank hi ) { //就地堆排序

    T* A = _elem + lo; Rank n = hi - lo; heapify( A , n ); //待排序区间建堆,  $O(n)$ 

    while ( 0 < --n ) //反复地摘除最大元并归入已排序的后缀, 直至堆空

        { swap( A[0], A[n] ); percolateDown( A, n, 0 ); } //堆顶与末元素对换后下滤

    } //heapsort
```

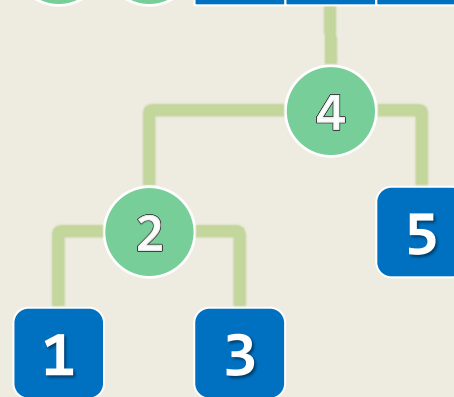


实例：建堆

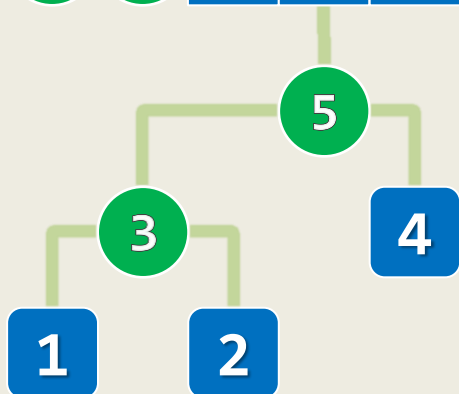


4 2 5 1 3

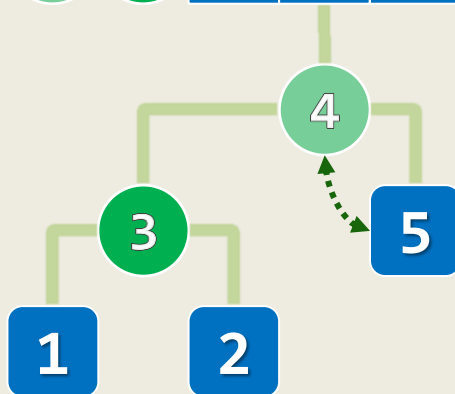
4 2 5 1 3



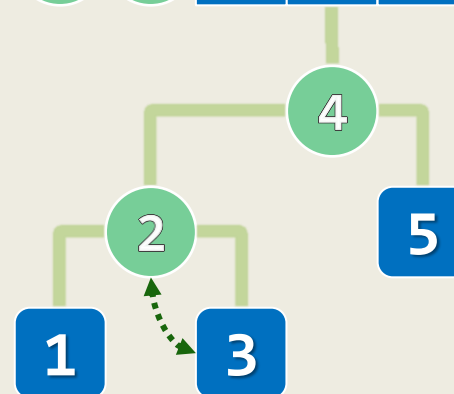
5 3 4 1 2



4 3 5 1 2

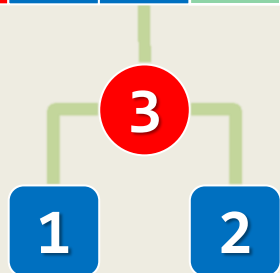


4 2 5 1 3



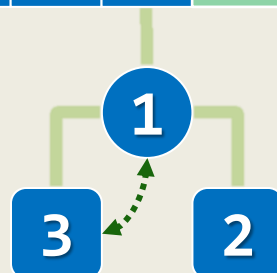
实例：选取 + 调整 (1/2)

3 1 2 4 5



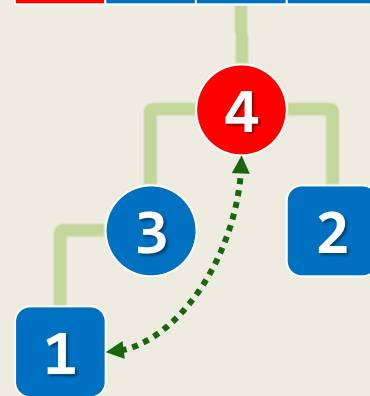
4

1 3 2 4 5

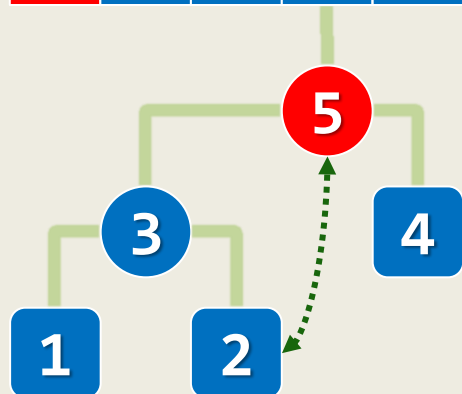


4

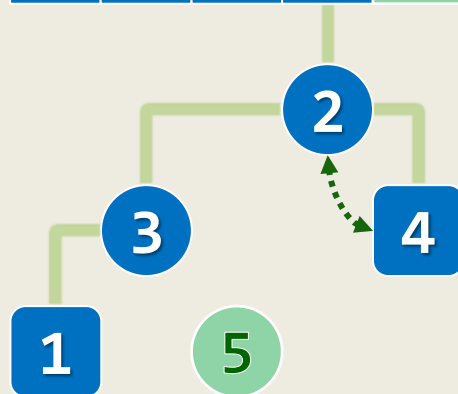
4 3 2 1 5



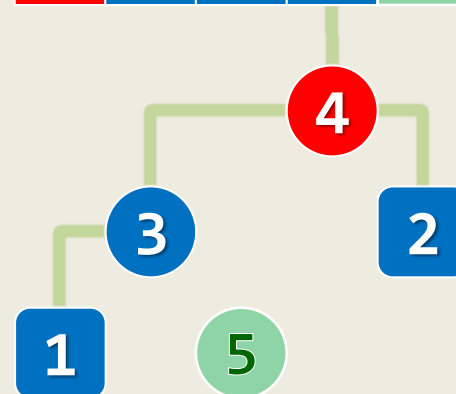
5 3 4 1 2



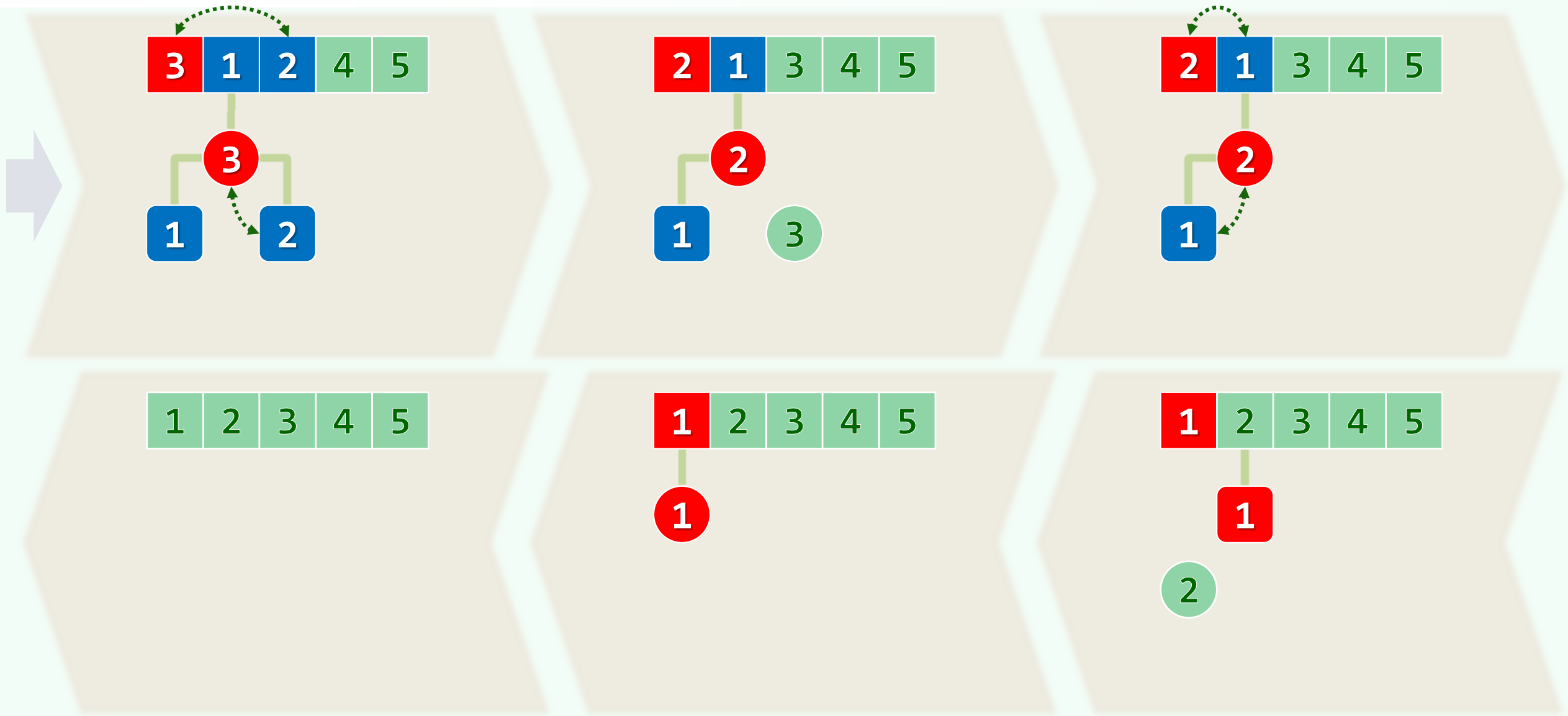
2 3 4 1 5



4 3 2 1 5



实例：选取 + 调整 (2/2)



12-C1

优先级队列

锦标赛树：胜者树

怎麼叫做分瓣梅花計？

如今把洞中大小群妖，點將起來，千中選百，百中選十，十中只選三個...

邓俊辉

deng@tsinghua.edu.cn

锦标赛树/Tournament Tree

❖ 完全二叉树

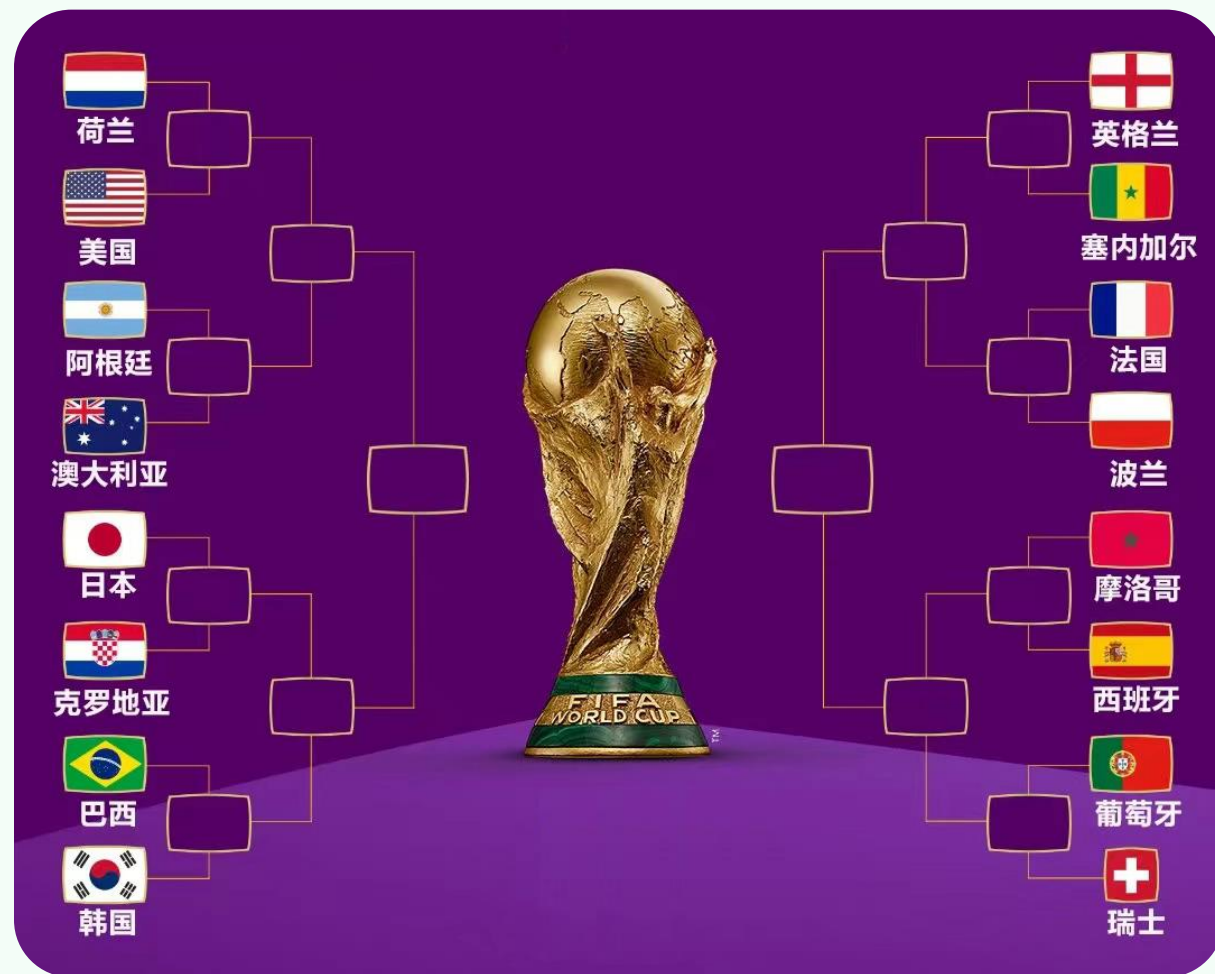
- 叶节点: 参赛选手/团队
 - 内部节点: 孩子对决后之胜者
- 可能有重复 (连胜)

❖ `create()` $O(n)$

`remove()` $O(\log n)$

`insert()` $O(\log n)$

❖ 树根总是全局冠军: 类似于堆顶

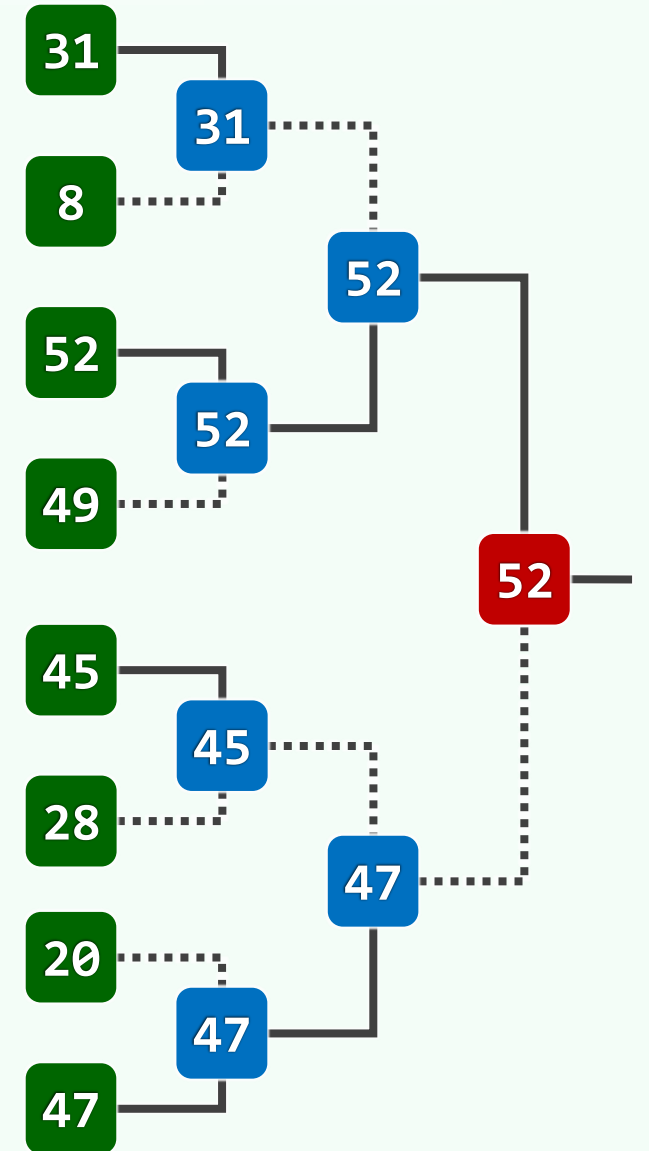


Tournamentsort()

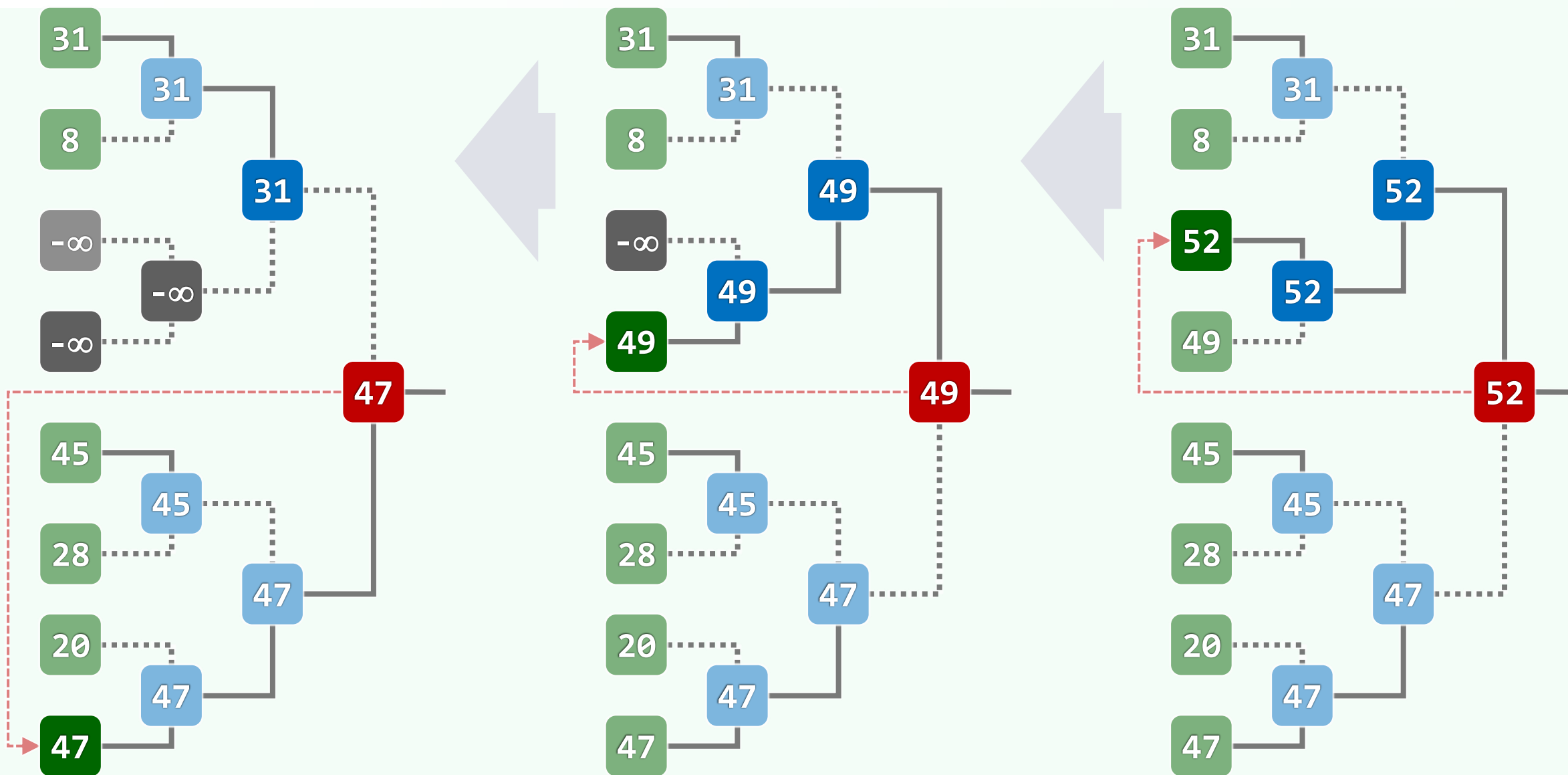
CREATE a tournament tree for the input list

while there are active leaves

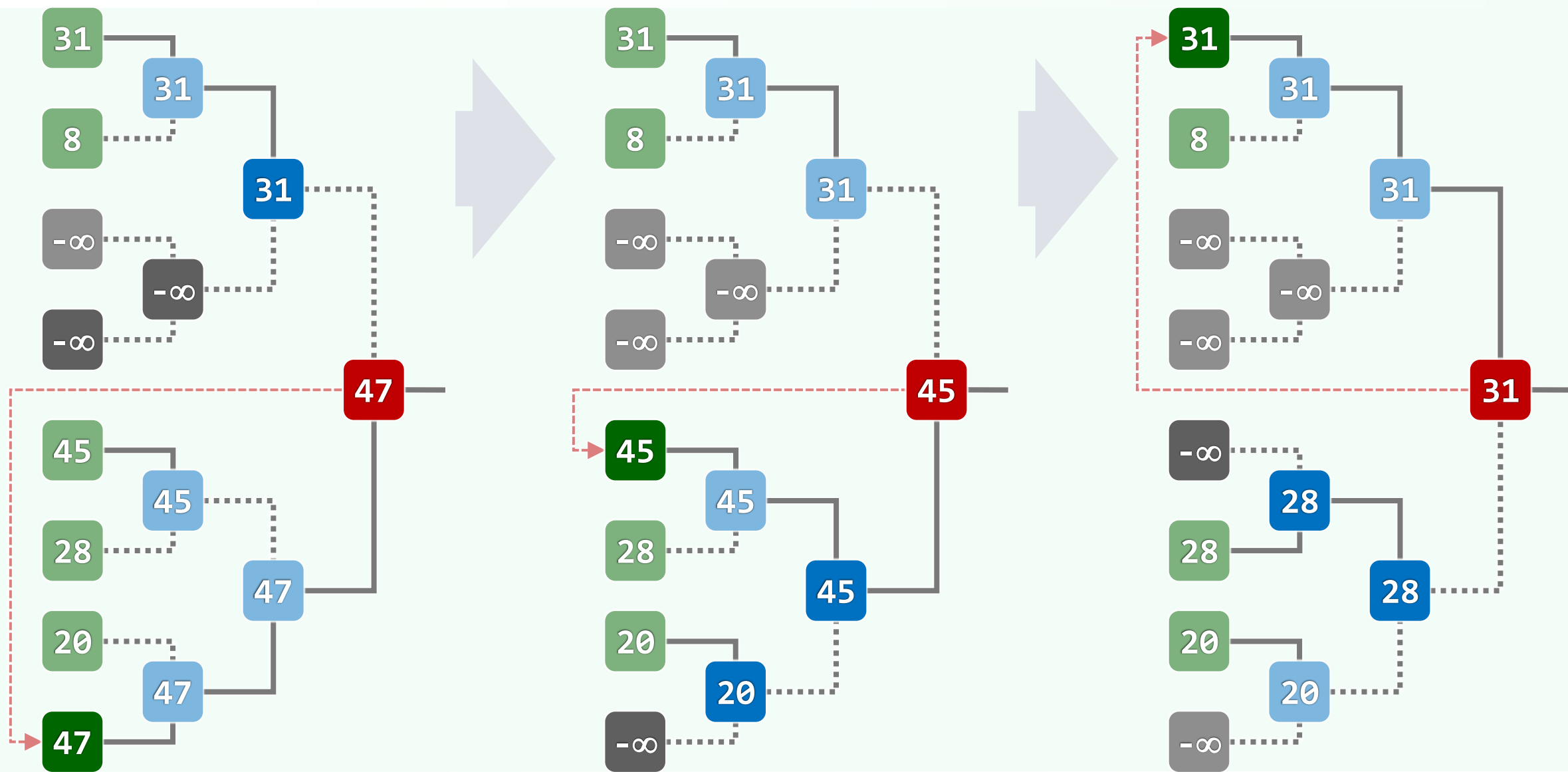
- **REMOVE** the root
- **RETRACE** the root down to its leaf
- **DEACTIVATE** the leaf
- **REPLAY** along the path back to the root



实例 (1/2)

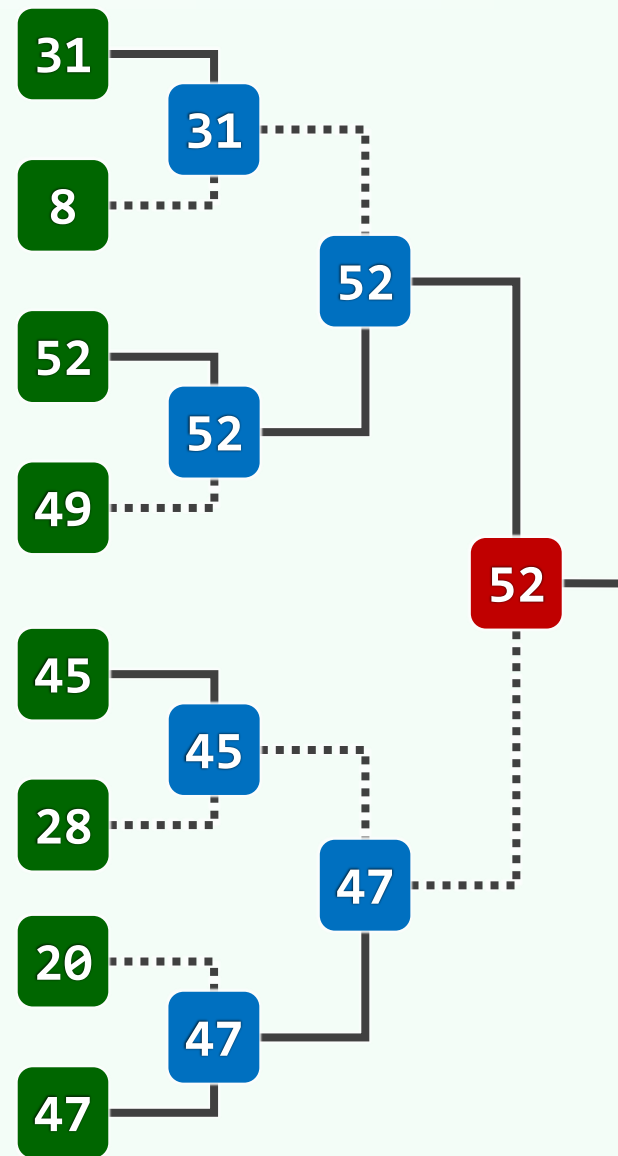


实例 (2/2)



效率

- ❖ 空间: $O(\text{节点数}) = O(\text{叶节点数}) = O(n)$
- ❖ 构造: 仅需 $O(n)$ 时间
- ❖ 更新: 每次都须全体重赛 (replay) ?
只有优胜者的祖先们, 才有必要!
- ❖ 为此 只需从其所在叶节点出发, 逐层上溯直到树根
如此 为确定各轮优胜者, 总共所需时间仅 $O(\log n)$
- ❖ 时间: $n \text{ 轮} \times O(\log n) = O(n \log n)$, 达到下界



不完全选取

❖ 借助锦标赛树，从 n 个元素中找出最小的 k 个， $k \ll n$

- 初始化: $O(n)$ // $n-1$ 次比较
- 迭代 k 步: $O(k \cdot \log n)$ // 每步 $\log n$ 次比较

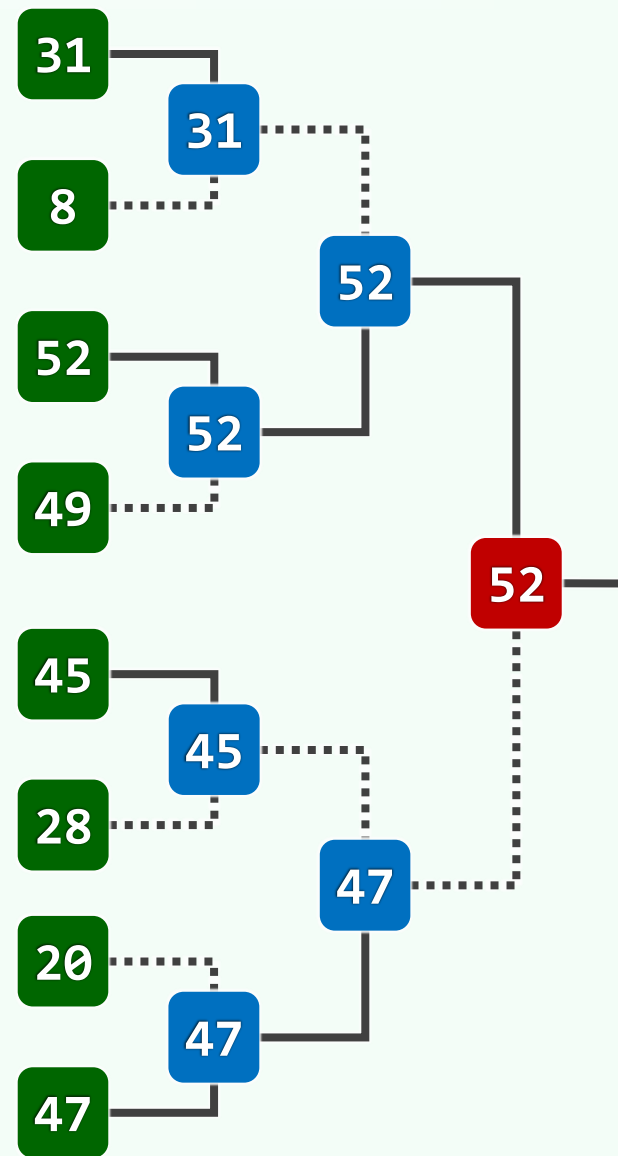
与小顶堆旗鼓相当?

❖ 渐近意义上，的确如此

但就常数而言，区别不小...

❖ Floyd算法、delMax()中的percolateDown()

在每一层需做2次比较，累计大致 $2 \cdot \log n$ 次



12-C2

优先级队列

锦标赛树：败者树

We are the champions.

No time for losers

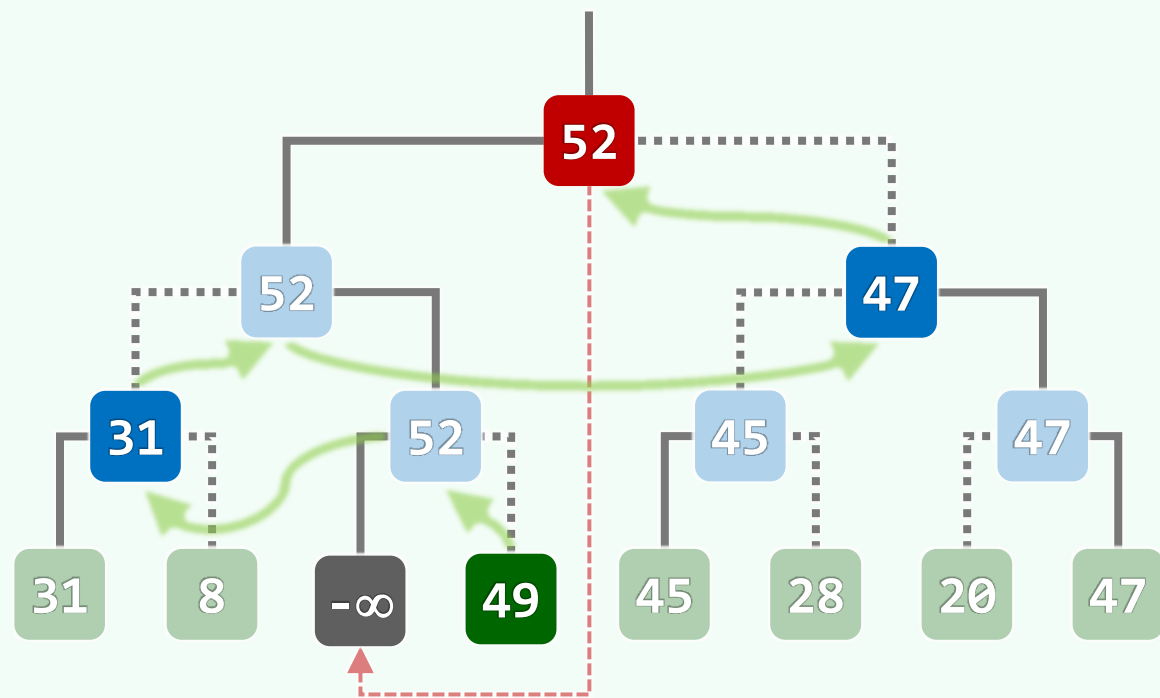
'Cause we are the champions of the world

善勝者不陣，善陣者不戰，善戰者不敗，善敗者終勝

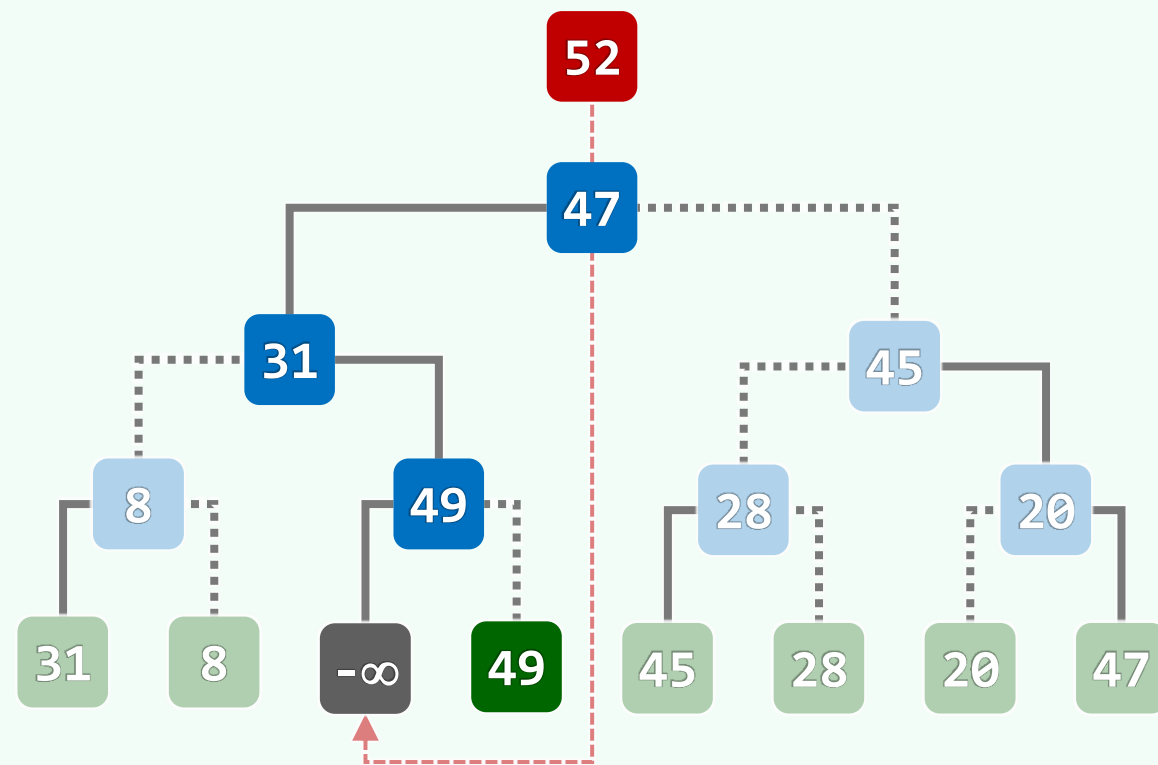
邓俊辉

deng@tsinghua.edu.cn

结构 + 重赛算法

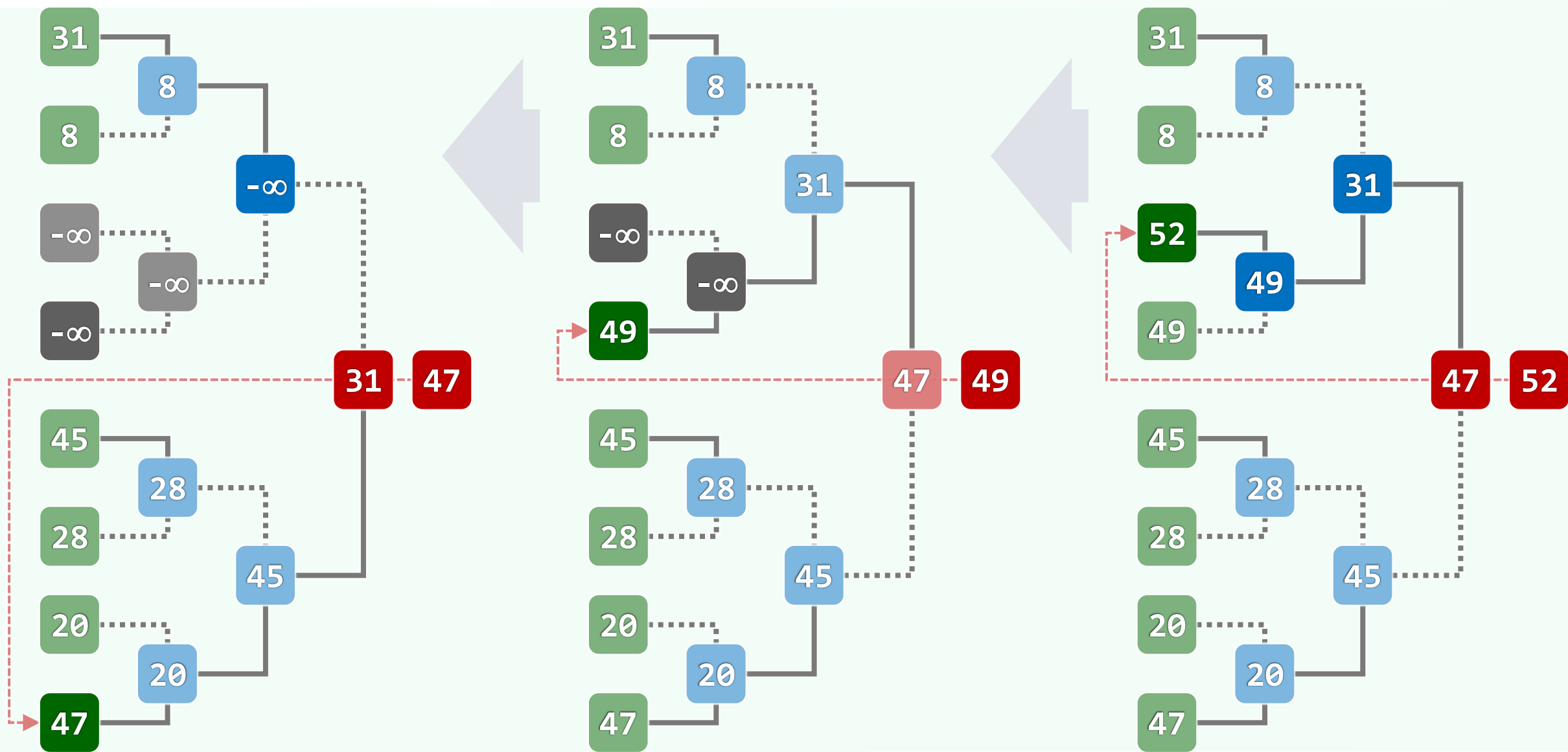


- ❖ 重赛过程中，须交替访问沿途节点及其兄弟
- 如何避免这类迂回？

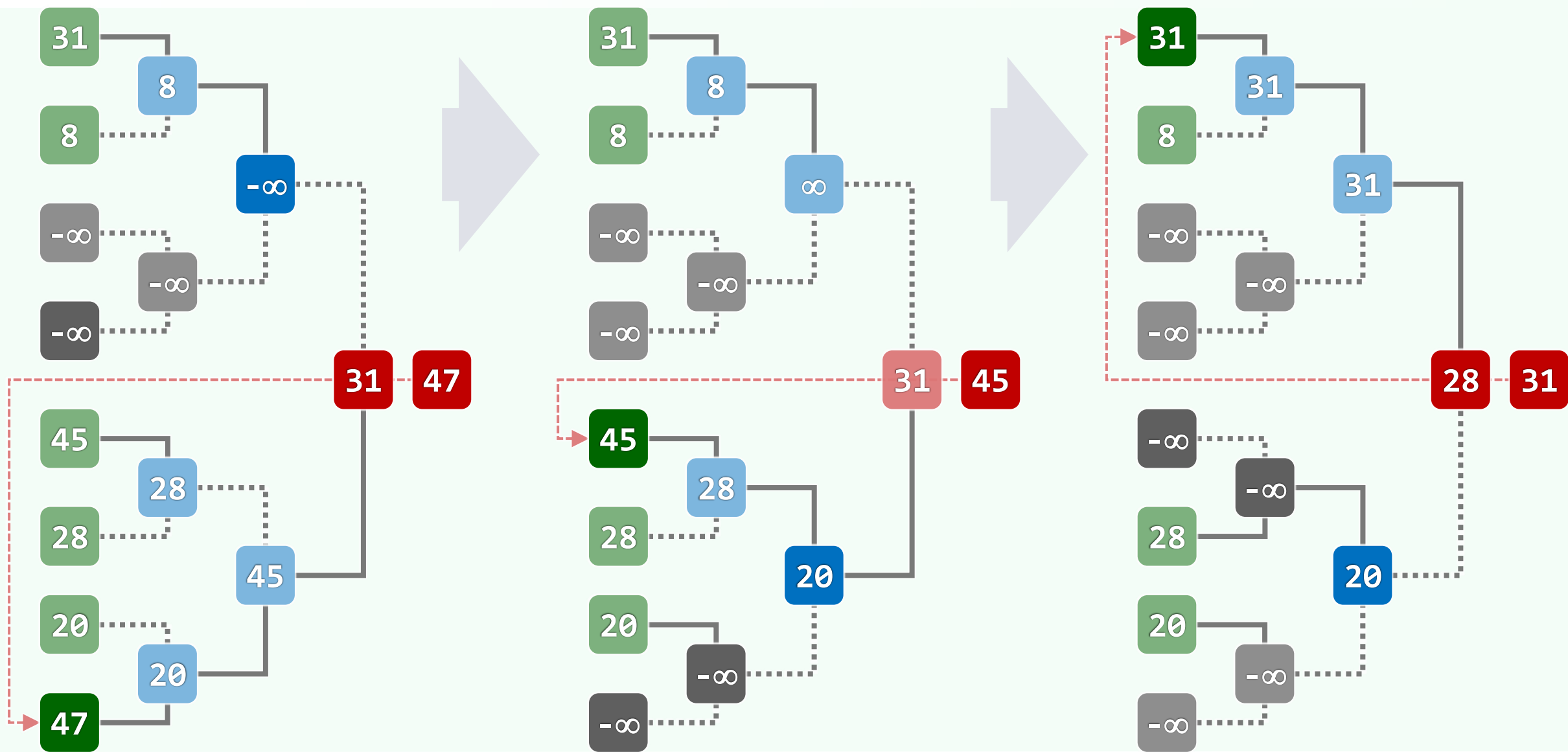


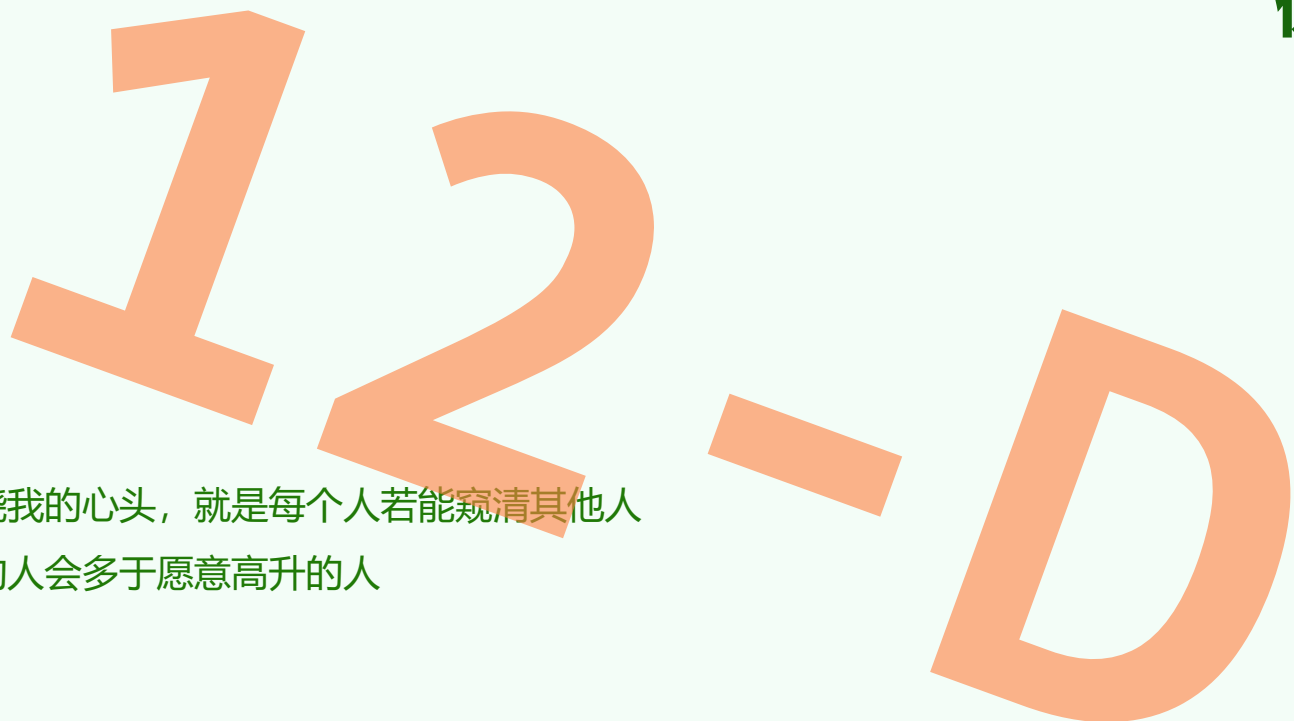
- ❖ 内部节点，记录对应比赛的败者
- 增设根的“父节点”，记录冠军

实例 (1/2)



实例 (2/2)





优先级队列

多叉堆

他说到一件事，时常萦绕我的心头，就是每个人若能窥清其他人的心意，那么愿意下来的人会多于愿意高升的人

飽有苦葉，濟有深涉

深則厲，淺則揭

邓俊辉

deng@tsinghua.edu.cn

优先级搜索

- ❖ 回顾图的PFS以及统一框架: $g \rightarrow \text{pfs}() \dots$
- ❖ 无论何种算法, 差异仅在于所采用的优先级更新器prioUpdater()
 - Prim算法: $g \rightarrow \text{pfs}(\emptyset, \text{PrimPU}());$
 - Dijkstra算法: $g \rightarrow \text{pfs}(\emptyset, \text{DijkPU}());$
- ❖ 每一节点引入遍历树后, 都需要
 - 更新树外顶点的优先级 (数) , 并
 - 选出新的优先级最高者
- ❖ 若采用邻接表, 两类操作的累计时间, 分别为 $\mathcal{O}(n + e)$ 和 $\mathcal{O}(n^2)$...能否更快呢?

优先级队列

❖ 自然地，PFS中的各顶点可组织为**优先级队列**

❖ 为此需要使用PQ接口

heapify() ~ percolateDown() : 由 n 个顶点创建初始PQ, $\mathcal{O}(n)$

delMax() ~ percolateDown() : 取优先级最高（极短）跨边 (u, w) , $\mathcal{O}(n \cdot \log n)$

increase() ~ percolateUp() : 更新（缩短）所有关联顶点到 u 的距离, $\mathcal{O}(e \cdot \log n)$

❖ 总体运行时间 = $\mathcal{O}((n + e) \cdot \log n)$

- 对于**稀疏图**，处理效率很高
- 对于**稠密图**，反而不如常规实现的版本

❖ 有无更好的办法？

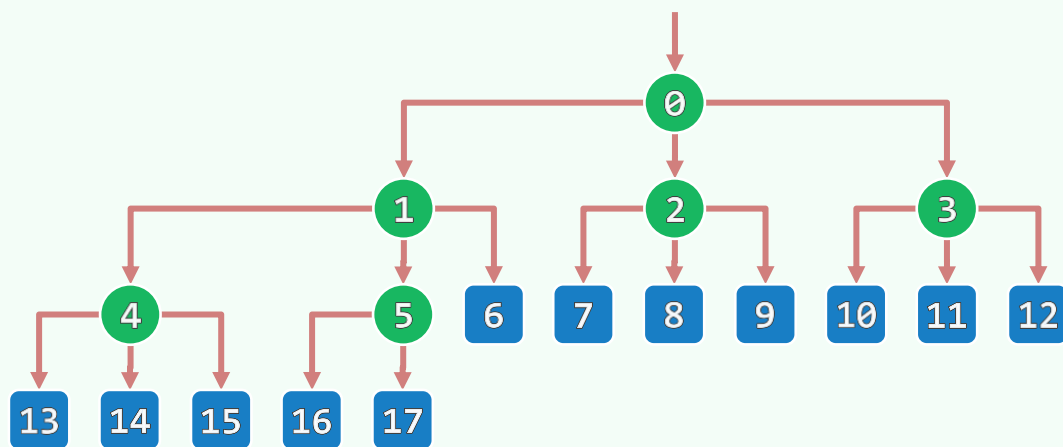
多叉堆 = 多叉完全树 + 堆序性

❖ 仍可基于向量实现，且

父、子节点的**秩**可简明地相互换算

- $parent(k) = \lfloor (k - 1) / d \rfloor$
- $child(k, i) = k \cdot d + i, 0 < i \leq d$

//d未必是2的幂，不能借助**移位**运算加速



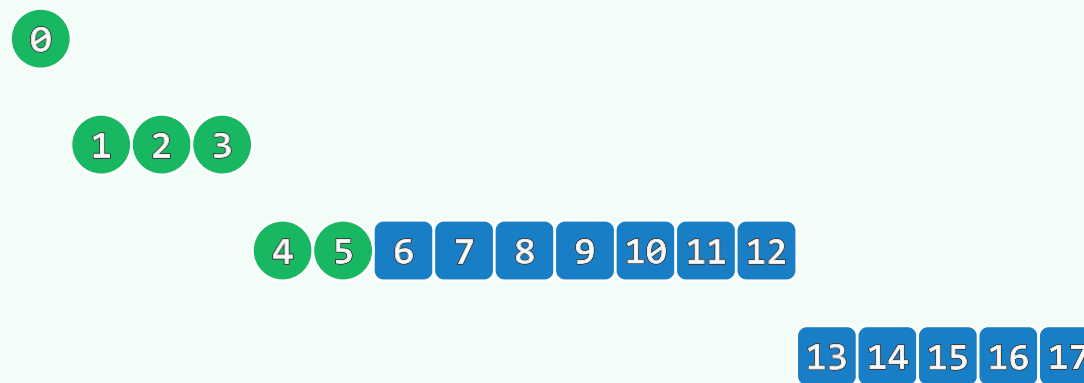
❖ heapify(): $O(n)$ //不可能再快了

❖ delMax(): $O(\log n)$

//实质即percolateDown()，已是**极限**

❖ increase(): $O(\log n)$

//实质即percolateUp() —— 似乎**仍可**改进



上山容易，下山难

❖ 若将**二**叉堆改成**多**叉堆 (d-heap)

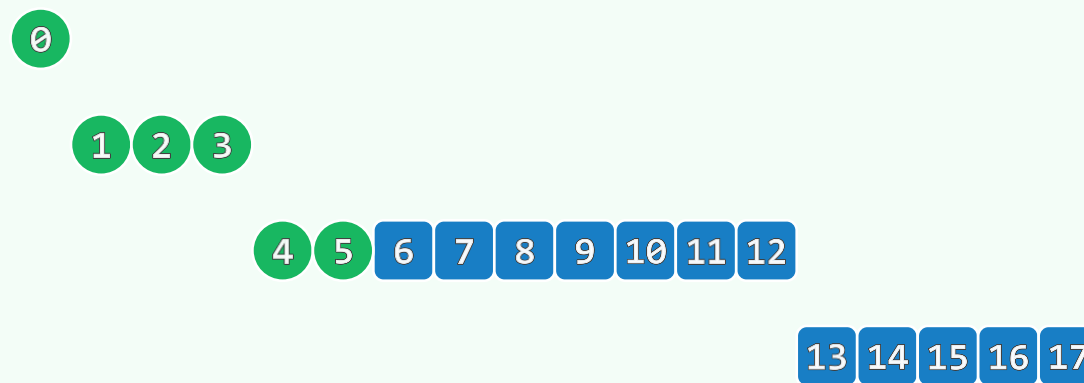
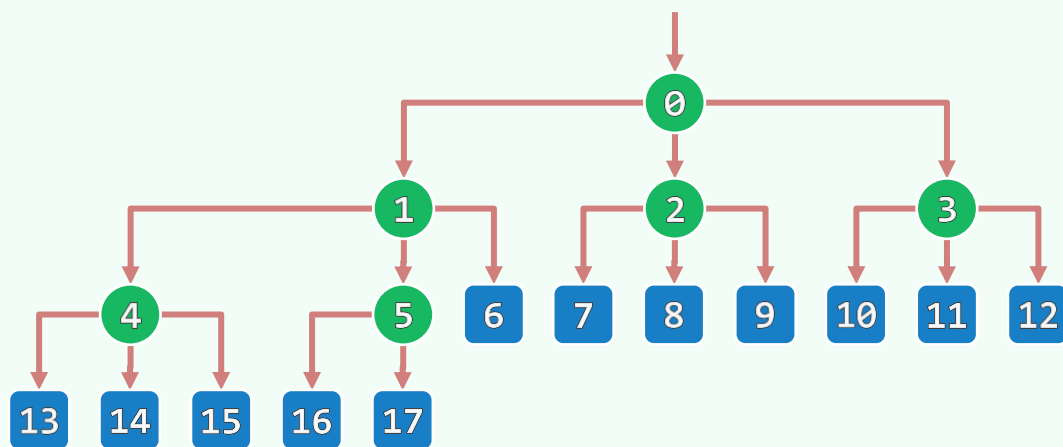
则堆高降至 $\log_d n$

❖ 相应地，**上**滤成本**降**至 $\log_d n$

❖ 但 (只要 $d > 4$) **下**滤成本却**增**至

$$d \cdot \log_d n = \frac{d}{\ln d} \cdot \ln n$$

❖ 综合考量，**利**大于**弊**...

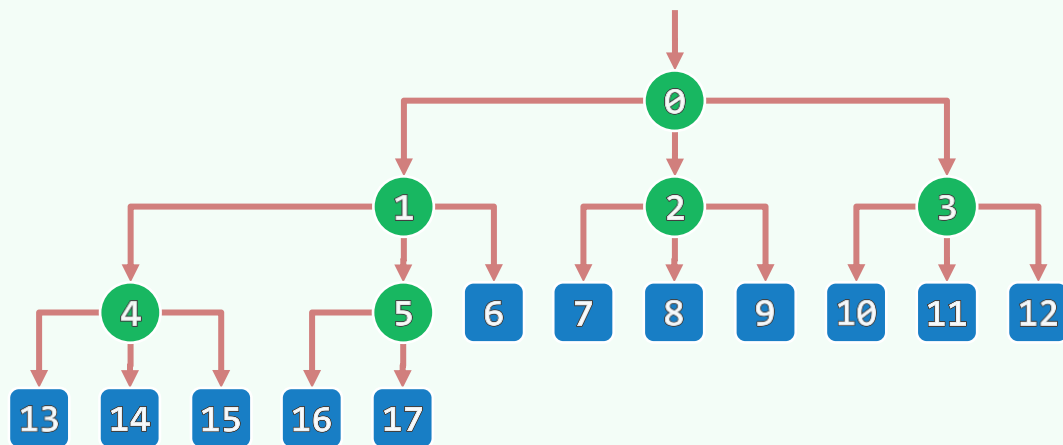


❖ 如此，PFS的运行时间将是：

$$\begin{aligned} & n \cdot d \cdot \log_d n + e \cdot \log_d n \\ &= (n \cdot d + e) \cdot \log_d n \end{aligned}$$

❖ 取 $d \approx e/n + 2$ 时

总体性能达到最优： $\mathcal{O}(e \cdot \log_{(e/n+2)} n)$



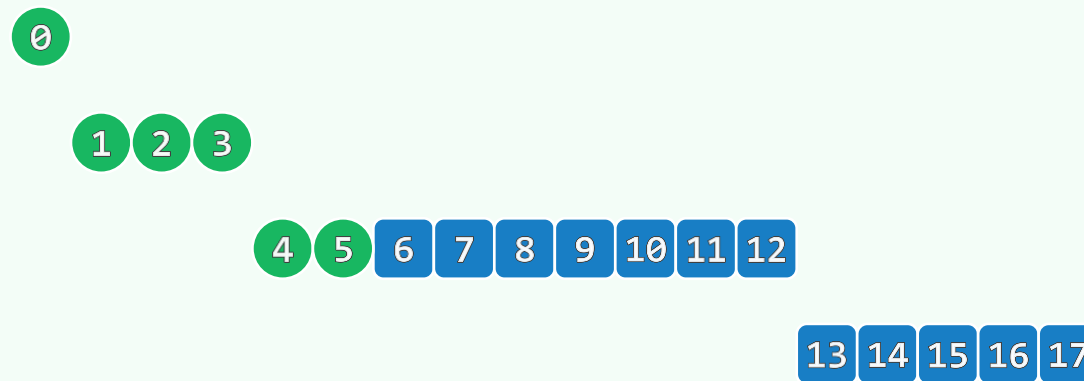
❖ 对于**稀疏**图保持高效：

$$\approx n \cdot \log_{(n/n+2)} n = \mathcal{O}(n \log n)$$

❖ 对于**稠密**图改进极大：

$$\approx n^2 \cdot \log_{(n^2/n+2)} n \approx n^2 = \mathcal{O}(e)$$

❖ 对于一般的图，会**自适应**地实现最优



12-E1

优先级队列

左式堆：沿藤合并

God's right hand is gentle
But terrible is his left hand

邓俊辉

deng@tsinghua.edu.cn

堆合并

❖ $H = \text{merge}(A, B)$: 将堆A和B合二为一

//不妨设 $|A| = n \geq m = |B|$

❖ 方法一: $A.\text{insert}(B.\text{delMax}())$

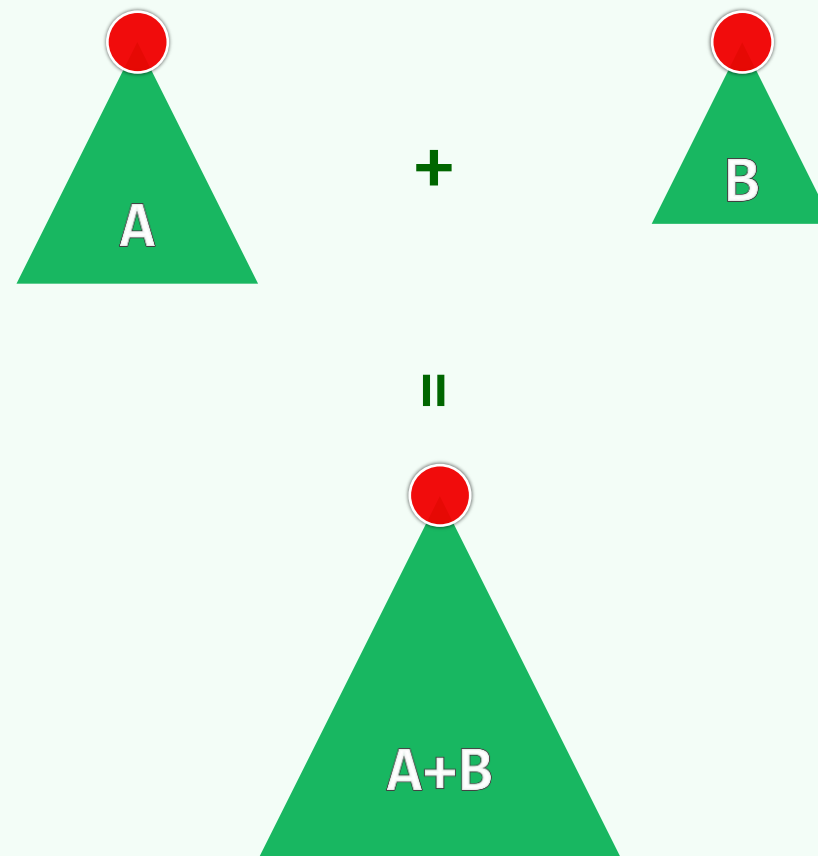
$$m \cdot (\log m + \log(n + m)) = \mathcal{O}(m \cdot \log(n + m))$$

❖ 方法二: $\text{union}(A, B).\text{heapify}(n + m)$

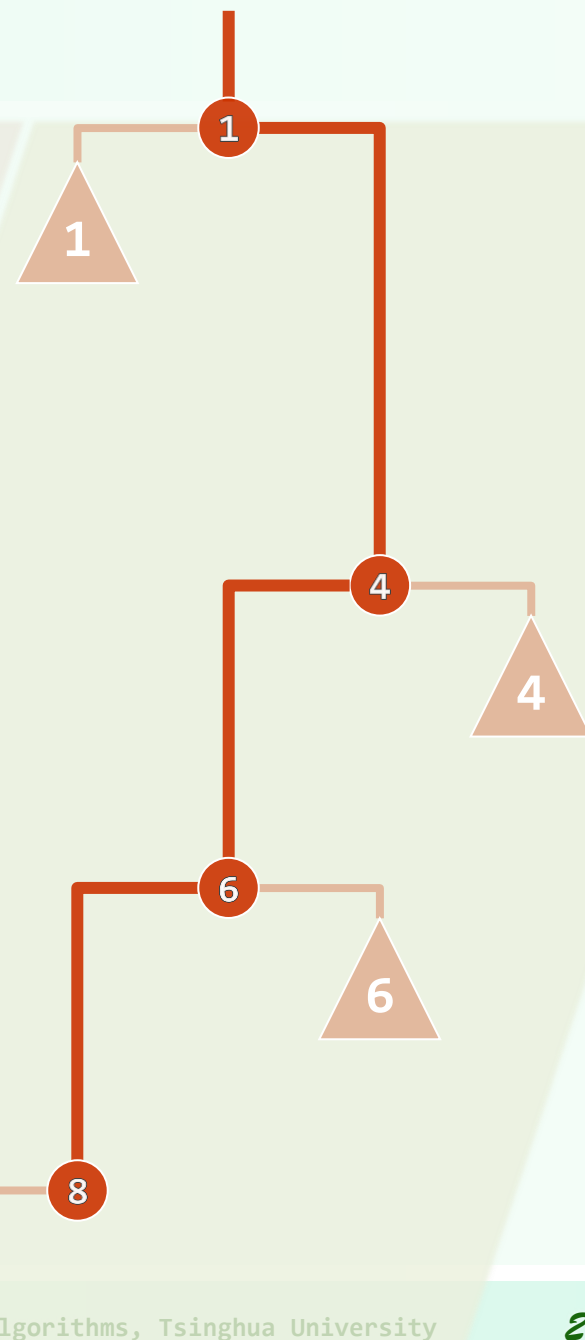
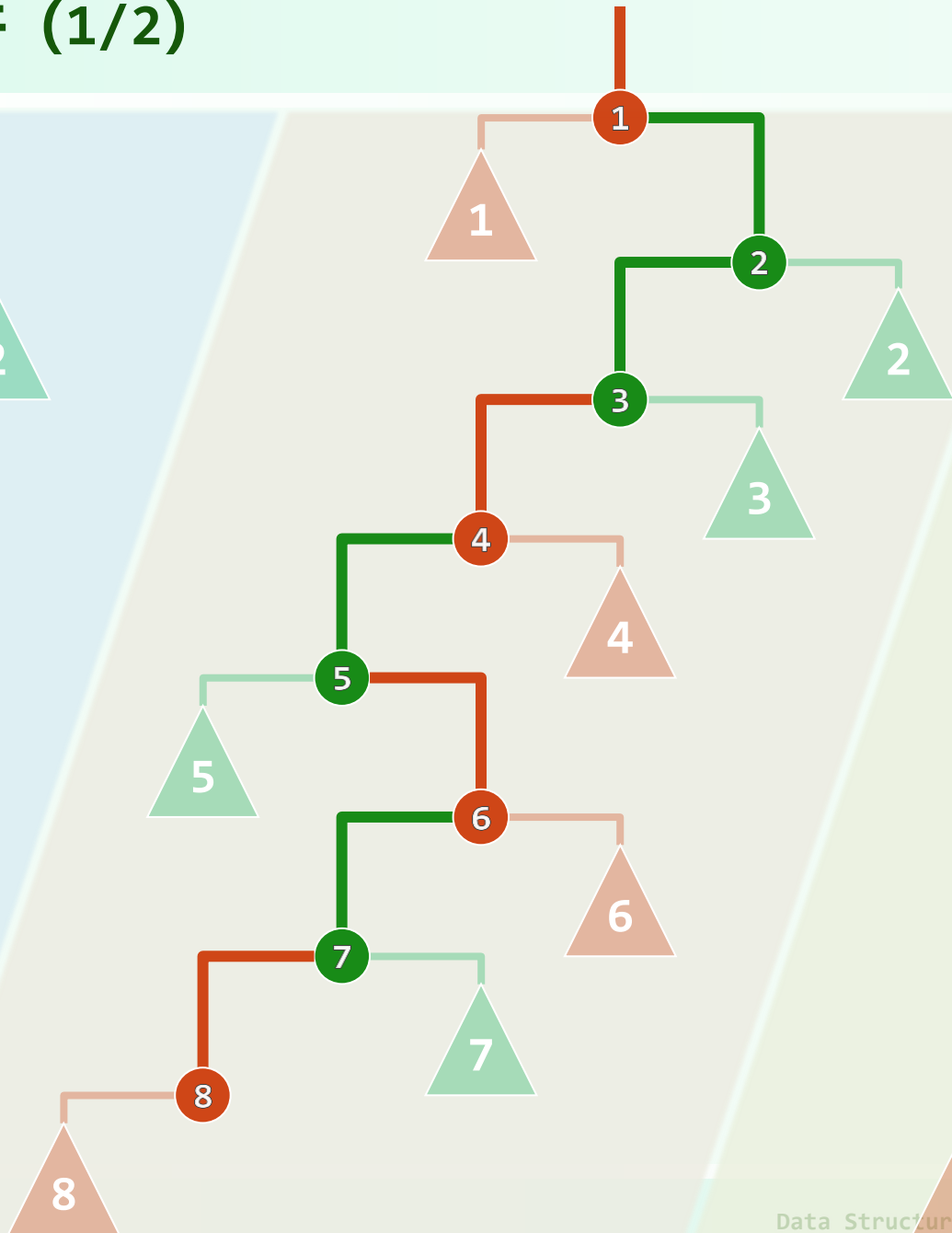
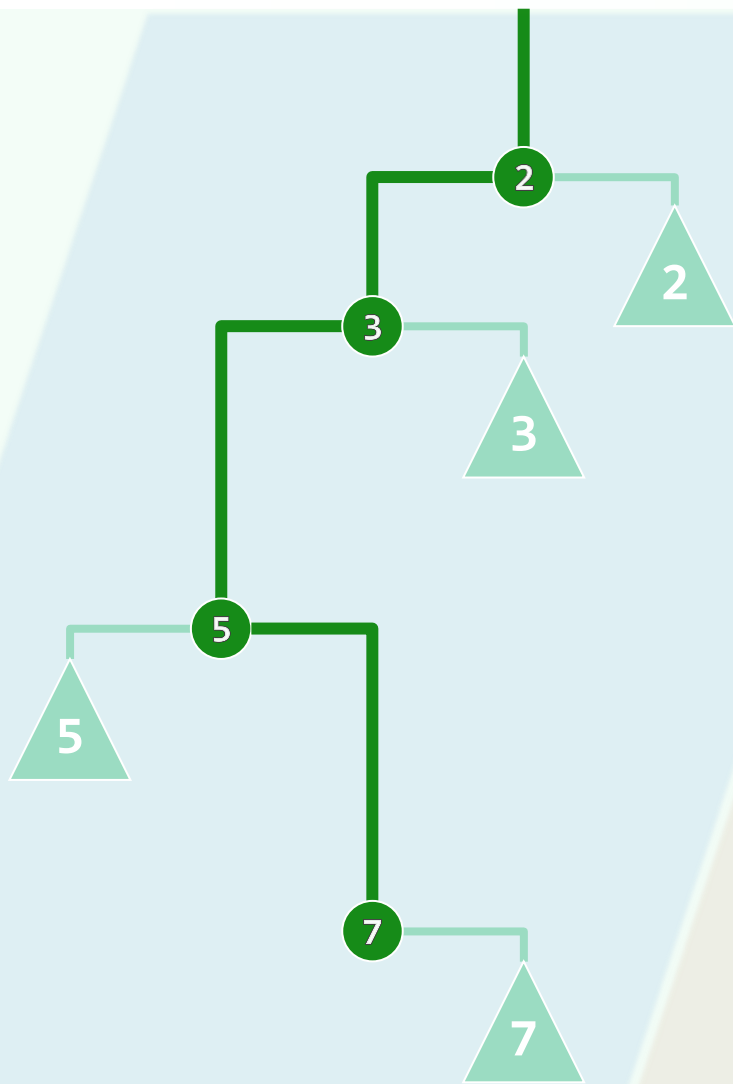
$$\mathcal{O}(n + m)$$

❖ 有没有更好的办法? 比如...

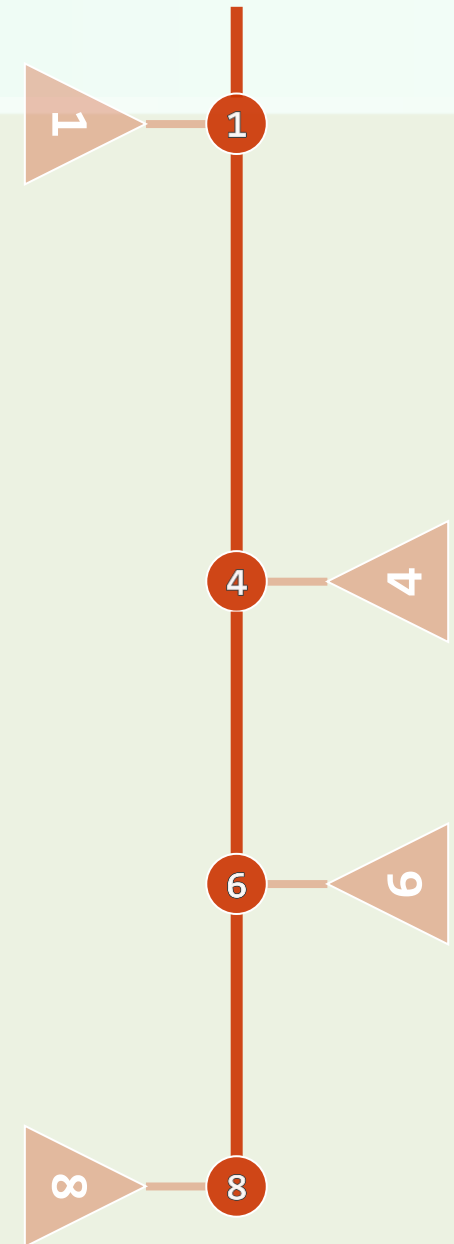
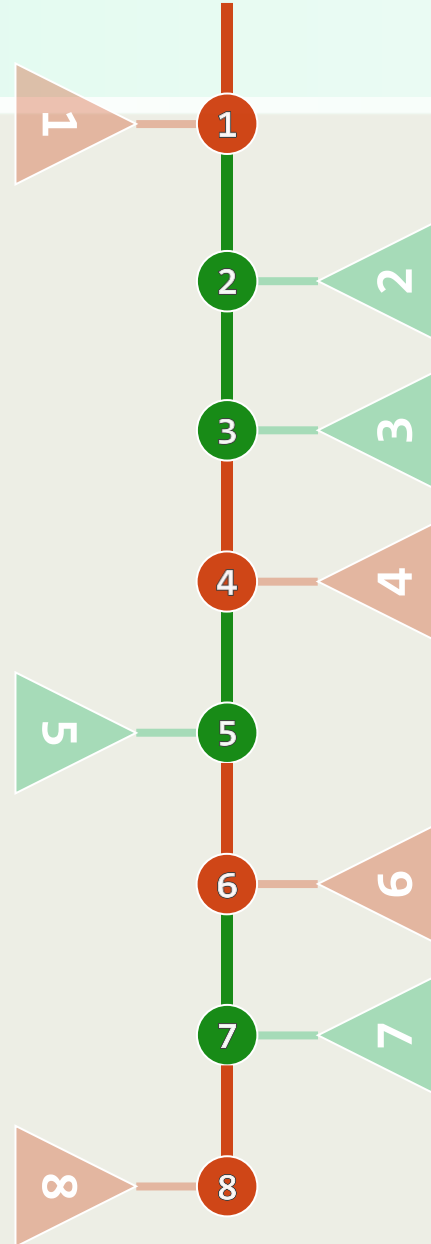
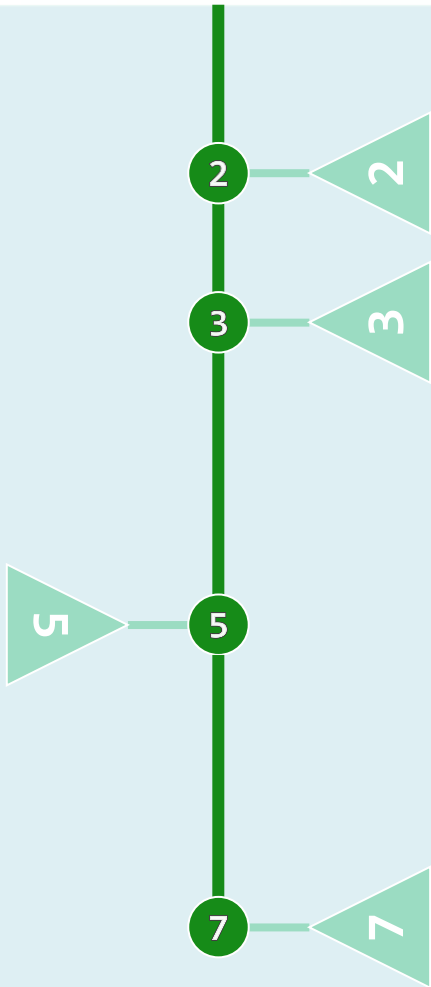
❖ 可否奢望在... $\mathcal{O}(\log n)$...时间内实现 $\text{merge}()$?



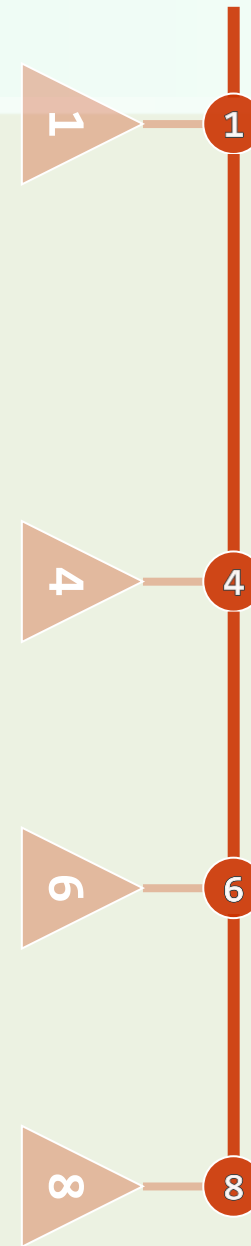
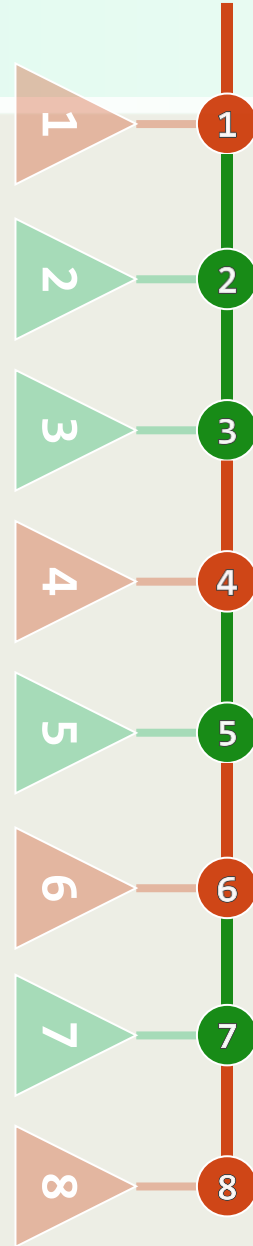
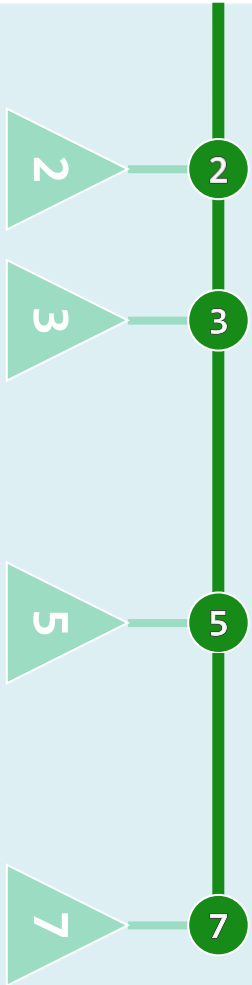
两堆合并 = 二路归并 (1/2)



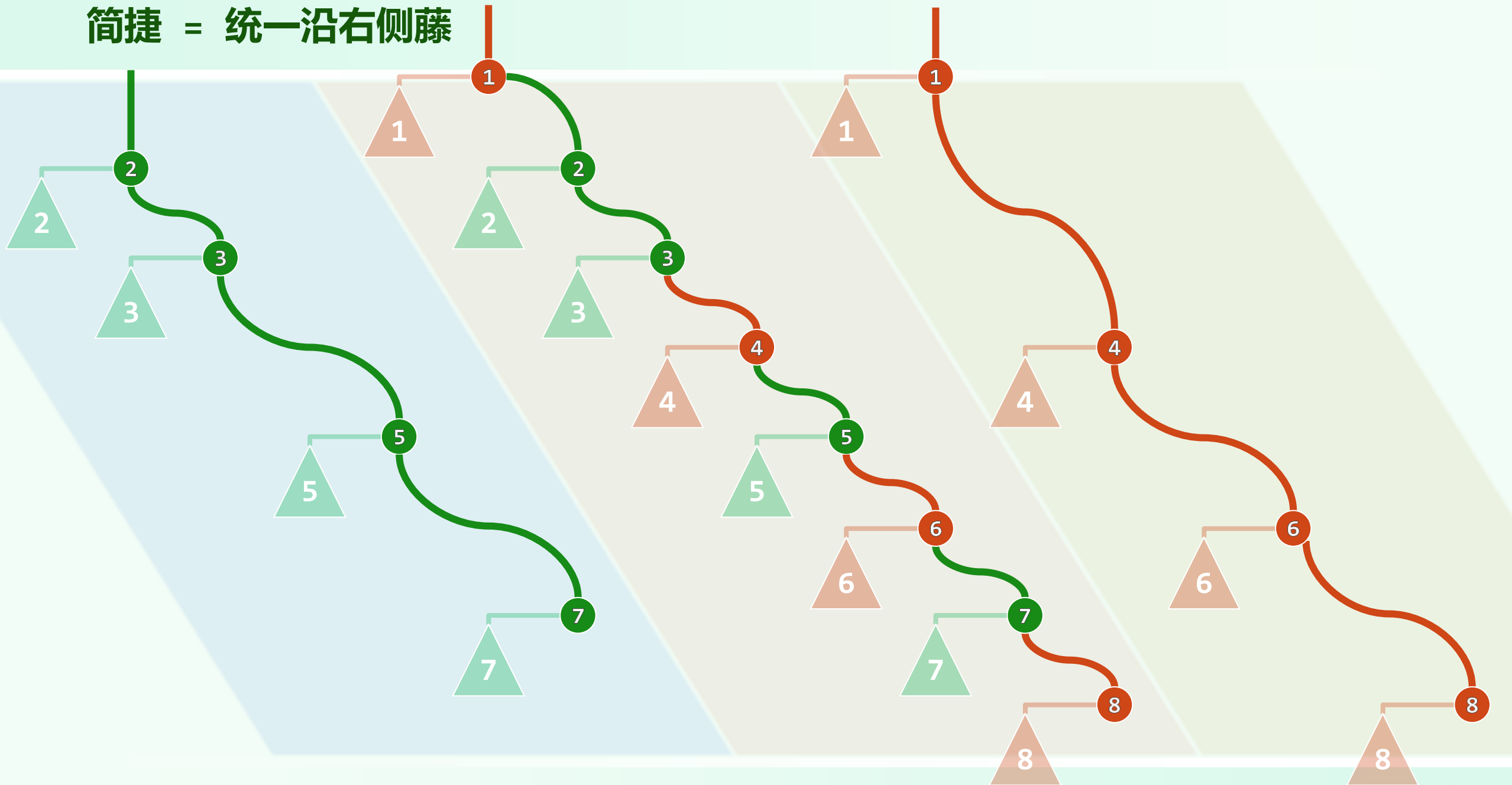
两堆合并 = 二路归并 (2/2)



简捷 = 统一沿右侧藤



简捷 = 统一沿右侧藤



LeftHeap = PQ + BinTree

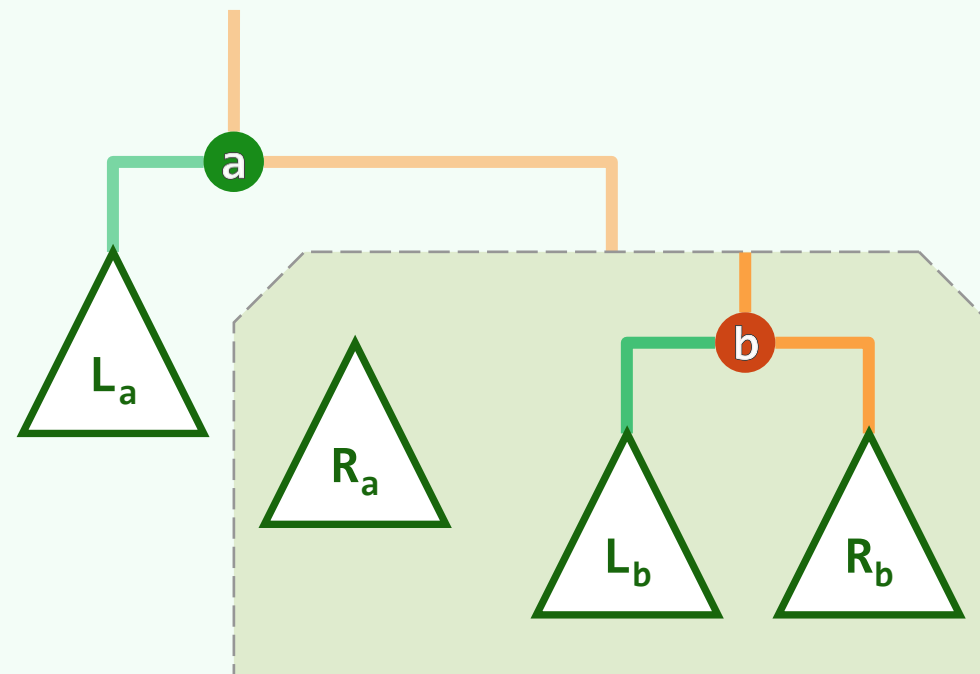
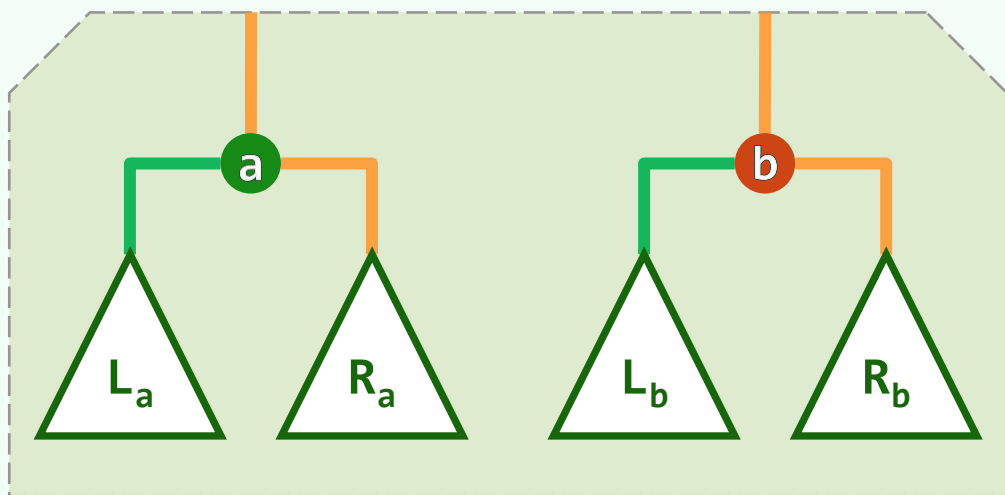
```
template <typename T> struct PQ_LeftHeap : public PQ<T>, public BinTree<T> {  
    T& getMax() { return _root->data; }  
    void insert(T); T delMax(); //均基于统一的合并操作实现...  
    PQ_LeftHeap( PQ_LeftHeap& A, PQ_LeftHeap& B ) {  
        _root = merge( A._root, B._root ); _size = A._size + B._size;  
        A._root = B._root = NULL; A._size = B._size = 0;  
    }  
}; //PQ_LeftHeap  
  
template <typename T> BNP<T> merge( BNP<T>, BNP<T> );
```

递归实现

所需时间 $\propto$ 右侧藤总长

~~$O(n)$~~

$O(\log n)$



如何...控制藤长
以...**持续**高效地合并?

12-E2

优先级队列

左式堆：NPL与控制藤长

邓俊辉

deng@tsinghua.edu.cn

君子居則貴左，用兵則貴右

可持续 = 单侧倾斜

❖ C. A. Crane, 1972:

保持**堆序性**，附加**新条件**，使得

在堆合并过程中，只涉及**少量节点**: $O(\log n)$

❖ 新条件 = 单侧**倾斜**

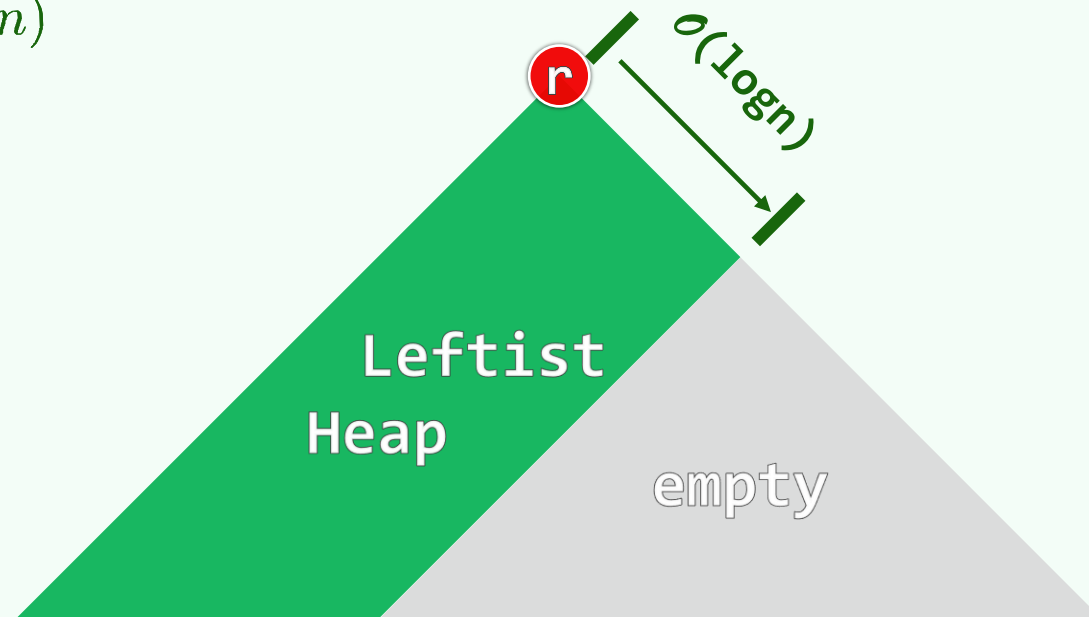
节点**分布**偏向于**左侧**

合并操作只涉及**右侧**

❖ 可是，果真如此，则拓扑上...

不见得是**完全**二叉树，说好的**结构性**呢？

❖ 实际上，结构性并非堆结构的**本质**要求



空节点路径长度

❖ 引入所有的外部节点

消除一度节点，转为真二叉树

❖ Null Path Length

```
#define npl height //假借
```

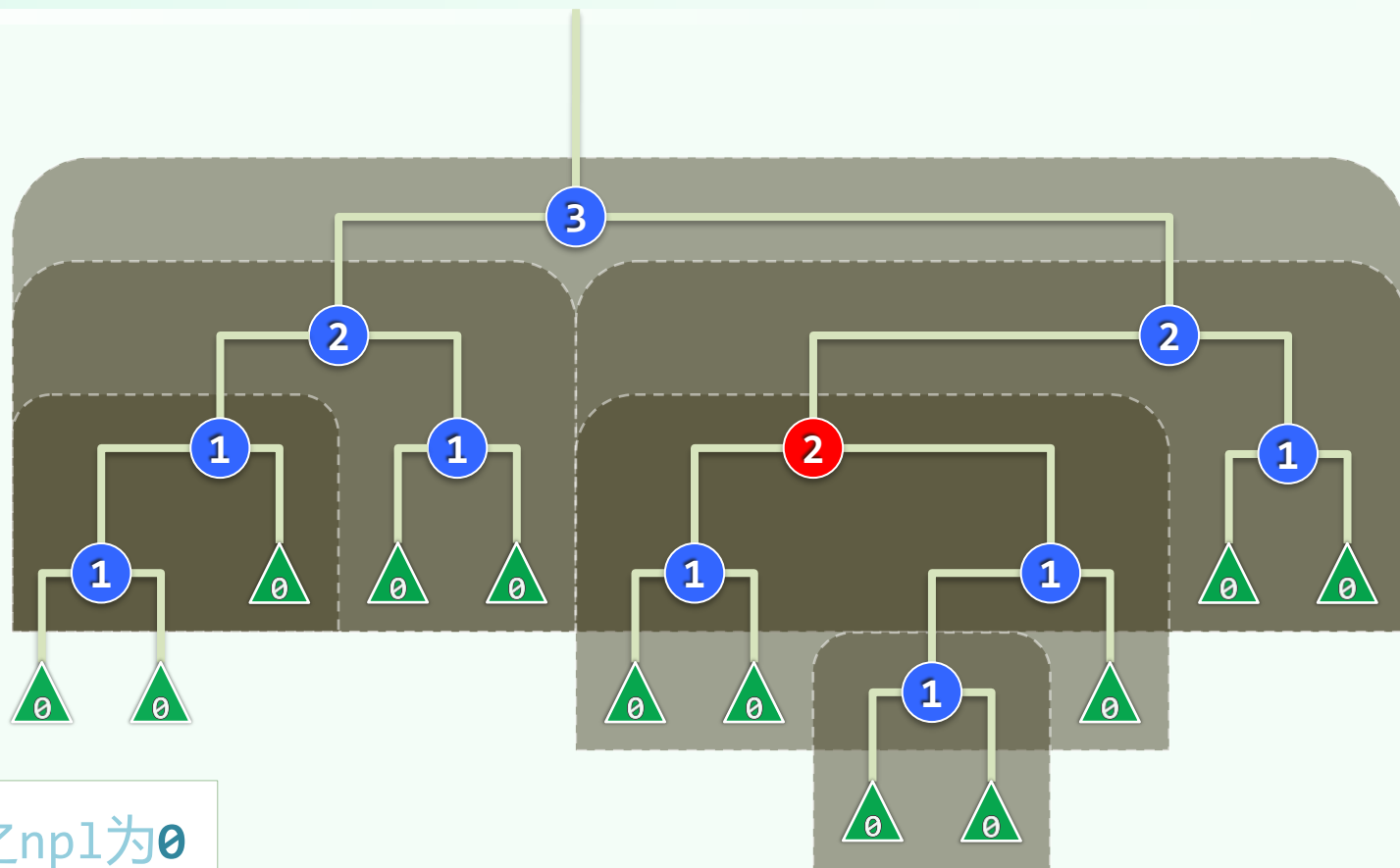
```
#define NPL(x) Height(x)
```

- $npl(NULL) = 0$

```
#define H_NULL 0 //空左式堆之npl为0
```

- $npl(x) = 1 + \min\{ npl(lc(x)), npl(rc(x)) \}$

❖ 验证: $npl(x) = x$ 到外部节点的最近距离 = 以 x 为根的最大满“子”树的高度



左式堆 = 处处左倾

❖ 对任何内节点x，都有：

$$npl(\text{lc}(x)) \geq npl(\text{rc}(x))$$

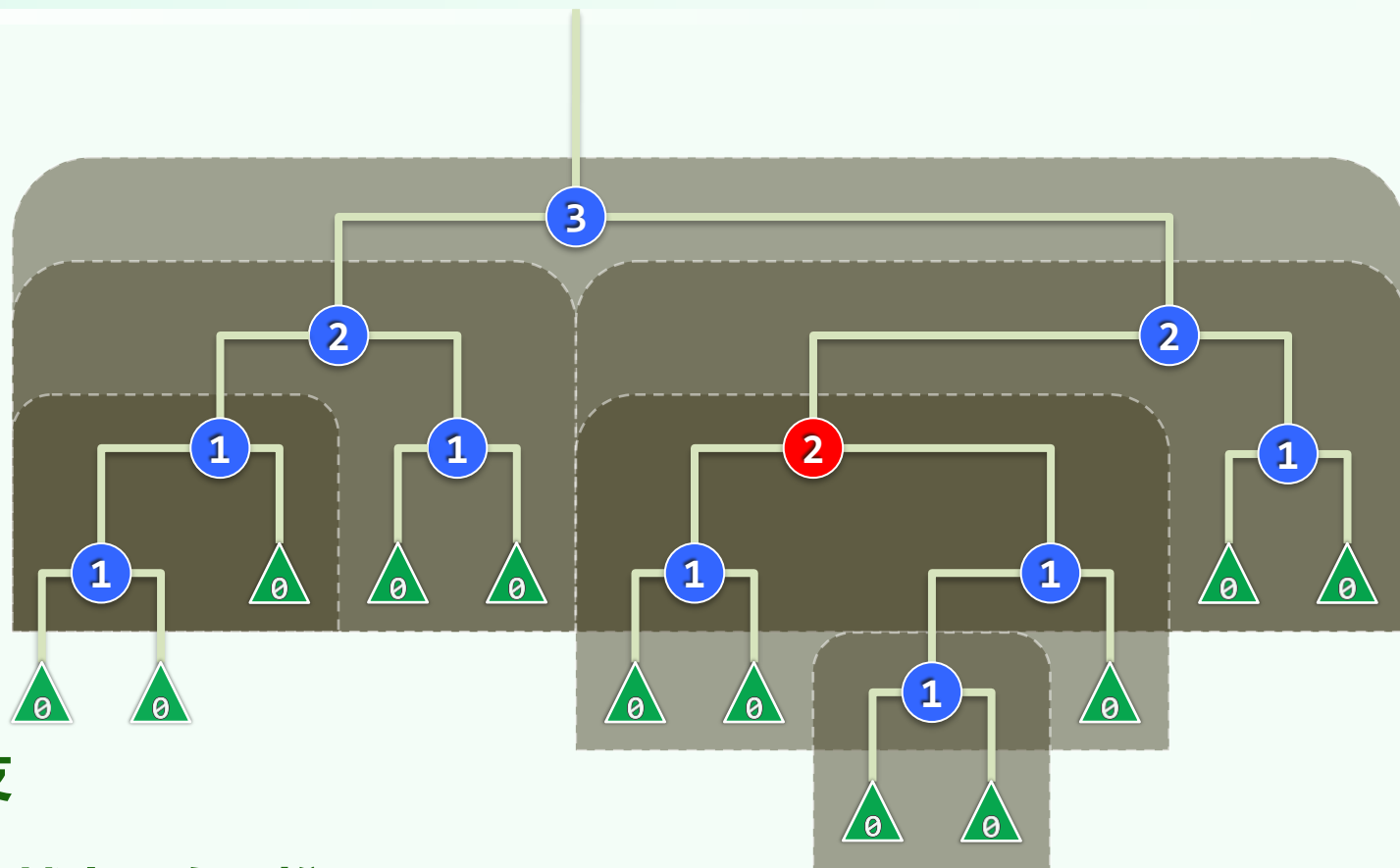
❖ 左倾性与堆序性，**相容而不矛盾**

❖ 左式堆的子堆，**必是左式堆**

❖ 左式堆**倾向于**更多节点分布于左侧分支

这是否意味着，左子堆的**规模**和**高度**必然大于右子堆？

❖ 推论： $npl(x) = 1 + npl(\text{rc}(x))$



$$\text{rpn}(x) \sim |\text{rChain}(x)|$$

❖ 右侧链 $\text{rChain}(x)$: 从 x 出发, 一直沿**右分支**前进

❖ 由左倾性, $\text{npl}(x) = |\text{rChain}(x)| + 1$

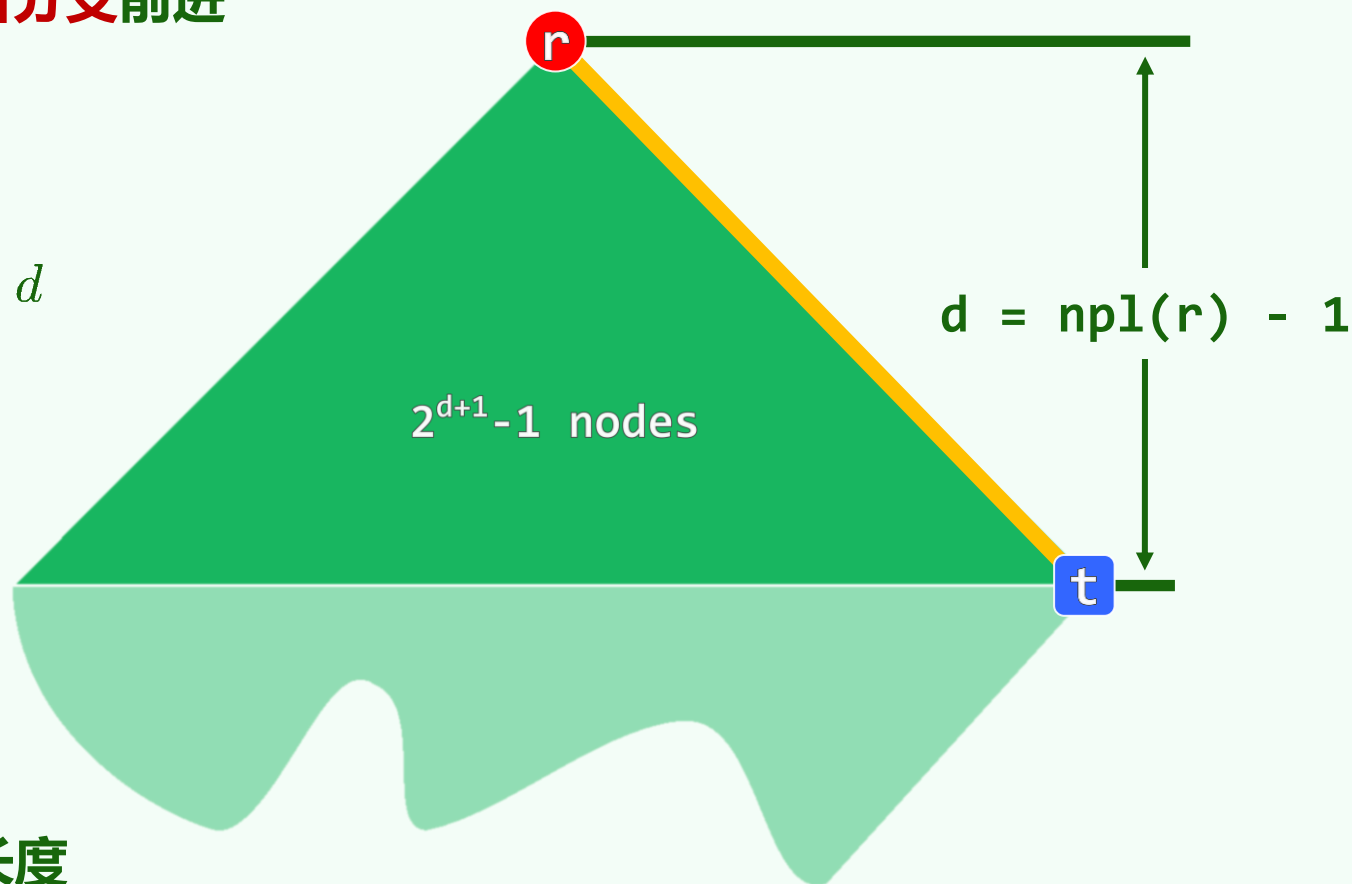
特别地, $\text{npl}(r) - 1 = |\text{rChain}(r)| = d$

❖ 右侧链长度为 d 的左式堆, **至少**包含

- 高度为 d 的**满**“子”树
- $2^{d+1} - 1$ 个节点

❖ 反之, 包含 n 个节点的左式堆, 右侧链长度

$$d \leq \log_2(n + 1) - 1 = \mathcal{O}(\log n)$$



12-E3

优先级队列

左式堆：合并算法

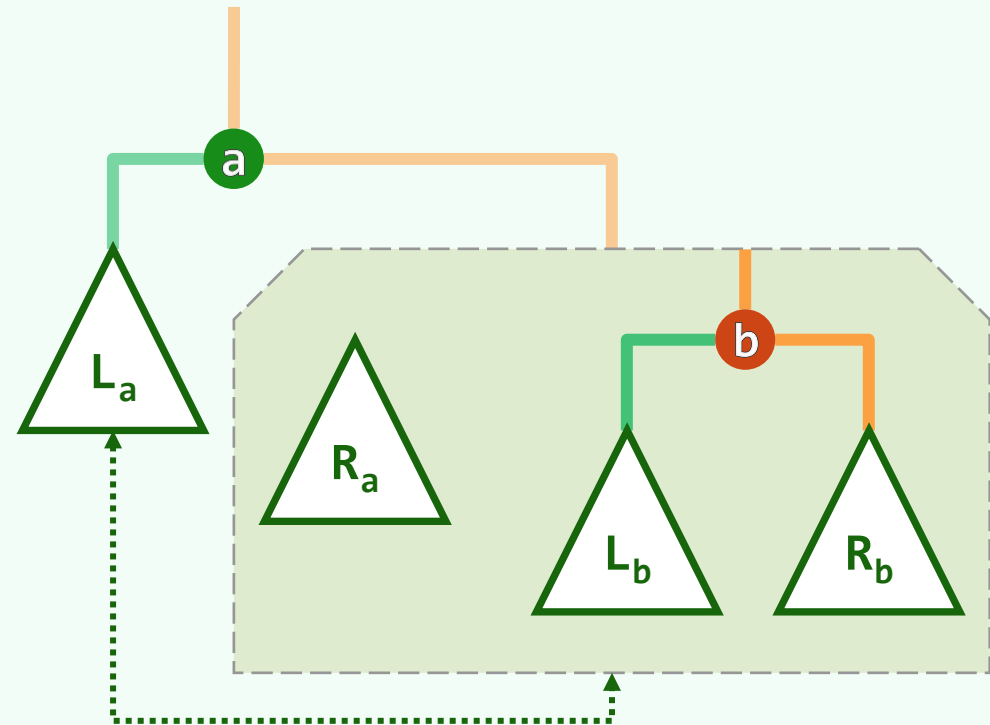
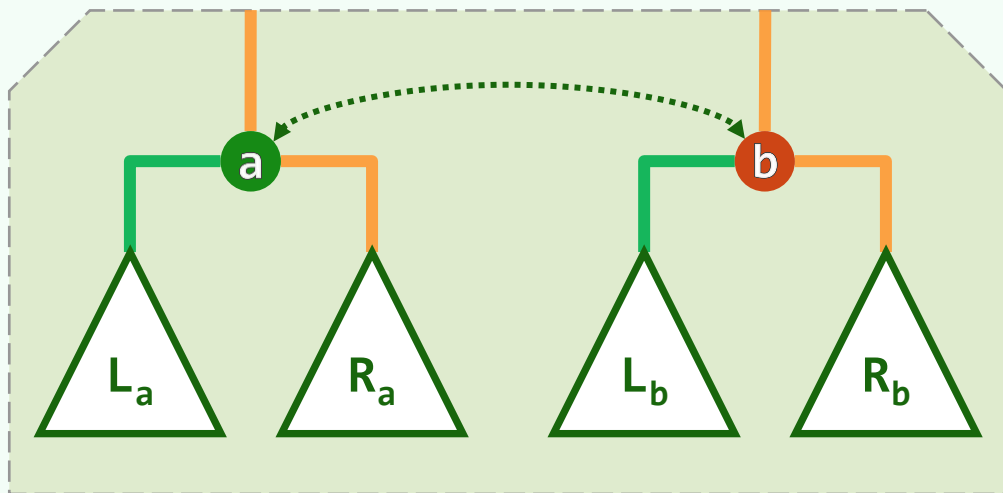
邓俊辉

deng@tsinghua.edu.cn

左之左之，君子宜之；右之右之，君子有之

递归：前处理 + 后处理

Before:
compare priority & swap if necessary

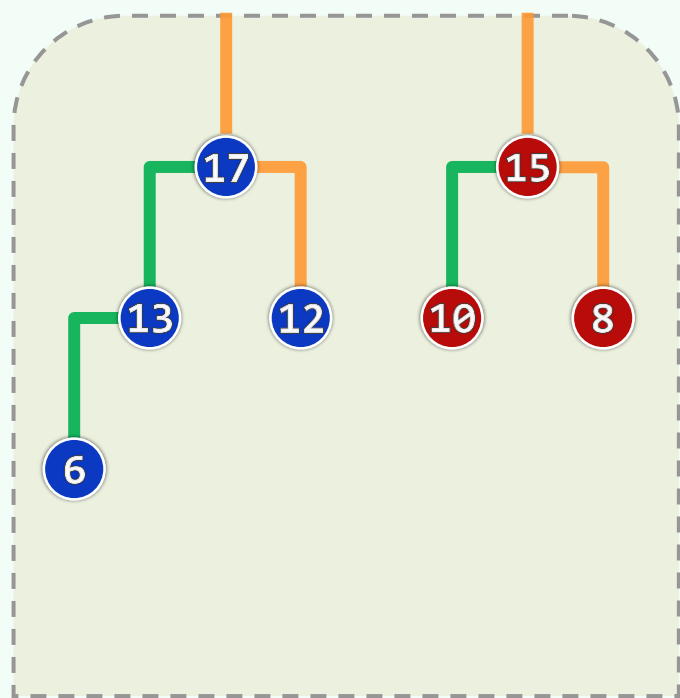


After:
compare NPL & flip if necessary

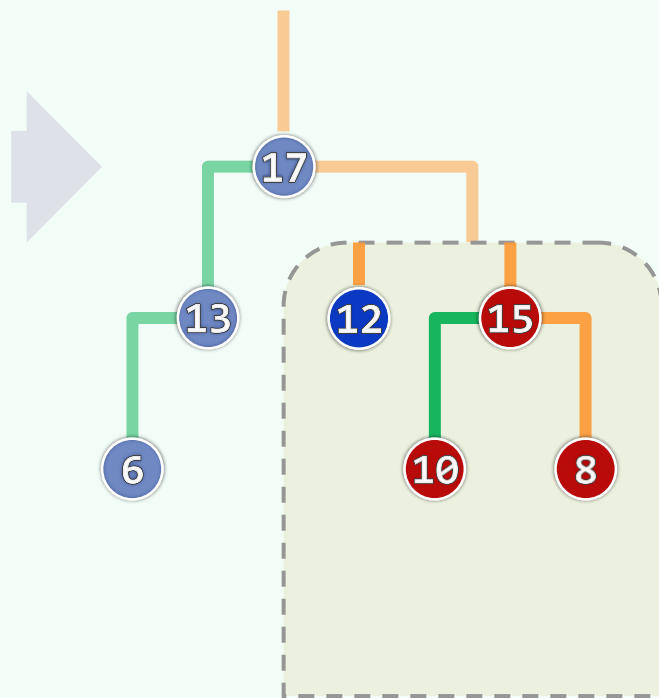
递归实现

```
template <typename T> BNP<T> merge( BNP<T> a, BNP<T> b ) {  
    if ( !a ) return b; if ( !b ) return a; //递归基  
    if ( *a < *b ) swap( a, b ); //确保a>=b: BinNode已重载定义过 "<"  
    ( a->rc = merge( a->rc, b ) )->parent = a; //将a的右子堆, 与b合并  
    if ( NPL(a->lc) < NPL(a->rc) ) //若有必要 (宏隐藏了边界情况)  
        swap( a->lc, a->rc ); //交换a的左、右子堆, 以确保左子堆的npl不小  
    a->npl = NPL(a->rc) + 1; //更新a的npl (宏隐藏了边界情况)  
    return a; //返回合并后的堆顶  
} //堆的规模须由上层调用者负责更新
```

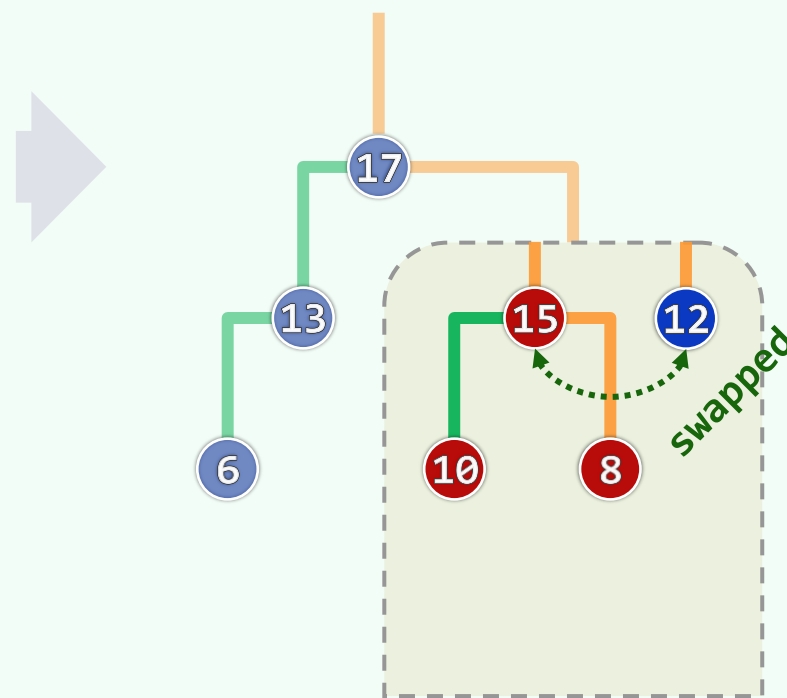
实例 (1/5)



(a)

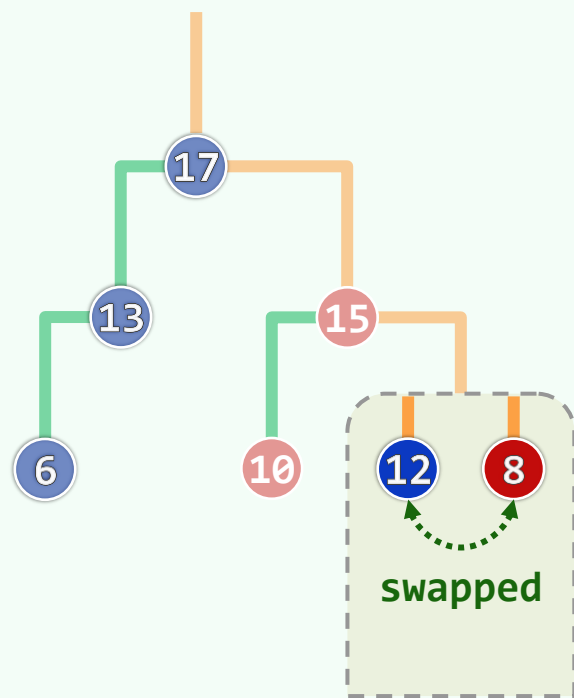


(b)

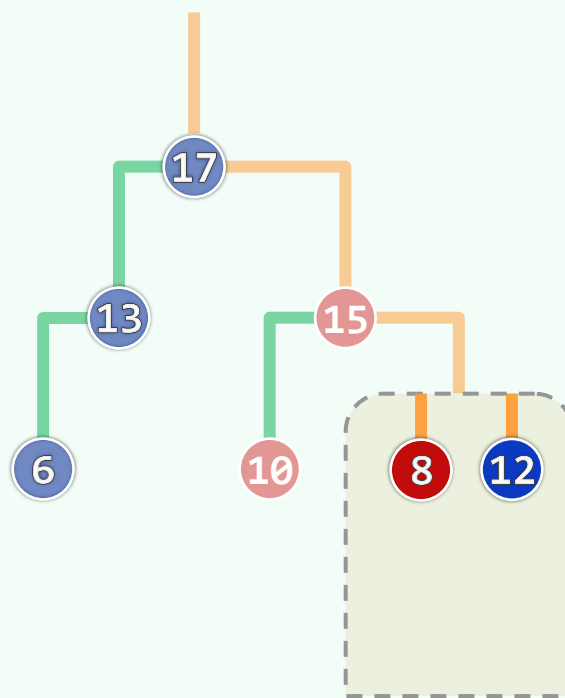


(c)

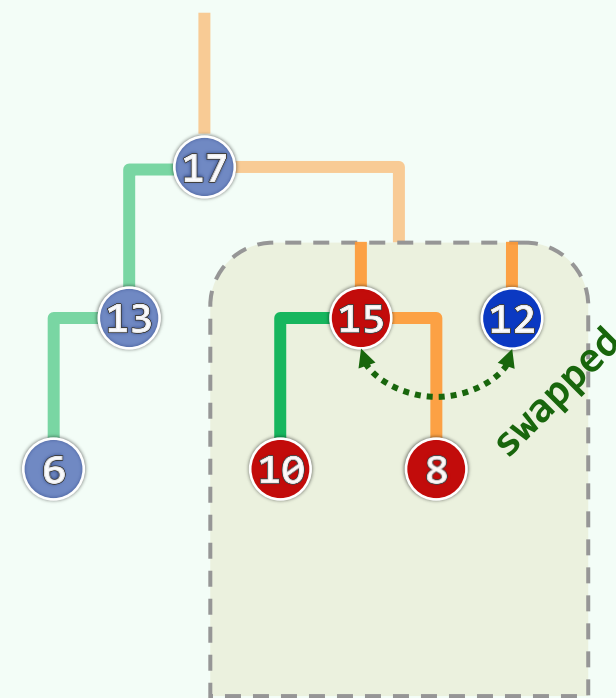
实例 (2/5)



(e)

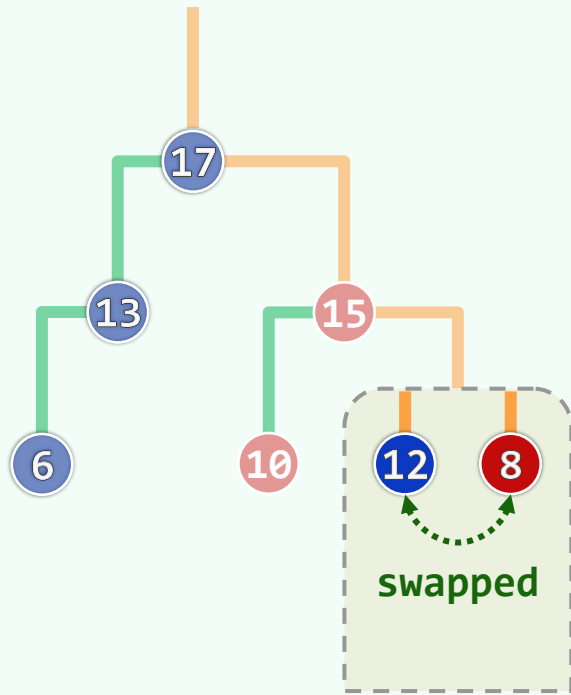


(d)

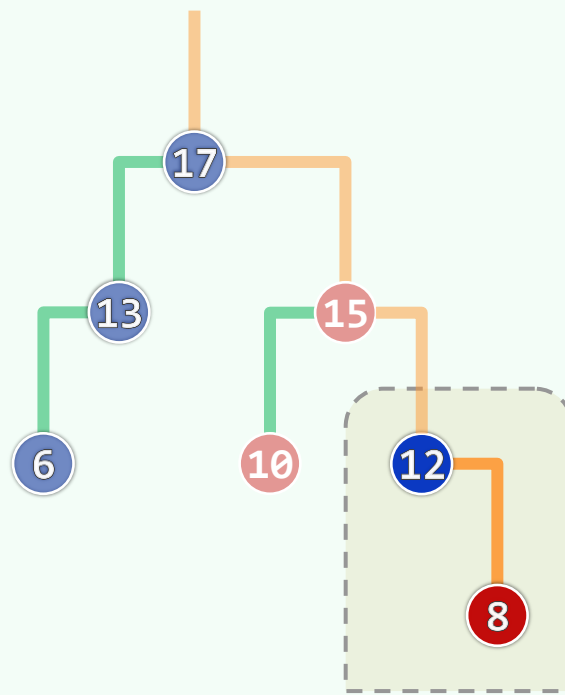


(c)

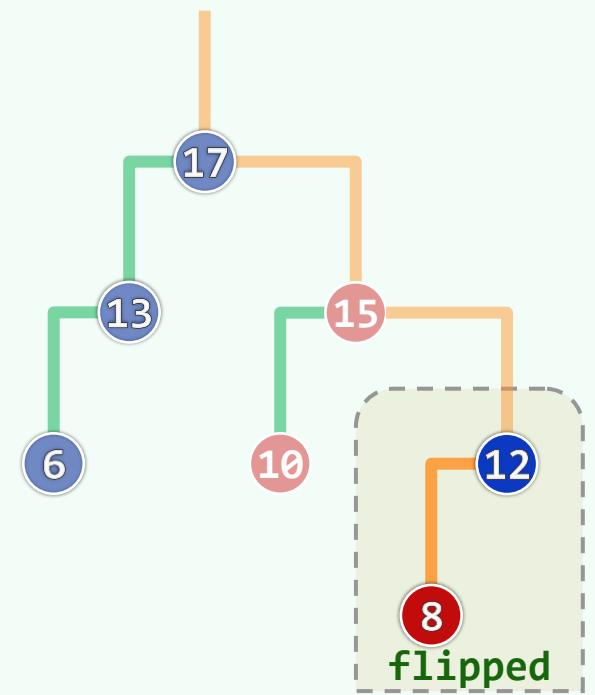
实例 (3/5)



(e)

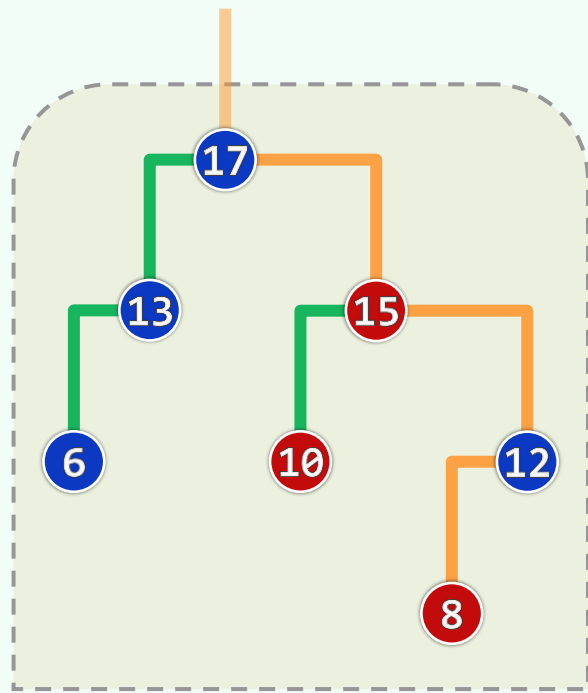


(f)

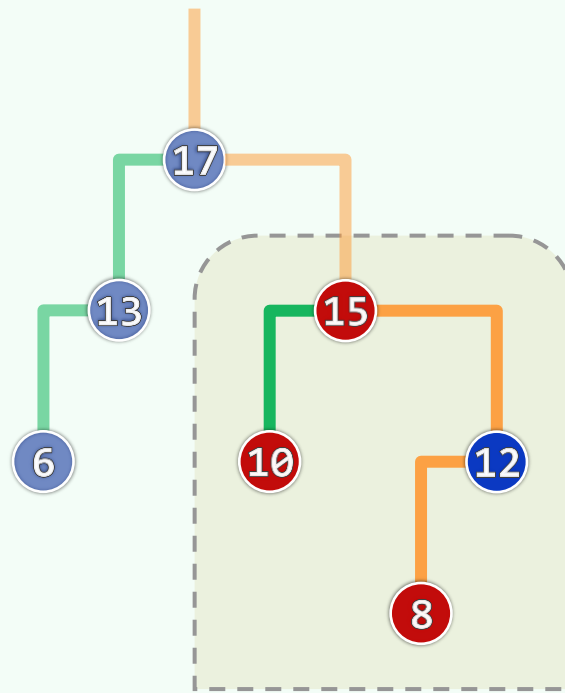


(g)

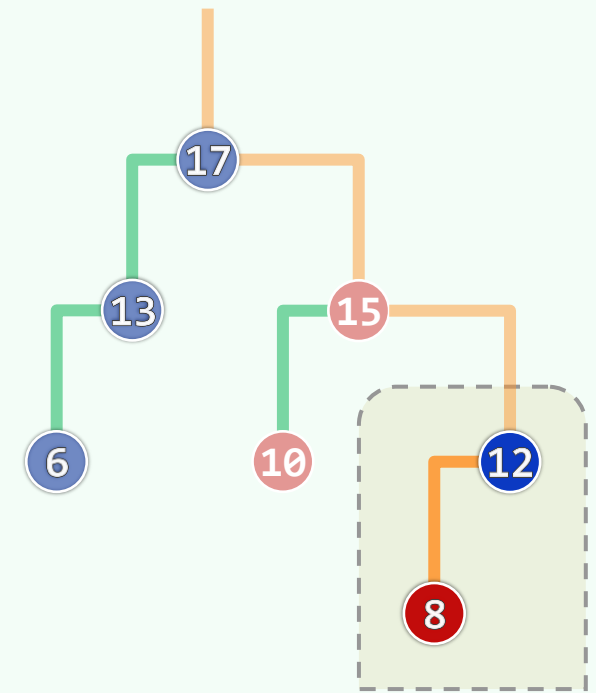
实例 (4/5)



(i)

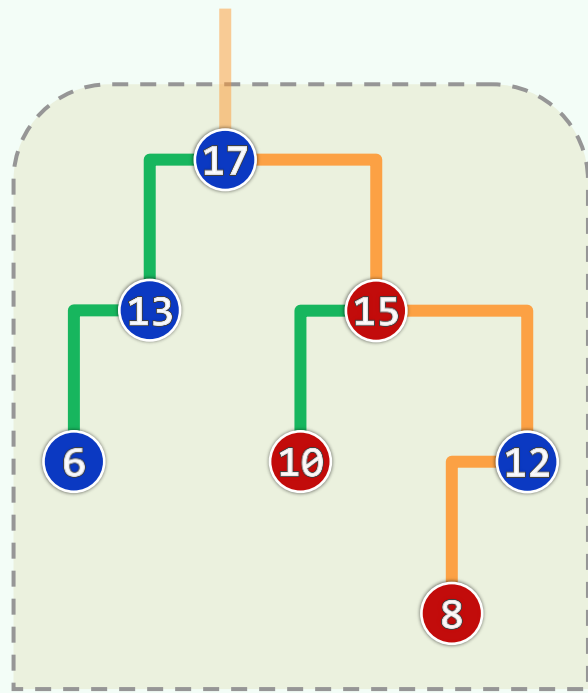


(h)

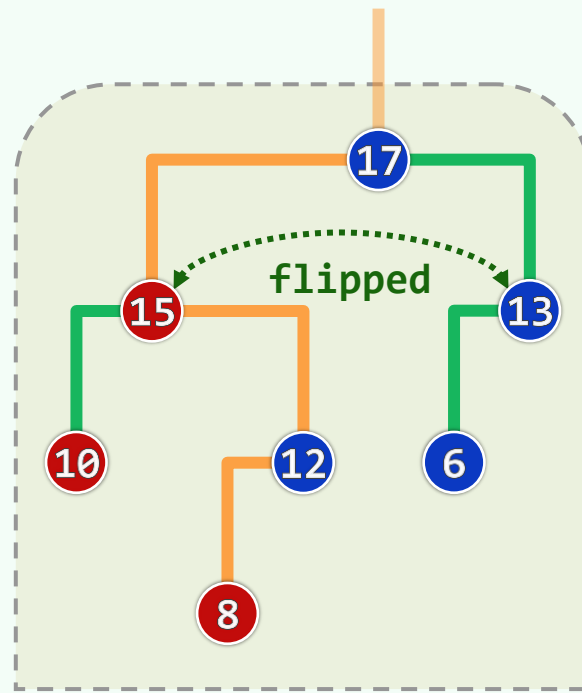


(g)

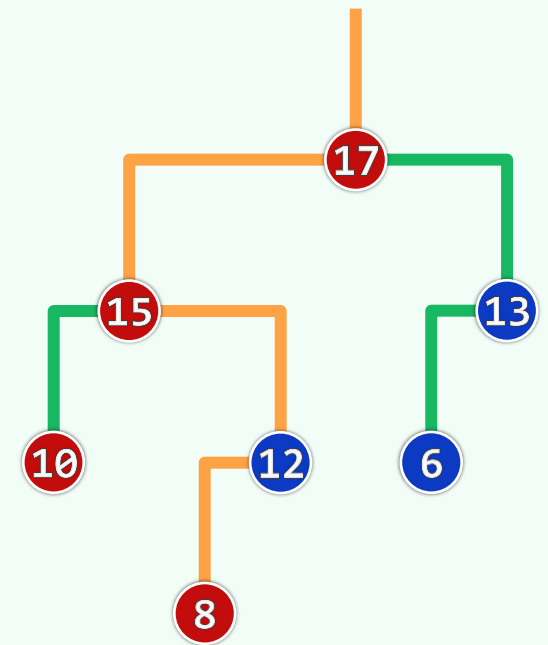
实例 (5/5)



(i)



(j)



(k)

迭代实现

```
template <typename T> BNP<T> merge( BNP<T> a, BNP<T> b ) {  
    if ( !a ) return b; if ( !b ) return a; if ( *a < *b ) swap( a, b );  
    for ( ; a->rc; a = a->rc ) //沿右侧链做二路归并, 直至堆a->rc先于b变空  
        if ( *(a->rc) < *b ) { b->parent = a; swap( a->rc, b); } //接入b  
    (a->rc = b)->parent = a; //直接接入b的残余部分 (必然非空)  
    for ( ; a; a->npl = NPL(a->rc) + 1, b = a, a = a->parent ) //逐层回溯、更新npl  
        if ( NPL(a->lc) < NPL(a->rc) ) swap( a->lc, a->rc ); //确保npl合法  
    return b; //返回合并后的堆顶  
} //堆的规模须由上层调用者负责更新
```

12-E4

优先级队列

左式堆：插入与删除

一招鲜，吃遍天

良田千顷，不如薄技在身

邓俊辉

deng@tsinghua.edu.cn

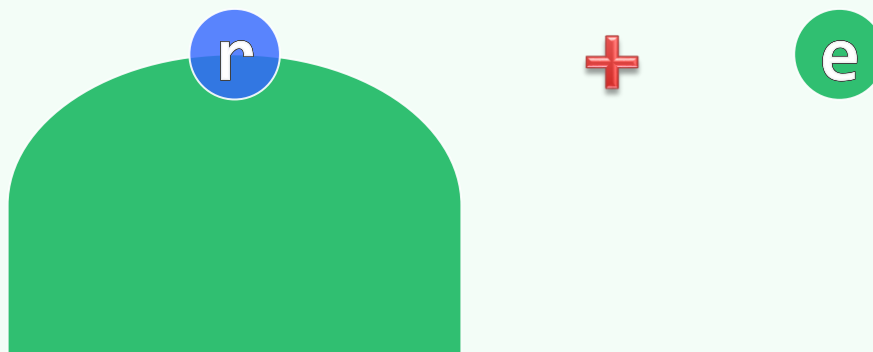
插入

```
template <typename T> void PQ_LeftHeap<T>::insert( T e ) { //O(logn)
```

```
    _root = merge( _root, new BinNode<T>( e, NULL ) );
```

```
    _size++;
```

```
} //insert
```



删除

```
template <typename T> T PQ_LeftHeap<T>::delMax() { //O(logn)

    BNP<T> lHeap = _root->lc; if (lHeap) lHeap->parent = NULL;

    BNP<T> rHeap = _root->rc; if (rHeap) rHeap->parent = NULL;

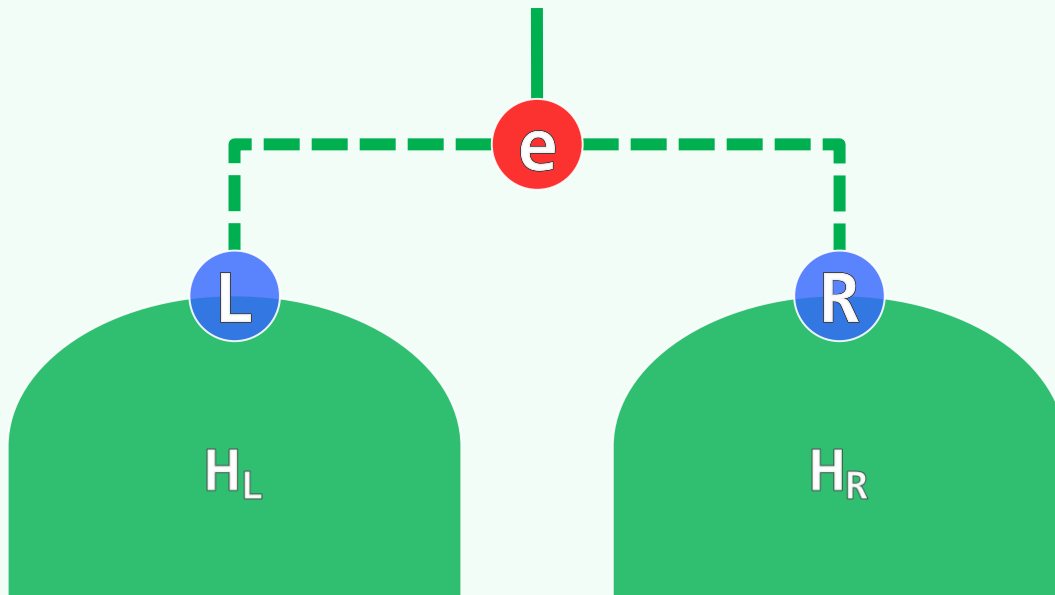
    T e = _root->data;

    delete _root; _size--;

    _root = merge( lHeap, rHeap );

    return e;

} //delMax
```



12-XA

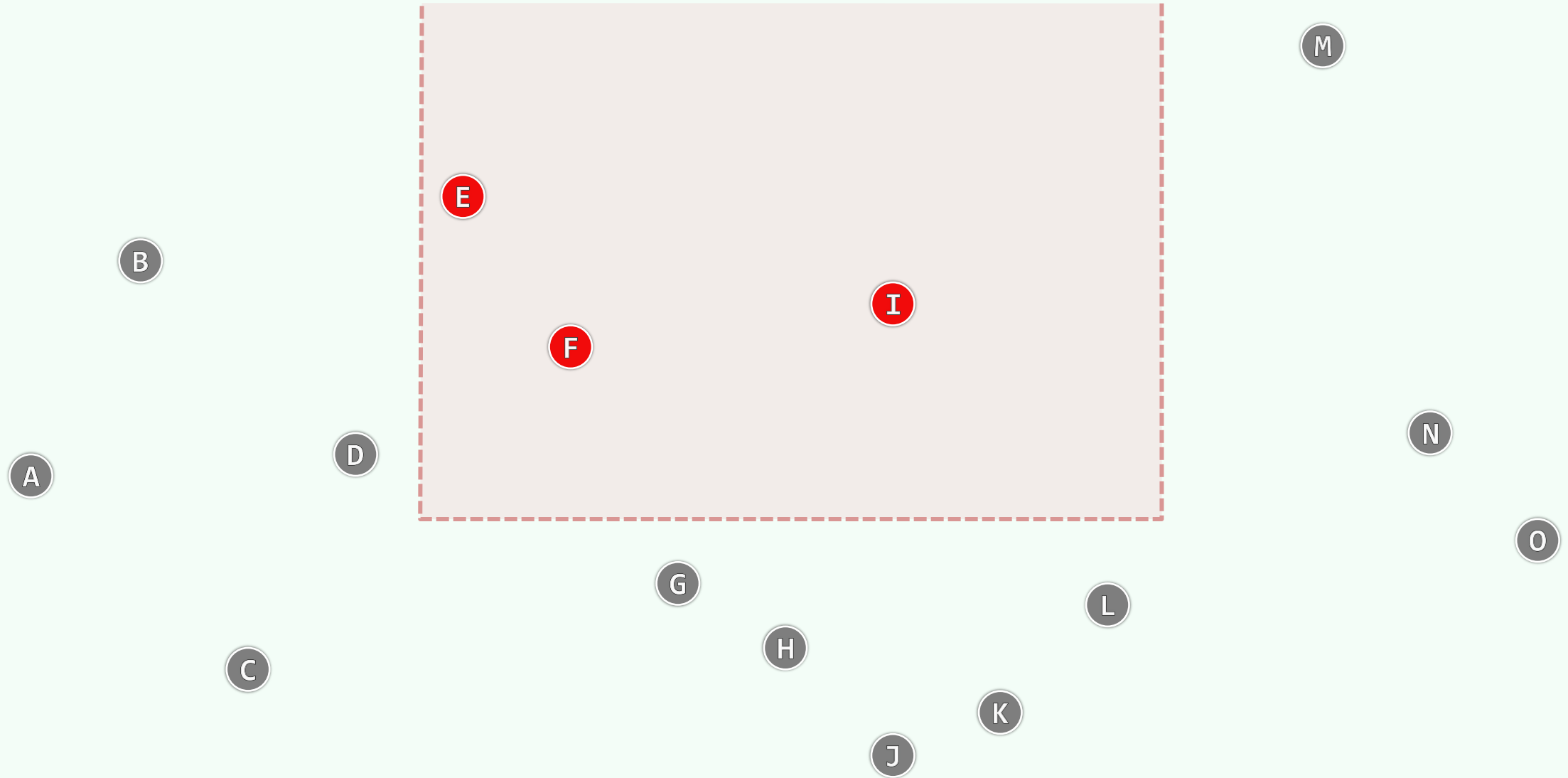
优先级队列

优先级搜索树

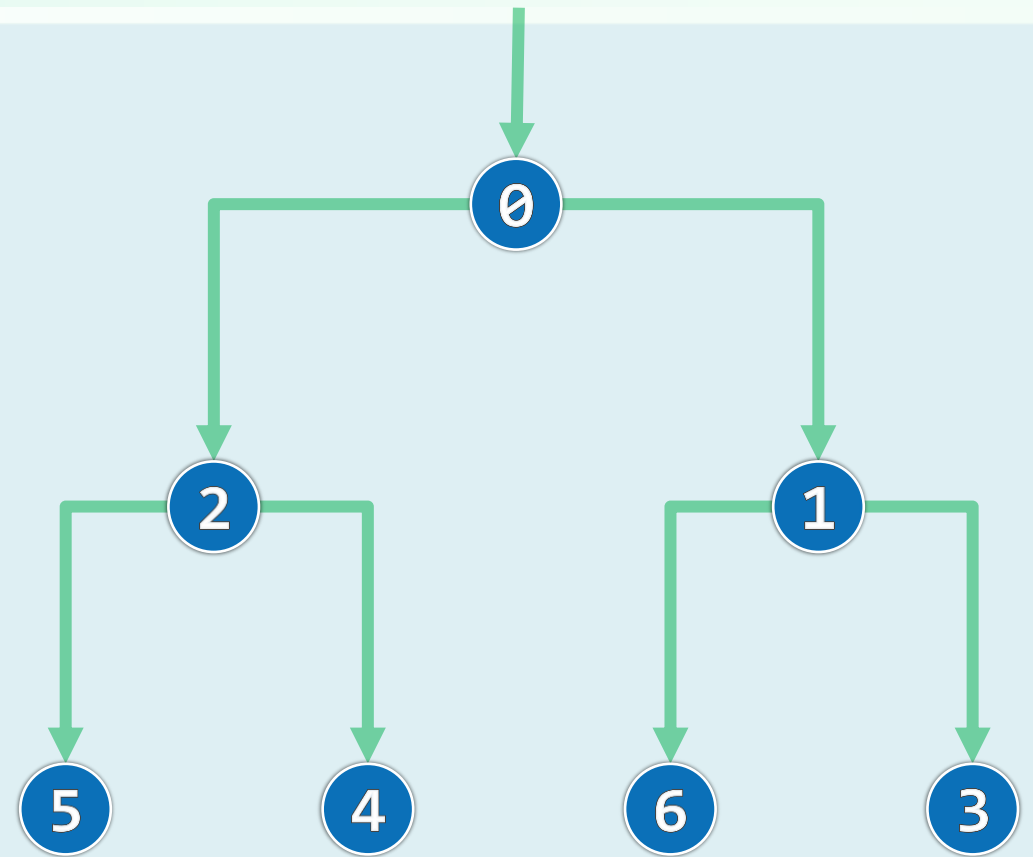
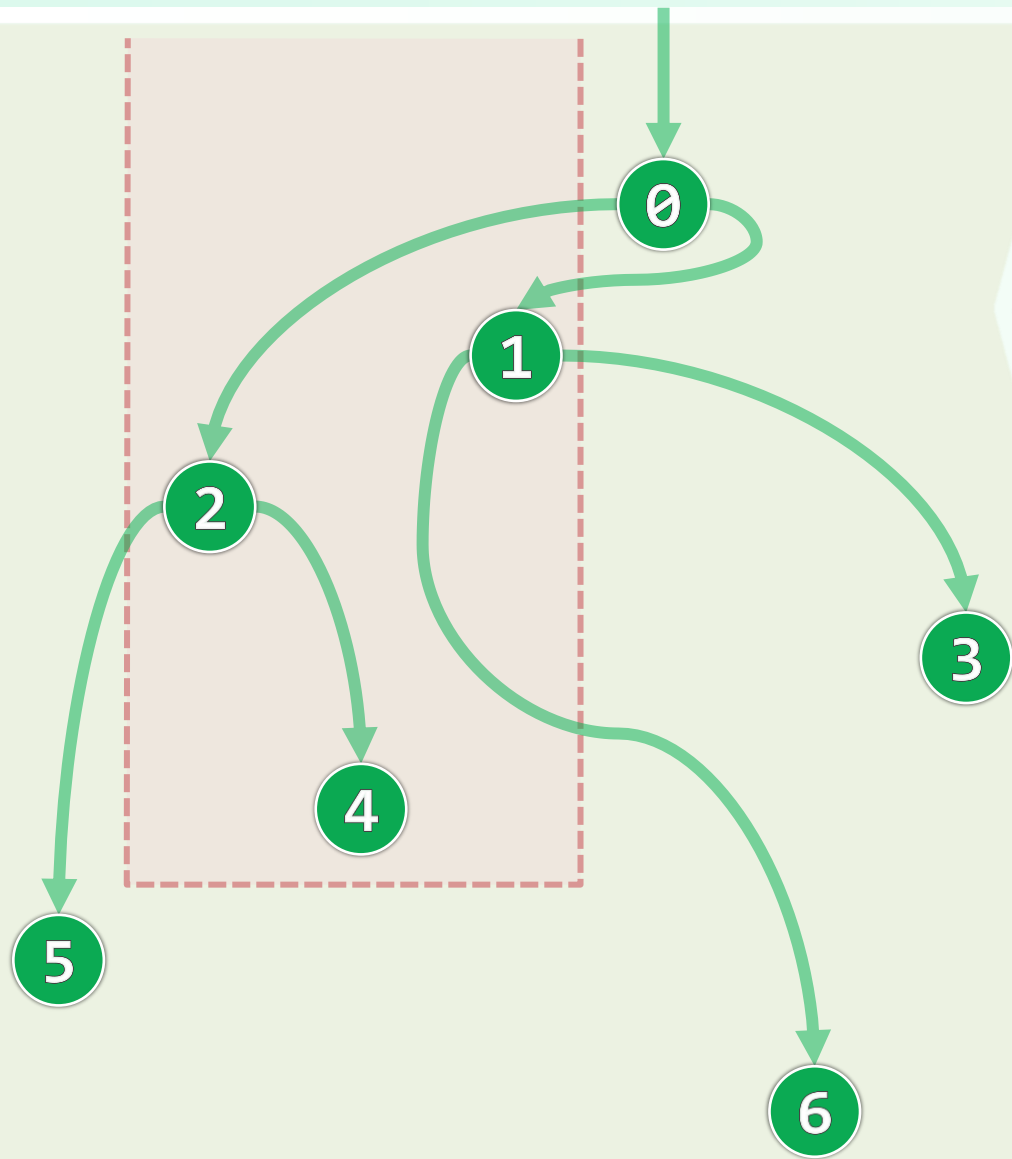
邓俊辉

deng@tsinghua.edu.cn

Grounded Range Query

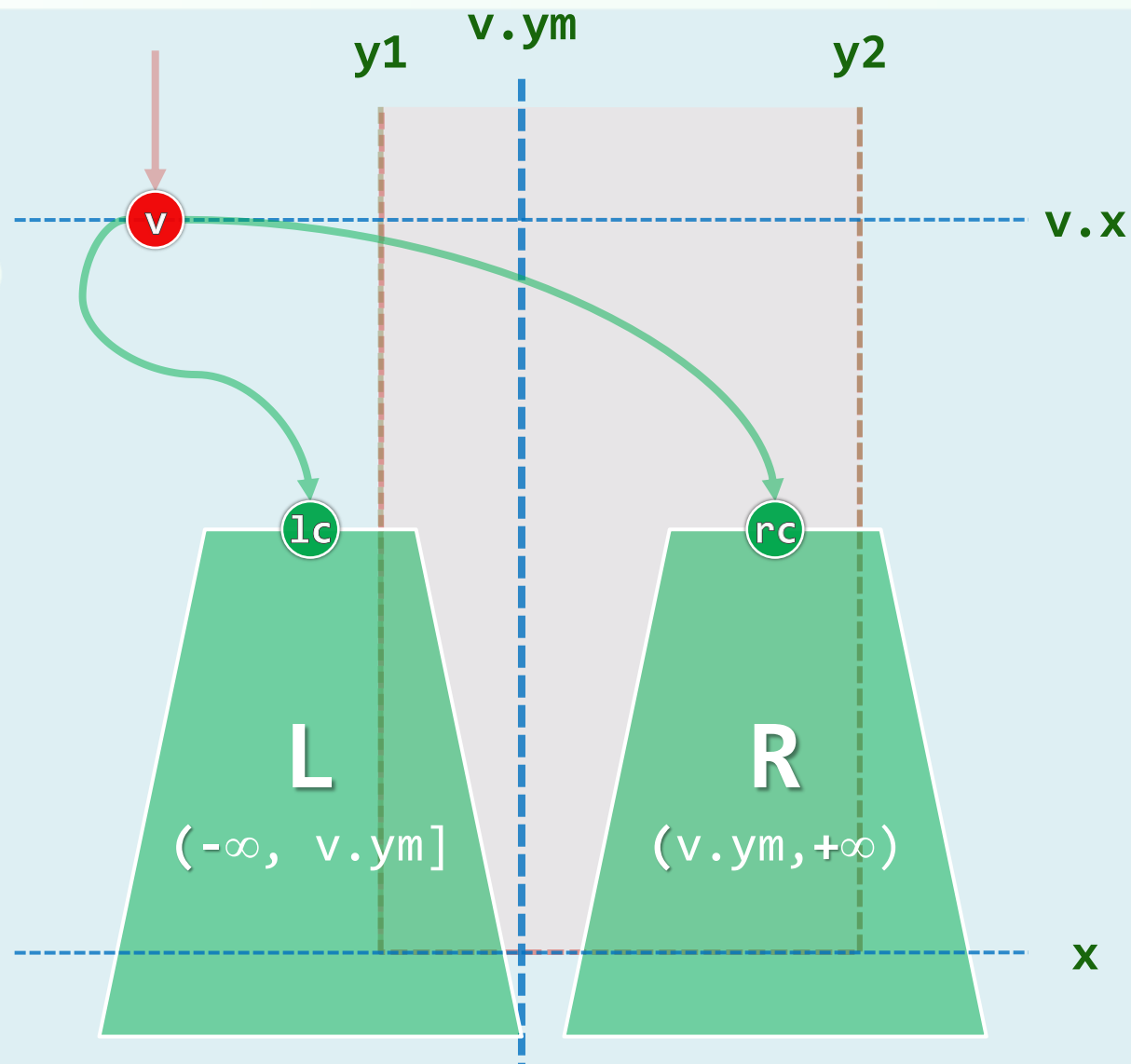


Priority Search Tree = BST + PQ

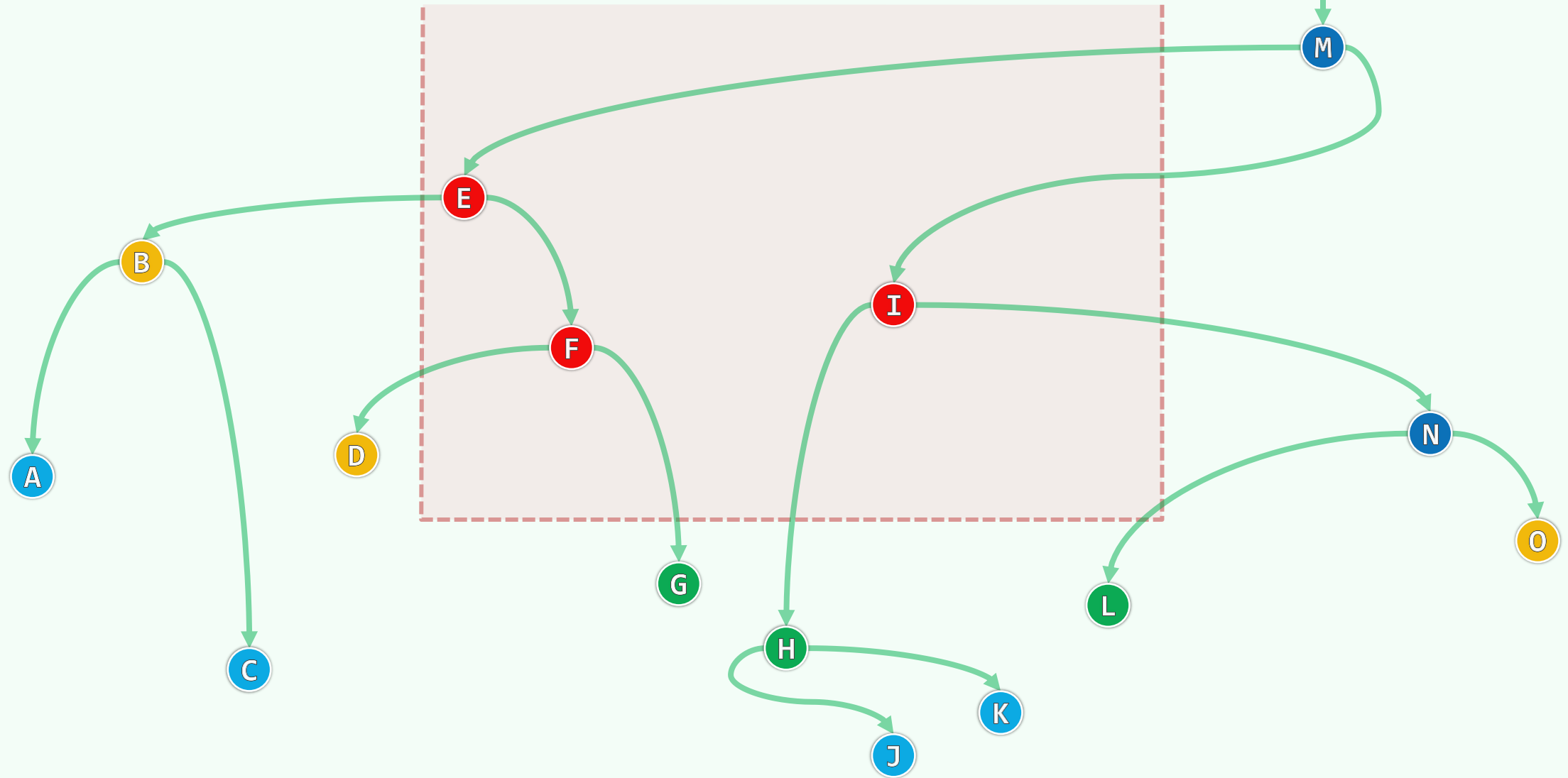


```
queryPST( PSTNode v, int x, int y1, int y2 )
```

```
if ( !v || x < v.x )  
    return //pruned for bad X  
if ( ( y1 < v.y ) && ( v.y < y2 ) )  
    output(v) //accepted  
//else rejected for bad Y  
if ( y1 ≤ v.ym )  
    queryPST( v.lc, x, y1, y2 )  
//else pruned for bad Y  
if ( v.ym < y2 )  
    queryPST( v.rc, x, y1, y2 )  
//else pruned for bad Y
```



Example



Query Time

- P: Pruned with descendants due to bad Y
 - no more time cost
- A: Visited and accepted
 - exactly r = output size
- BY: Visited but rejected due to bad Y
 - no more than 2 for each level
 - altogether $O(\log n)$
- BX: Visited but rejected due to bad X
 - having an A or BY parent
 - no more than $O(r + \log n)$

